



复习课（一）

（词法解析与LL(1)文法）

冯洋
南京大学计算机学院
fengyang@nju.edu.cn



词素



词法单元

我们已有的工具：

- 1.RE
- 2.DFA
- 3.NFA



正则表达式到NFA



- 输入：字母表 Σ 上的一个正则表达式 r
- 输出：一个接受 $L(r)$ 的NFA N
- 基本思想
 - 根据正则表达式的递归定义，按照正则表达式的结构递归地构造出相应的NFA
 - 算法分成两个部分：
 - 基本规则处理 ϵ 和单符号的情况
 - 对于每个正则表达式的运算，建立构造相应NFA的方法



第一题



题目 1 (正则表达式与自动机 (15 = 7 + 8 分))

考虑正则表达式 $r = (a \mid b)^*a^+$ 。

- (1) 请使用 Thompson 构造法构造等价的 NFA。
- (2) 请使用子集构造法构造等价的 DFA。(注：请标明状态之间的对应关系)



正则表达式到自动机



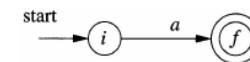
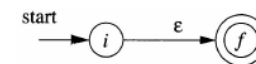
- 正则表达式可以简洁、精确地描述词法单元的模式
- 进行模式匹配时，模拟DFA的执行比模拟NFA更为简单有效
 - ϵ 的模拟，难
 - 不确定性的模拟，难
- 因此，需要将正则表达式转换为DFA
- 步骤：
 - 正则表达式到NFA
 - NFA到DFA



转换算法(1)



- 基本规则：
 - 表达式 ϵ ,
 - 表达式 a ,





转换算法(2)



归纳规则

- 正则表达式 s 和 t 的NFA分别是 $N(s)$ 和 $N(t)$
- $r=s|r$, r 的NFA $N(r)$

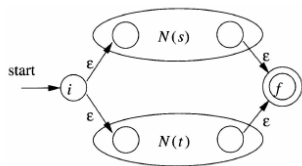


图 3-40 两个正则表达式的并的 NFA



转换算法(3)



归纳规则

- 正则表达式 $r=s^*$, $N(r)$

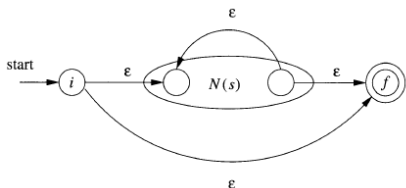


图 3-42 一个正则表达式的闭包的 NFA

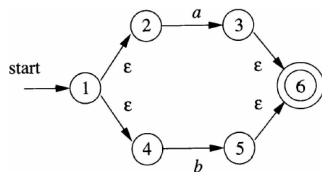
- $r=(s)$, $N(r)=N(s)$



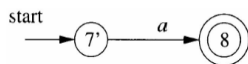
正则表达式到NFA的例子 (2)



(a|b)的NFA



第二个a的NFA



转换算法(2)



归纳规则

- 正则表达式 $r=st$, $N(r)$

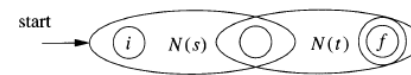


图 3-41 两个正则表达式的连接的 NFA

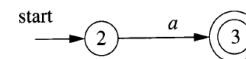


正则表达式到NFA的例子 (1)

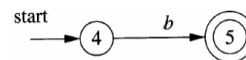


正则表达式 $a|b)^*abb$

第一个a对应的NFA



第一个b对应的NFA



正则表达式到NFA的例子 (3)



(a|b)^*的NFA

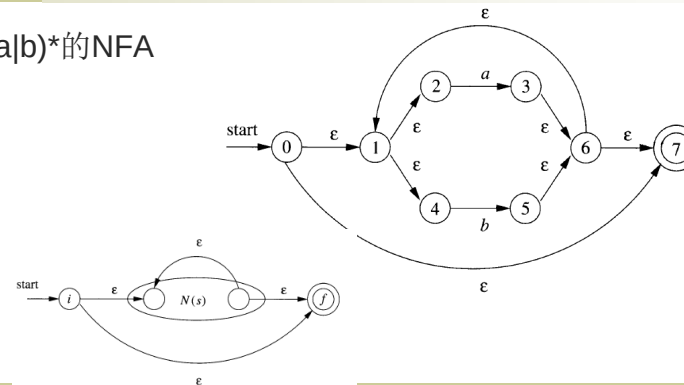


图 3-42 一个正则表达式的闭包的 NFA



NFA转换成DFA - 子集构造法



- 对NFA的模拟往往不如对DFA的模拟直接，除非转换花费更多的时间
- 基本思想：DFA每个状态 $\leftrightarrow$ NFA一个状态集
 - “并行地模拟” NFA在遇到一个给定输入串时可能执行的所有动作
 - 构造得到的DFA的每个状态和NFA的状态子集对应
 - DFA读入 $a_1, a_2, \dots, a_n$ 后到达的状态对应于从NFA开始状态出发沿着 $a_1, a_2, \dots, a_n$ 可能到达的状态集合
- 理论上，最坏情况下DFA的状态个数会是NFA状态个数的指数多个。但是对于大部分应用，NFA和相应的DFA的状态数量大致相同



NFA转换成DFA - 子集构造法



- D的开始状态是 $\epsilon\text{-closure}(s_0)$ ，D的接受状态是所有至少包含了N的一个接受状态的状态集合。

```

一开始,  $\epsilon\text{-closure}(s_0)$  是  $Dstates$  中的唯一状态, 且它未加标记;
while (在  $Dstates$  中有一个未标记状态  $T$ ) {
    给  $T$  加上标记;
    for (每个输入符号  $a$ ) {
         $U = \epsilon\text{-closure}(\text{move}(T, a))$ ;
        if ( $U$  不在  $Dstates$  中)
            将  $U$  加入到  $Dstates$  中, 且不加标记;
         $Dtran[T, a] = U$ ;
    }
}

```

图 3-32 子集构造法



NFA到DFA转换的示例



- A: $\epsilon\text{-closure}(0) = \{0, 1, 2, 4, 7\}$
- B: $Dtran[A, a] = \epsilon\text{-closure}(\text{move}(A, a)) = \epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\}$
- C: $Dtran[A, b] = \epsilon\text{-closure}(\text{move}(A, b)) = \epsilon\text{-closure}(\{5\}) = \{1, 2, 4, 5, 6, 7\}$
- D: $Dtran[B, b] = \epsilon\text{-closure}(\text{move}(B, b)) = \{1, 2, 4, 5, 6, 7, 9\}$
-
- E: $Dtran[D, b] = \epsilon\text{-closure}(\text{move}(D, b)) = \{1, 2, 4, 5, 6, 7, 10\}$

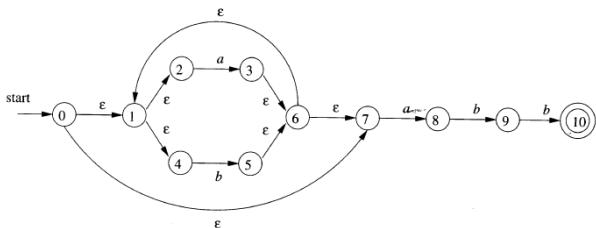


图 3-34 $(a|b)^*abb$ 对应的 NFA N



NFA转换成DFA - 子集构造法



- 输入：一个NFA N
 - 输出：一个接受相同语言的DFA D
- s 表示 N 中的单个状态， T 代表 N 的一个状态集

操作	描述
$\epsilon\text{-closure}(s)$	能够从NFA的状态 s 开始只通过 ϵ 转换到达的 NFA 状态集合
$\epsilon\text{-closure}(T)$	能够从 T 中某个NFA状态 s 开始只通过 ϵ 转换到达的 NFA 状态集合, 即 $\cup_{s \in T} \epsilon\text{-closure}(s)$.
$\text{move}(T, a)$	能够从 T 中某个状态 s 出发通过标号为 a 的转换到达的 NFA 状态的集合

图 3-31 NFA 状态集上的操作



NFA转换成DFA - 子集构造法



- 对NFA的任何状态集合 $\epsilon\text{-closure}(T)$ 的计算
- 一个图搜索过程

```

将  $T$  的所有状态压入  $stack$  中;
将  $\epsilon\text{-closure}(T)$  初始化为  $T$ ;
while (  $stack$  非空 ) {
    将栈顶元素  $t$  弹出栈中;
    for (每个满足如下条件的  $u$ : 从  $t$  出发有一个标号为  $a$  的转换到达状态  $u$ )
        if ( $u$  不在  $\epsilon\text{-closure}(T)$  中) {
            将  $u$  加入到  $\epsilon\text{-closure}(T)$  中;
            将  $u$  压入栈中;
        }
}

```

图 3-33 计算 $\epsilon\text{-closure}(T)$



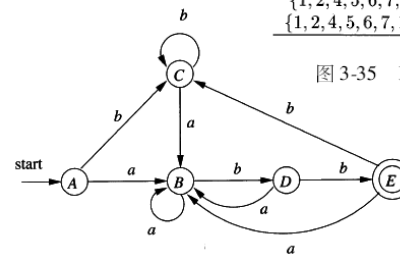
NFA到DFA转换的示例



- 开始状态：A
- 接受状态：E

NFA 状态	DFA 状态	a	b
$\{0, 1, 2, 4, 7\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8\}$	B	B	D
$\{1, 2, 4, 5, 6, 7\}$	C	B	C
$\{1, 2, 4, 5, 6, 7, 9\}$	D	B	E
$\{1, 2, 4, 5, 6, 7, 10\}$	E	B	C

图 3-35 DFA D 的转换表 $Dtran$





对NFA的运行进行模拟:子集构造法

- 输入: 一个以文件结束符eof结尾的输入串 x , 一个NFA N , 其开始状态是 S_0 , 接受状态集为 F , 转换函数为 $move$
- 输出: 如果 N 接受 x , 返回yes, 否则返回no

```

1)  $S = \epsilon\text{-closure}(s_0);$ 
2)  $c = \text{nextChar}();$ 
3) while (  $c \neq \text{eof}$  ) {
4)    $S = \epsilon\text{-closure}(\text{move}(S, c));$ 
5)    $c = \text{nextChar}();$ 
6) }
7) if (  $S \cap F \neq \emptyset$  ) return "yes";
8) else return "no";

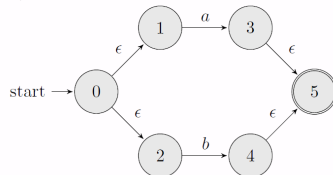
```

图 3-37 模拟的一个 NFA

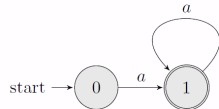


第一题

$a \mid b$:



a^+ :



第一题

$$A = \epsilon\text{-CLOSURE}(\{0\}) = \{0, 1, 2, 3, 7, 8\}$$

$$B = \epsilon\text{-CLOSURE}(\text{MOVE}(A, a)) = \epsilon\text{-CLOSURE}(\{4, 9\}) = \{1, 2, 3, 4, 6, 7, 8, 9\}$$

$$C = \epsilon\text{-CLOSURE}(\text{MOVE}(A, b)) = \epsilon\text{-CLOSURE}(\{5\}) = \{1, 2, 3, 5, 6, 7, 8\}$$

NFA	DFA	a	b
$\{0, 1, 2, 3, 7, 8\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8, 9\}$	B	B	C
$\{1, 2, 3, 5, 6, 7, 8\}$	C	B	C



第一题

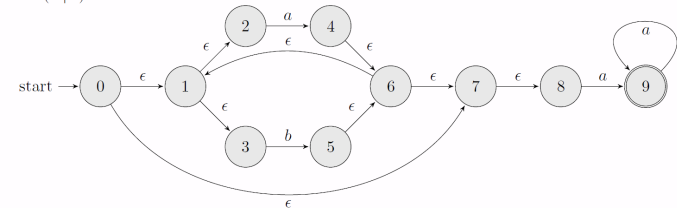
题目 1 (正则表达式与自动机 (15 = 7 + 8 分))

考虑正则表达式 $r = (a \mid b)^* a^+$.

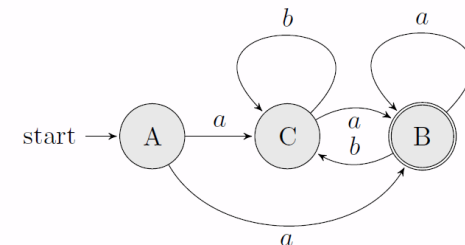
(1) 请使用 Thompson 构造法构造等价的 NFA。

(2) 请使用子集构造法构造等价的 DFA。(注: 请标明状态之间的对应关系)

$r = (a \mid b)^* a^+$:



第一题





第二题

考虑一个密码强度验证系统，需要识别符合以下规则的密码字符串：

- 标识符由字母 a、b 和数字 0、1 组成
- 标识符必须以字母开头
- 标识符必须以 abb 结尾
- 标识符中间部分不能包含连续的两个数字（即不能出现 00、01、10、11）

例如：

合法标识符：aabb, babb, ababb, a0aabb, b1babb

非法标识符：0abb（以数字开头），ab（不以abb结尾），a00abb（包含连续数字），a01abb（包含连续数字）

请完成以下任务：

- 写出描述这类字符串的正则表达式
- 使用Thompson构造法构造与该正则表达式等价的非确定有限自动机（NFA）
- 使用子集构造法将NFA转换为确定有限自动机（DFA）
- 最小化得到的DFA



第三题

题目 2 (LL 语法分析 (20 = 8 + 8 + 4 分))

考虑文法 G ：

- $$\begin{aligned} S &\rightarrow cS & (1) \\ S &\rightarrow AB & (2) \\ A &\rightarrow aA & (3) \\ A &\rightarrow \epsilon & (4) \\ B &\rightarrow bBa & (5) \\ B &\rightarrow d & (6) \end{aligned}$$

(1) 请为文法 G 计算必要的 First 集合与 Follow 集合；

(2) 请为文法 G 构造预测分析表；

(3) 请问文法 G 是否是 LL(1) 文法，并说明理由。



立即左递归的消除

- 假设非终结符号A存在立即左递归的情形，假设以A为左部的规则有：

$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

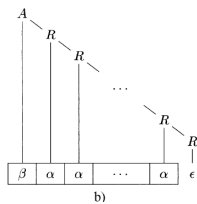
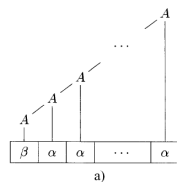
首先对A产生式**分组**(所有 α_i 不等于 ϵ ， β_i 不以A开头)

可以**替换**为

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon$

- 由A生成的串总是以某个 β_i 开头，然后跟上零个或者多个 α_i 的重复



第二题

考虑一个密码强度验证系统，需要识别符合以下规则的密码字符串：

- 标识符由字母 a、b 和数字 0、1 组成
- 标识符必须以字母开头
- 标识符必须以 abb 结尾
- 标识符中间部分不能包含连续的两个数字（即不能出现 00、01、10、11）

例如：

合法标识符：aabb, babb, ababb, a0aabb, b1babb

非法标识符：0abb（以数字开头），ab（不以abb结尾），a00abb（包含连续数字），a01abb（包含连续数字）

1. 以字母开头：(a|b)
 2. 以 abb 结尾：abb
 3. 中间部分不能包含连续数字：中间可以是字母，或者数字后必须跟字母
- 正则表达式为：

$(a|b)((a|b)|0(a|b)|1(a|b))*abb$



消除左递归

■ 左递归的定义

- 文法中一个非终结符号A使得对某个串 α ，存在一个推导 $A \Rightarrow A\alpha$ ，则称这个文法是左递归的

■ 立即左递归

- 如果存在 $A \rightarrow A\alpha$ ，则称为立即左递归

■ 为什么要消除左递归？

- 自顶向下的语法分析技术不能处理左递归的文法

■ 立即左递归的消除实例：



消除立即左递归

■ $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

■ 分组

$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

■ 替换

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon$



立即左递归消除示例



$$\begin{array}{lcl}
 E \rightarrow E + T \mid T & \longrightarrow & E \rightarrow TE' \\
 T \rightarrow T * F \mid F & & E' \rightarrow + TE' \mid \epsilon \\
 F \rightarrow (E) \mid \text{id} & & T \rightarrow FT' \\
 & & T' \rightarrow * FT' \mid \epsilon \\
 & & F \rightarrow (E) \mid \text{id}
 \end{array}$$



消除算法



- 输入：没有环 A_i A_i 和 $\overset{\cdot}{A_i} \rightarrow \epsilon$
- 输出：一个等价的无左递归的文法
- 算法原理：
 - 给非终结符号**排序**， $A_1, A_2, \dots, A_n$
 - 如果只有 $A_i \Rightarrow A_j$ ($i < j$)，则不会有左递归
 - 如果发现 $A_i \Rightarrow A_j$ ($i > j$)，**代入** A_j 的当前产生式，若替换后有 A_i 的直接左递归，再**消除**



通用的左递归消除方法（续）



1. 内层循环的每一次迭代保证了 A_i 的产生式的右部首符号都比 A_j 更加靠后
2. 当内层循环结束时， A_i 的产生式的右部首符号不先于 A_i
3. 消除立即左递归就保证了 A_i 的产生式的右部首符号必然比 A_i 后。（不能有环和 ϵ 产生式）
4. 当外层循环结束时，所有的产生式都是如此。使用这些产生式当然不会产生左递归的情形



消除多步左递归



- 消除立即左递归的方法并不能消除因为两步或多步推导而产生的左递归
- 文法： $S \rightarrow Aa \mid b, A \rightarrow Ac \mid Sd \mid \epsilon$
- $S \Rightarrow Aa \Rightarrow Sda$
- 如何消除？



通用的左递归消除方法



- 输入：没有环和 ϵ 产生式的文法 G
- 输出：等价的无左递归的文法
- 步骤：
 - 将文法的非终结符号任意**排序**为 $A_1, A_2, \dots, A_n$
 - for $i=1$ to n do {
 - for $j=1$ to $i-1$ do {
 - {
 - 将形如 $A_i \rightarrow A_j \gamma$ 的产生式**替换**为 $A_i \rightarrow \delta_1 \gamma \mid \dots \mid \delta_k \gamma$,
 - 其中 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ 是以 A_j 为左部的所有产生式；
 - }
 - 消除** A_i 的立即左递归；



消除算法（示例）



- $S \rightarrow Aa \mid b, A \rightarrow Ac \mid Sd \mid \epsilon$
- **排序**： $S \ A$
- **替换**：
 - $i=1$ ，没有处理
 - $i=2$ ，替换 $A \rightarrow Sd$ 中的 S ，得到 $A \rightarrow Ac \mid Aad \mid bd \mid \epsilon$
- **消除**立即左递归

$$\begin{array}{l}
 S \rightarrow Aa \mid b \\
 A \rightarrow b d A' \mid A' \\
 A' \rightarrow c A' \mid a d A' \mid \epsilon
 \end{array}$$



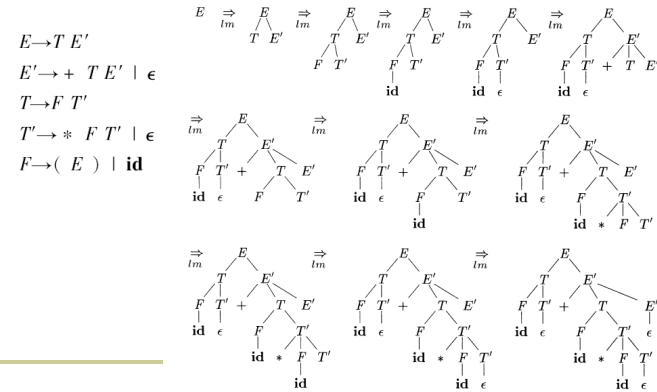
自顶向下分析技术



- 自顶向下分析可以被看作是为输入串构造语法分析树的问题，也可以看作一个寻找输入串的**最左推导**的过程
- 问题
 - 在推导的每一步，对非终结符号A，应用哪个产生式，以可能产生于输入串相匹配的终结符号串



id+id*id的自顶向下分析



自顶向下语法分析



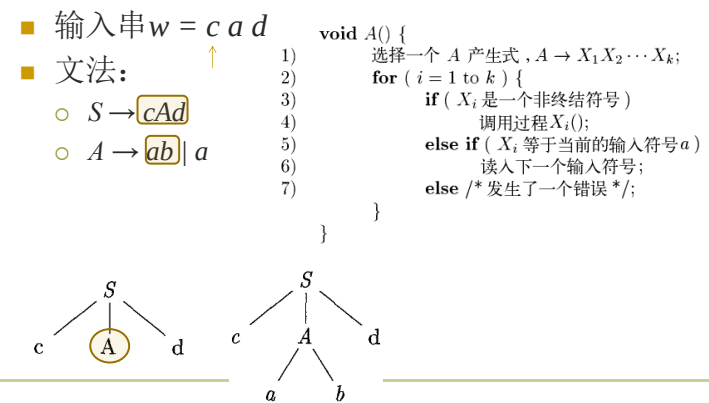
- 问题：对于非终结符号A，选择哪一个产生式
- 一种**通用**的**递归下降**分析框架
 - 由一组过程组成，每个非终结符号对应一个过程
 - 程序的执行从开始符号对应的过程开始
 - 每个过程的功能是：选择一个产生式体，扫描相应的句子。若遇到非终结符号，调用该符号对应的过程

```

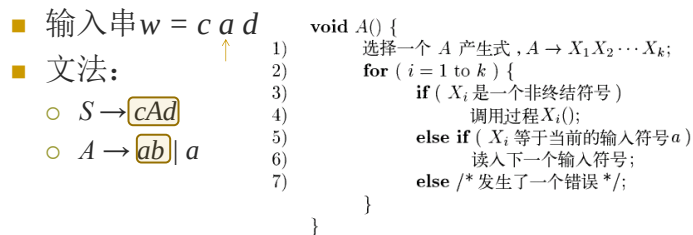
void A() {
1)  选择一个 A 产生式,  $A \rightarrow X_1 X_2 \dots X_k$ ;
2)  for (  $i = 1$  to  $k$  ) {
3)      if (  $X_i$  是一个非终结符号 )
4)          调用过程  $X_i()$ ;
5)      else if (  $X_i$  等于当前的输入符号  $a$  )
6)          读入下一个输入符号;
7)      else /* 发生了一个错误 */;
    }
}
    
```



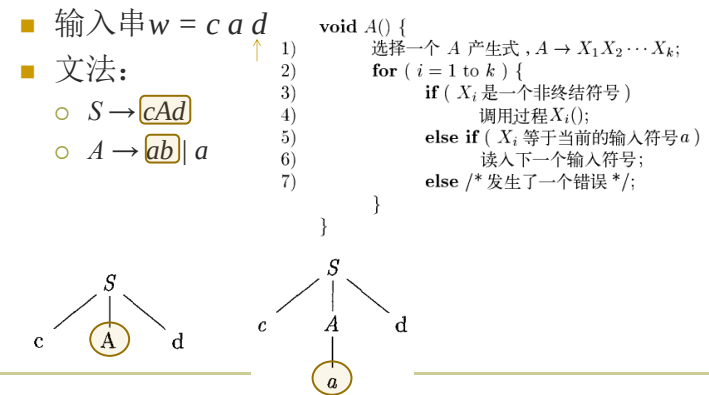
递归下降分析过程示例



递归下降分析过程示例



递归下降分析过程示例





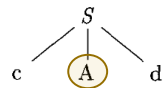
递归下降分析过程示例

- 输入串 $w = c a d$

- 文法:

- $S \rightarrow cAd$
- $A \rightarrow ab \mid a$

```
void A() {
1) 选择一个 A 产生式,  $A \rightarrow X_1 X_2 \dots X_k$ ;
2)  for (  $i = 1$  to  $k$  ) {
3)      if (  $X_i$  是一个非终结符号 )
4)          调用过程  $X_i()$ ;
5)      else if (  $X_i$  等于当前的输入符号  $a$  )
6)          读入下一个输入符号;
7)      else /* 发生了一个错误 */;
    }
}
```



预测分析法简介

- 试图从开始符号推导出输入符号串
- 以开始符号作为初始的当前句型
- 每次为最左边的非终结符号选择适当的产生式
 - 通过查看下一个输入符号来选择这个产生式
 - 有多个可能的产生式时预测分析法无能为力
- 比如文法: $E \rightarrow +EE \mid -EE \mid id$; 输入为 $+ id - id id$
- 当两个产生式具有相同的前缀时无法预测
 - 文法: $stmt \rightarrow if\ expr\ then\ stmt\ else\ stmt \mid if\ expr\ then\ stmt$
 - 输入: $if\ a\ then\ \dots$
 - 新文法: $stmt \rightarrow if\ expr\ then\ stmt\ elsePart$
 - $elsePart \rightarrow else\ stmt \mid \epsilon$
- 需要提取公因子



预测分析技术 (续)

- 例2: 文法 $G(S)$ 为:

$S \rightarrow Ap$
 $S \rightarrow Bq$
 $A \rightarrow a \mid cA$
 $B \rightarrow b \mid dB$

- 输入 $W = ccap$



递归下降过程示例(续)

- 可能需要回溯, 或者说, 可能需要重复扫描输入
- “在回到A时, 我们必须把输入指针重新设置到位置2, 即我们第一次尝试展开A时该指针指向的位置。这意味着A的过程必须将输入指针存放在一个局部变量中。”
- 如果文法中存在左递归.....
- 回溯 (穷试 ☹) 显然是笨办法



预测分析技术

消除二义性
 消除左递归
 提取左公因子

- 一种确定性的、无回溯的分析技术
- 在每一步选择正确的产生式

- 例1: 文法 $G(S)$:

$S \rightarrow pA$
 $S \rightarrow qB$
 $A \rightarrow cAd$
 $A \rightarrow a$

- 输入串 $W = pccadd$



预测分析技术 (续)

- 通过在输入中向前看固定多个符号来选择正确的产生式
- 通常情况下, 我们只需要向前看一个符号
 - 递归下降分析一般只适合于每个子表达式的第一个终结符能够为产生式选择提供足够信息的那些文法
- 给出两个与文法相关的两个函数
 - FIRST
 - FOLLOW
- 基于上述两个函数, 可以根据下一个输入符号来选择应用哪个产生式



FIRST和FOLLOW (1)

很重要

- 在自顶向下的分析技术中，通常使用向前看几个符号来唯一地确定产生式（这里假定只看一个符号）
- 当前句型是 $xA\beta$ ，而输入是 $xa\dots$ 。那么选择产生式 $A \rightarrow \alpha$ 的必要条件是下列之一：
 - $A \xRightarrow{*} \epsilon$ ，且 β 以 a 开头；也就是说在某个句型中 a 跟在 A 之后；
 - $A \xRightarrow{*} a\dots$ ；
 - 如果按照这两个条件选择时能够保证唯一性，那么我们就可以避免回溯。
- 因此，我们定义FIRST和FOLLOW



FIRST和FOLLOW (2)

FIRST(α):

- 可以从 α 推导得到的串的首符号的集合；
- 如果 $\alpha \xRightarrow{*} \epsilon$ ，那么 ϵ 也在FIRST(α)中；

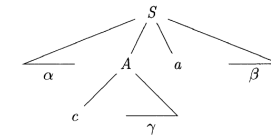


图 4-15 终结符号 c 在 FIRST(A) 中且 a 在 FOLLOW(A) 中

FOLLOW(A):

- 可能在某些句型中紧跟在 A 右边的终结符号的集合。



FIRST(α)

- 定义：可从 α 推导得到的串的首符号的集合，其中 α 是任意的文法符号串。如果 $\alpha \xRightarrow{*} \epsilon$ ，那么 ϵ 也在FIRST(α)中
- FIRST函数的意义
 - 如果两个 A 产生式 $A \rightarrow \alpha \mid \beta$ ，其中First(α)和First(β)是不相交的集合。下一个输入符号是 a ，若 $a \in \text{First}(\alpha)$ ，则选择 $A \rightarrow \alpha$ ，若 $a \in \text{First}(\beta)$ ，则选择 $A \rightarrow \beta$
 - 用于预测产生式的选择



First的计算

- 对于文法符号 X 的FIRST(X)，通过不断应用下列规则，直到没有新的终结符号或者 ϵ 可以加入到任何FIRST集合为止
 - 如果 X 是终结符号，那么FIRST(X)= $\{X\}$
 - 如果 X 是非终结符号，且有规则 $X \rightarrow a\dots$ ，那么将 a 添加到FIRST(X)中；如果 $X \rightarrow \epsilon$ ，那么 ϵ 也在FIRST(X)中
 - 对于规则 $X \rightarrow Y_1 Y_2 \dots Y_n$ ，把FIRST(Y_1)中的非 ϵ 符号添加到FIRST(X)中。如果 ϵ 在FIRST(Y_1)中，把FIRST(Y_2)中的非 ϵ 符号添加到FIRST(X)中...；如果 ϵ 在FIRST(Y_n)中，把 ϵ 添加到FIRST(X)中



First的计算（续）

- 对于文法符号串 $X_1 X_2 \dots X_n$ 的First集合
 - 向First($X_1 X_2 \dots X_n$)加入First(X_1)中所有的非 ϵ 符号。
 - 如果 ϵ 在First(X_1)中，再加入First(X_2)中的所有非 ϵ 符号。如果 ϵ 在First(X_1)和First(X_2)中，再加入First(X_3)中的所有非 ϵ 符号。依次类推。
 - 如果对所有的 i （1到 n ）， ϵ 都在First(X_i)中，则 ϵ 加入First($X_1 X_2 \dots X_n$)中。



First的计算示例

$E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \epsilon$
 $T \rightarrow F T'$
 $T' \rightarrow * F T' \mid \epsilon$
 $F \rightarrow (E) \mid \text{id}$

- First(F)=First(T)=First(E)= $\{ (, \text{id} \}$
- First(E')= $\{ +, \epsilon \}$
- First(T')= $\{ *, \epsilon \}$
- First(TE')=First(T)= $\{ (, \text{id} \}$
- First($+TE'$)= $\{ + \}$
-



一个实例

■ 文法G[S], 输入bcd

- $S \rightarrow AB \mid CD$
- $A \rightarrow aD \mid \varepsilon$
- $C \rightarrow cD$
- $B \rightarrow bC$
- $D \rightarrow d$
- $\text{First}(S) = \{a, b, c\}$
- $\text{First}(A) = \{a, \varepsilon\}$
- $\text{First}(B) = \{b\}$
- $\text{First}(C) = \{c\}$
- $\text{First}(D) = \{d\}$



FOLLOW计算

- 计算各个非终结符号A的FOLLOW(A)集合，不断应用下列规则，直到没有新的终结符号可以被加入到任意FOLLOW集合中
 - 将\$放入FOLLOW(S)，S是开始符号，而\$是输入串的结束标记
 - 如果存在产生式 $A \rightarrow \alpha B \beta$ ，那么 $\text{First}(\beta)$ 中除 ε 之外的所有符号都在 $\text{Follow}(B)$ 中
 - 如果存在一个产生式 $A \rightarrow \alpha B$ ，或存在产生式 $A \rightarrow \alpha B \beta$ 且 $\text{First}(\beta)$ 包含 ε ，那么 $\text{Follow}(A)$ 中的所有符号都在 $\text{Follow}(B)$ 中



课堂练习

- 有如下文法：
 - $E \rightarrow id \ X$
 - $X \rightarrow \varepsilon \mid (A) \mid [E]$
 - $A \rightarrow EY$
 - $Y \rightarrow \varepsilon \mid ;A$
- 给出各个非终结符号的First集和Follow集合。



FOLLOW函数

文法G[S], 输入bcd

$S \rightarrow AB \mid CD$
 $A \rightarrow aD \mid \varepsilon$
 $C \rightarrow cD$
 $B \rightarrow bC$
 $D \rightarrow d$

- 对于非终结符号A，FOLLOW(A)定义为可能在某些句型中紧跟在A右边的终结符号的集合
 - 例如 $S \xRightarrow{*} aAa\beta$ ，终结符号 $a \in \text{Follow}(A)$
- 如果A是某些句型的最右符号，那么 $\$ \in \text{Follow}(A)$ 。\$是特殊的输入串“结束标记”
- FOLLOW函数的意义：
 - 如果 $A \rightarrow \alpha$ ，当 $\alpha \rightarrow \varepsilon$ 或 $\alpha \xRightarrow{*} \varepsilon$ 时，FOLLOW(A)可以帮助我们做出选择恰当的产生式
 - 例如：如果 $A \rightarrow \alpha$ ，b属于FOLLOW(A)，如果 $\alpha \xRightarrow{*} \varepsilon$ ，则若当前输入符号是b，可以选择 $A \rightarrow \alpha$ ，因为A最终到达了 ε ，而且后面跟着b



Follow的计算示例

- 文法

$$\begin{aligned} E &\rightarrow TE' & E' &\rightarrow +TE' \mid \varepsilon & T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid \varepsilon & F &\rightarrow (E) \mid i \end{aligned}$$
- $\text{FOLLOW}(E) = \{ \}, \$$ $\leftarrow \text{FIRST}(\) \cup \{ \$ \}$
- $\text{FOLLOW}(E') = \{ \}, \$$ $\leftarrow \text{FOLLOW}(E)$
- $\text{FOLLOW}(T) = \{ +, \}, \$$ $\leftarrow \text{FIRST}(E') \cup \text{FOLLOW}(E')$
- $\text{FOLLOW}(T') = \{ +, \}, \$$ $\leftarrow \text{FOLLOW}(T)$
- $\text{FOLLOW}(F) = \{ (, +, \}, \$$ $\leftarrow \text{FIRST}(T') \cup \text{FOLLOW}(T')$
- 将\$放入FOLLOW(S)，S是开始符号，而\$是输入串的结束标记
- 如果存在产生式 $A \rightarrow \alpha B \beta$ ，那么 $\text{First}(\beta)$ 中除 ε 之外的所有符号都在 $\text{Follow}(B)$ 中
- 如果存在一个产生式 $A \rightarrow \alpha B$ ，或存在产生式 $A \rightarrow \alpha B \beta$ 且 $\text{First}(\beta)$ 包含 ε ，那么 $\text{Follow}(A)$ 中的所有符号都在 $\text{Follow}(B)$ 中



思考：为什么

- 消除二义性
 - 消除左递归
 - 提取左公因子
 - 求First和Follow集合
- 改造文法
- LL(1)文法的预测分析



LL(1) 文法 (1)



- LL(1)文法, 第一个L表示自左向右扫描, 第二个L指产生最左推导, 而1则表示向前看一个输入符号
- LL(1)文法可以利用不回溯的确定性的预测分析技术, 因为只需要检查当前输入符号就可以为一个非终结符号选择正确的产生式



LL(1) 文法 (3)



- 思考: 对于一个LL(1)文法, 我们期望从A推导出以终结符号a开头的符号串
 - 我们如何选择产生式?
 - 有多个选择吗?



LL(1) 文法的预测分析技术



- LL(1)文法 ==> 预测分析表
- 将First和Follow集合中的信息放入一个预测分析表M[A,a], 该预测表告诉我们当非终结符号为A, 当前输入符号为a时, 要选择哪条产生式
- 预测分析表的构造
 - 若当前输入符号a在First(α)中, 选择产生式 $A \rightarrow \alpha$
 - 若 $\alpha \xrightarrow{*} \epsilon$, 如果a在Follow(A)中, 选择产生式 $A \rightarrow \alpha$
 - 如果当前符号a是\$, 且\$在Follow(A)中, 则选择产生式 $A \rightarrow \alpha$

非终结符号	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'	$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$	
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		



LL(1) 文法 (2)



- 定义: 对于文法的任意两个不同的产生式 $A \rightarrow \alpha | \beta$
 - 不存在终结符号a使得 α 和 β 都可以推导出以a开头的串
 - α 和 β 最多只有一个可以推导出空串
 - 如果 β 可以推导出空串, 那么 α 不能推导出以FOLLOW 中任何终结符号开头的串;
- 等价于:
 - $FIRST(\alpha) \cap FIRST(\beta) = \Phi$; (条件一、二)
 - 如果 $\epsilon \in FIRST(\beta)$, 那么 $FIRST(\alpha) \cap FOLLOW(A) = \Phi$; 反之亦然。(条件三)



LL(1) 文法 (4)



- 对于LL(1)文法, 可以在自顶向下分析过程中, 根据当前输入符号来确定使用的产生式
- 例如:
 - 产生式: $stmt \rightarrow \text{if}(\text{exp})\ stmt\ \text{else}\ stmt\ |\ \text{while}(\text{exp})\ stmt\ |\ a$
 - 输入: $\text{if}(\text{exp})\ \text{while}(\text{exp})\ a\ \text{else}\ a$
 - 首先选择产生式 $stmt \rightarrow \text{if}(\text{exp})\ stmt\ \text{else}\ stmt$ 得到句型 $\text{if}(\text{exp})\ stmt\ \text{else}\ stmt$
 - 然后发现要把句型中的第一个stmt展开, 选择产生式 $stmt \rightarrow \text{while}(\text{exp})\ stmt$, 得到句型 $\text{if}(\text{exp})\ \text{while}(\text{exp})\ stmt\ \text{else}\ stmt$
 - 然后再展开第一个stmt, 得到 $\text{if}(\text{exp})\ \text{while}(\text{exp})\ a\ \text{else}\ stmt$
 - ...



预测分析表构造



- 输入: 文法G
- 输出: 预测分析表M
- 方法: 对于文法G的每个产生式 $A \rightarrow \alpha$, 进行如下处理
 - 对于First(α)中的每个终结符号a, 将 $A \rightarrow \alpha$ 加入到M[A,a]
 - 如果 ϵ 在First(α)中, 那么对于Follow(A)中的每个终结符号b, 将 $A \rightarrow \alpha$ 加入到M[A,b]中
 - 如果 ϵ 在First(α)中, 且\$在Follow(A)中, 将 $A \rightarrow \alpha$ 加入到M[A,\$]中
- 完成上述操作后, 若M[A,a]中没有产生式, 填为Error



预测分析表的例子



- 文法:
 - $E \rightarrow TE'$ $E' \rightarrow +TE' \mid \epsilon$
 - $T \rightarrow FT'$ $T' \rightarrow *FT' \mid \epsilon$
 - $F \rightarrow (E) \mid id$
- FIRST集:
 - $F: \{(, id\};$ $E, T: \{(, id\};$ $E': \{+, \epsilon\};$ $T': \{*, \epsilon\}$
- FOLLOW集:
 - $E: \{ \$, \);$ $E': \{ \$, \);$ $T, T': \{+, \), \$\};$ $F: \{+, *, \), \$\}$

这个例子恰巧使得每个产生式的右部的第一个符号的FIRST集合就等于产生式右部的FIRST集合。但是在一般情况下不总是这样的。

非终结符号	输入符号					
	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		



二义性文法的预测分析表



- 对于某些文法，表中可能会有一些多重定义的条目，比如左递归或二义性文法。
 - $S \rightarrow iEtSS' \mid a$ -FIRST(ϵS)= $\{e\}$, 使得 $S' \rightarrow \epsilon S$ 在 $M[S', e]$ 中;
 - $S' \rightarrow eS \mid \epsilon$ -FOLLOW(S')= $\{ \$, e \}$ 使得 $S' \rightarrow \epsilon$ 也在 $M[S', e]$ 中
 - $E \rightarrow b$

非终结符号	输入符号					
	a	b	e	i	t	\$
S	$S \rightarrow a$			$S \rightarrow iEtSS'$		
S'			$S' \rightarrow \epsilon$ $S' \rightarrow eS$			$S' \rightarrow \epsilon$
E		$E \rightarrow b$				



第三题



题目 2 (LL 语法分析 (20 = 8 + 8 + 4 分))

考虑文法 G :

- $S \rightarrow cS$ (1)
- $S \rightarrow AB$ (2)
- $A \rightarrow aA$ (3)
- $A \rightarrow \epsilon$ (4)
- $B \rightarrow bBa$ (5)
- $B \rightarrow d$ (6)

- 请为文法 G 计算必要的 FIRST 集合与 FOLLOW 集合;
- 请为文法 G 构造预测分析表;
- 请问文法 G 是否是 LL(1) 文法, 并说明理由。



预测分析方法



- 对于任何文法 G , 都可以构造预测分析表
- 对于 LL(1) 文法, 预测表中每个条目都唯一地指定了一个产生式, 或者标明 Error



文法和自顶向下分析



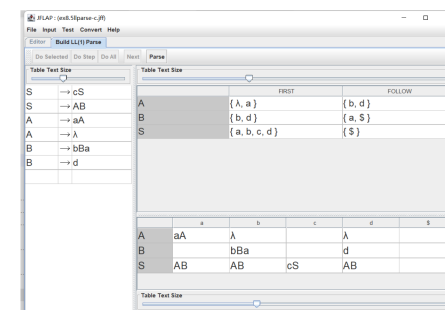
- 设计文法(保证描述能力, 且适用于自顶向下分析)
 - 消除二义性
 - 消除左递归
 - 提取左公因子
- 自顶向下分析
 - 递归下降分析 (通用的可能需要回溯)
 - 预测分析
- LL(1) 文法及其预测分析



第三题



- FIRST集合与FOLLOW集合见下图
- 预测分析表见下图
- 因为预测分析表无冲突, G 是 LL(1) 文法。





第四题



题目 4 (语法制导定义与翻译 (20 分))

考虑如下文法 G ,

$$\begin{aligned} S &\rightarrow LL \mid L \\ L &\rightarrow LB \mid B \\ B &\rightarrow 0 \mid 1 \end{aligned}$$

请设计语法制导的翻译方案, 计算 S 产生的二进制小数对应的十进制值。如, $101.101_2 = 5.625_{10}$ 。你需要自行定义合适的属性。



第四题



首先定义合适的属性:

- $v(B) : B$ 对应的值
- $v(L) : L$ 对应的值
- $v(S) : S$ 对应的值
- $l(L) : L$ 的长度

以上均为综合属性。

然后给出语法制导的翻译方案:

$$\begin{aligned} S &\rightarrow L_1.L_2 & \{v(S) = v(L_1) + v(L_2)/2^{l(L_2)}\} \\ S &\rightarrow L & \{v(S) = v(L)\} \\ L_1 &\rightarrow L_2B & \{v(L_1) = 2v(L_2) + v(B), \quad l(L_1) = l(L_2) + 1\} \\ L &\rightarrow B & \{v(L) = v(B), \quad l(L) = 1\} \\ B &\rightarrow 0 & \{v(B) = 0\} \\ B &\rightarrow 1 & \{v(B) = 1\} \end{aligned}$$



复习课（二）

（自底向上文法解析）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn



自底向上的语法分析

- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，一次归约实质上是一个推导的反向操作
 - 核心操作：移进（shift）与 归约（reduce）



自底向上的语法分析

- 如何理解：一次归约实质上是一个推导的反向操作



自底向上的语法分析

- 为一个输入串构造语法分析树的过程
- 从叶子（输入串中的终结符号，将位于分析树的底端）开始，向上到达根结点
- 重要的自底向上语法分析的通用框架
 - 移入-归约
- LR：最大的可以构造出移入-归约语法分析器的语法类



自底向上的语法分析

- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，一次归约实质上是一个推导的反向操作（重要！）



自底向上的语法分析

- 如何理解：一次归约实质上是一个推导的反向操作
- 自底向上分析的整条归约序列，等价于某个“最右推导（rightmost derivation）”的逆序
如果：

$$S = \gamma_0 \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n = w$$

是一条最右推导，那么自底向上的分析从 w 出发，按

$$\gamma_n \xrightarrow{\text{reduce}} \gamma_{n-1} \xrightarrow{\text{reduce}} \dots \xrightarrow{\text{reduce}} \gamma_0 = S$$

逐步把右部还原为左部，直到得到开始符号 S



自底向上的语法分析



- 自底向上的分析：将一个串 w 归约为文法的开始符号过程
 - 在每一步的归约中，一个与某产生式体相匹配的特定子串被替换为该产生式头部的非终结符号，**一次归约实质上是一个推导的反向操作（重要！）**
 - 核心操作：**移进（shift）** 与 **归约（reduce）**
 - 主要解决问题：
 - 什么时候应该移进
 - 什么时候应该归约



自底向上的语法分析



- 问题来了：**冲突**
- 为什么会有冲突：
 - 移入-归约技术并不能处理所有上下文无关文法



自底向上的语法分析



- 规约-规约冲突
- 示例：
 - 输入：id and id or id
 - 语义上我们希望：
 - and 的优先级 高于 or
 - 即应该解析为 (id and id) or id
 - 但是在这个文法下，LR 分析器会遇到规约-规约冲突：
 - 在读入 id and id 之后，分析栈已经能规约为一个 E
 - 但同时，前两个 id 也能直接规约为 $E \rightarrow id$ （多种规约选择）
 - 在下一个符号 or 到来时，分析器会发现：
 - 可以规约 $E \rightarrow E \text{ and } E$
 - 也可以规约 $E \rightarrow id$ （针对最后那个 id）
- 两个不同的 reduce 操作同时出现 $\Rightarrow$ 规约-规约冲突



自底向上的语法分析



- 规约-规约冲突
- 示例：

(1)	stmt	$\rightarrow$	id (parameter_list)
(2)	stmt	$\rightarrow$	expr := expr
(3)	parameter_list	$\rightarrow$	parameter_list , parameter
(4)	parameter_list	$\rightarrow$	parameter
(5)	parameter	$\rightarrow$	id
(6)	expr	$\rightarrow$	id (expr_list)
(7)	expr	$\rightarrow$	id
(8)	expr_list	$\rightarrow$	expr_list , expr
(9)	expr_list	$\rightarrow$	expr



自底向上的语法分析



- 问题来了：**冲突**
- 为什么会有冲突：
 - 文法允许多种推导方式（即 二义性文法）
 - 语法分析器无法仅凭当前状态和下一个输入符号决定：
 - 现在就归约
 - 还是继续移进
 - 或者说，应该选择哪条产生式进行规约
- 两类冲突：
 - 移进-规约冲突：分析器无法决定“移进”还是“规约”
 - 规约-规约冲突：分析器无法决定“用哪条规则规约”



自底向上的语法分析



- 从输入串出发构造一个反向最右推导序列（归约）
- 每一步是对**句柄**进行归约 -- 寻找句柄
- 一种移进-归约的分析框架
 - 一个栈存放文法符号，一个输入缓冲区
 - 移进 or 归约
 - 总是能够在栈的顶端找到句柄
 - 通过证明是可行的
- 有时候会有冲突，是移进还是归约？怎么归约？



自底向上的语法分析



- LR语法分析技术
- LR(k)的语法分析概念
 - L表示最左扫描, R表示反向构造出最右推导
 - k表示最多向前看k个符号
- 当k的数量增大时, 相应的语法分析器的规模急剧增大
 - K=2时, 程序设计语言的语法分析器的规模通常非常庞大
 - 当k=0、1时已经可以解决很多语法分析问题, 因此具有实践意义
 - 因此, 我们只考虑k<=1的情况



自底向上的语法分析



- 表格驱动
 - 虽然手工构造表格工作量大, 但表格可以自动生成
- 对于几乎所有的程序设计语言, 只要写出上下文无关文法, 就能够构造出识别该构造的LR语法分析器
- 最通用的无回溯移入-归约分析技术, 且和其它技术一样高效
- 可以尽早检测到错误
- LR(1) 可以识别所有确定性上下文无关文法
- 能分析的文法集合是LL(k)文法的超集



自底向上的语法分析



- 移入-归约语法分析器如何知道何时该移入、何时该归约呢?
 - LR语法分析器试图用一些状态来表明我们在移进归约语法分析过程中所处的位置, 从而做出移入-归约决定
- 项的意义
 - 指明在语法分析过程中的给定点上, 我们已经看到了一个产生式的哪些部分。或者说, 如果我们想用这个产生式进行归约, 还需要看到哪些文法符号
- 项的集合(项集)对应于一个状态



自底向上的语法分析



- LR(0)分析技术
- LR(0)项
 - 文法的一个产生式加上在其产生式体中某处的一个点
 - $A \rightarrow .XYZ$, $A \rightarrow X.YZ$, $A \rightarrow XY.Z$, $A \rightarrow XYZ.$
 - 注意: $A \rightarrow \epsilon$ 只对应一个项 $A \rightarrow .$
 - 直观含义
 - 项 $A \rightarrow \alpha.\beta$ 表示已经扫描/归约到了 α , 并期望接下来的输入中经过扫描/归约得到 β , 然后把 $\alpha\beta$ 归约到A
 - 如果 β 为空, 表示我们可以把 α 归约为A
 - 项也可以用一对整数表示
 - (i,j)表示第i条规则, 点位于右部第j个位置



自底向上的语法分析



- LR(0)分析技术
- 规范LR(0)项集族
 - 规范LR(0)项集族提供构建LR(0)自动机的基础
 - LR(0)自动机可用于做出语法分析决定
 - LR(0)自动机中每个状态代表了LR(0)项集族中的一个项集



自底向上的语法分析



- LR(0)分析技术
- 以项为基础构造自动机(DFA)
 - 构造以项为状态的自动机
 - 开始状态 $S' \rightarrow .S$
 - 转换
 - $A \rightarrow \alpha.B\beta$ 到 $A \rightarrow \alpha.B\beta$ 有一个 ϵ 转换
 - 从 $A \rightarrow \alpha.X\beta$ 到 $A \rightarrow \alpha.X\beta$ 有一个X转换
 - 接受状态: $A \rightarrow \alpha.$, 即点在最后的项



自底向上的语法分析



- LR(0) 分析技术
- 规范LR(0) 项集族的构造
- 三个概念
 - 增广文法
 - 项集闭包: CLOSURE
 - GOTO



自底向上的语法分析



- LR(0) 分析技术
- 规范LR(0) 项集族的构造
- 增广文法
 - G的增广文法G'是在G中增加新开始符号S', 并加入产生式S'→S而得到的
 - G'和G接受相同的语言, 且按照S'→S进行归约实际上就表示已经将输入符号串归约成为开始符号
 - 引入的目的是告诉语法分析器何时宣布接受输入符号串, 即用S'→S进行归约时, 表明分析结束



自底向上的语法分析



- LR(0) 分析技术
- 规范LR(0) 项集族的构造
 - CLOSURE(I): I的项集闭包
 - 对应于DFA化算法的ε-CLOSURE
 - GOTO(I,X): I的X后继
 - 对应于DFA化算法的MOVE(I,X)



自底向上的语法分析



- CLOSURE(I) 的构造算法
- 如果I是文法G的一个项集, 那么CLOSURE(I)就是根据下列规则从I构造得到的项集
 - 将I中的各个项加入到CLOSURE(I)中
 - 如果A → α.Bβ在CLOSURE(I)中, 那么对B的任意产生式B → γ, 将B → γ加到CLOSURE(I)中
 - 不断重复第二步, 直到收敛
- 第二步的意义
 - 项A → α.Bβ表示期望在接下来的输入中归约到B
 - 显然, 要归约到B, 首先要扫描归约到B的某个产生式的右部
 - 因此对每个产生式B → γ, 加入B → γ
 - 表示它期望能够扫描归约到γ



自底向上的语法分析



- 增广文法:
 - E' → E E → E+T | T T → T*F | F F → (E) | id
- 项集 I={ [E' → E] } 的闭包
 - [E' → E]在闭包中
 - [E → E+T], [E → T]在闭包中
 - [T → T*F], [T → F]在闭包中
 - [F → (E)], [F → id]在闭包中

```

SetOfItems CLOSURE(I) {
    J = I;
    repeat
        for (J中的每个项 A → α.Bβ)
            for (G 的每个产生式 B → γ)
                if (项 B → γ 不在 J 中)
                    将 B → γ 加入 J 中;
    until 在某一轮中没有新的项被加入到 J 中;
    return J;
}
  
```



自底向上的语法分析



- LR(0) 项集中的内核项和非内核项
 - 内核项: 初始项S' → S、以及所有点不在最左边的项
 - 非内核项: 除了S' → S之外、点在最左边的项
- 由于表可能很庞大, 实现算法时可以考虑只保存相应的非终结符号; 甚至可以只保存内核项, 而在要使用非内核项时调用CLOSURE函数重新计算



自底向上的语法分析

- GOTO函数
- I 是一个项集, X 是一个文法符号, $GOTO(I, X)$ 定义为 I 中所有形如的项 $[A \rightarrow \alpha \cdot X \beta]$ 所对应的项 $[A \rightarrow \alpha X \cdot \beta]$ 的集合的闭包
 - 根据项的历史-期望的含义, $GOTO(I, X)$ 表示读取输入中的 X 或者归约到一个 X 之后的情况
 - $GOTO(I, X)$ 定义了 LR(0) 自动机中状态 I 在 X 之上的转换
- 例如:
 - $I = \{[E' \rightarrow E.], [E \rightarrow E. + T]\}$
 - $GOTO(I, +)$ 计算如下
 - I 中只有一个项的点后面跟着 $+$, 对应的项为 $[E \rightarrow E. + T]$
 - $CLOSURE(\{[E \rightarrow E. + T]\}) = \{[E \rightarrow E. + T], [T \rightarrow T * F], [T \rightarrow (E)], [F \rightarrow (E)], [F \rightarrow id]\}$



自底向上的语法分析

- 规范 LR(0) 项集族的构造
- 为文法 G 构造 LR(0) 项集规范族 C 的关键步骤总结
 - LR(0) 项目: 形如 $A \rightarrow \alpha \cdot \beta$, 点 $(\cdot)$ 表示在该产生式 “已经识别到” 的位置
 - $closure(I)$: 若 $A \rightarrow \alpha \cdot B \beta \in I$ ($\cdot$ 在非终结符 B 前), 则把所有 $B \rightarrow \gamma$ 加入 I , 并重复直到不再能加入
 - $goto(I, X)$: 对集合 I 中所有 $A \rightarrow \alpha \cdot X \beta$, 把点向右移一位得 $A \rightarrow \alpha X \cdot \beta$, 然后对该新集合做 $closure$, 得到 $goto(I, X)$
 - 从初始 $I_0 = closure(\{E' \rightarrow \cdot E\})$ 出发, 反复计算 $goto$, 直到无新集合为止 $\rightarrow$ 得到项目集族 (states)

给出来一个 step by step 的例子?



自底向上的语法分析

- 规范 LR(0) 项集族的构造示例:
 - $goto(I_0, id)$: 由 $F \rightarrow \cdot id$ 得 $F \rightarrow id \cdot$ (这是一个完成项, 表示能规约 $F \rightarrow id$); $closure$ 不再添加新项
 - $goto(I_0, '(')$: 由 $F \rightarrow \cdot (E)$ 得 $F \rightarrow (\cdot E)$, 此时 $\cdot$ 前是非终结符 E , 因此需要把 $E \rightarrow \cdot E + T, E \rightarrow \cdot T$ 等产生式一并加入, 这会形成一个结构近似于 I_0 的子集 (但 $\cdot$ 在不同位置)
 - $goto(I_0, E)$: 由 $E' \rightarrow \cdot E$ 与 $E \rightarrow \cdot E + T$ 等项目得到 $E' \rightarrow E \cdot$ 与 $E \rightarrow E \cdot + T$, 这构成了一个状态 (可以记为 I_1), 表示 “已识别一个完整的 E , 但也可看到 $+$ 接着更多的东西”
 - $goto(I_0, T), goto(I_0, F) \dots$



自底向上的语法分析

- 规范 LR(0) 项集族的构造
- 为文法 G 构造 LR(0) 项集规范族 C
 - 步骤一: 构造增广文法 G' : $S' \rightarrow S \dots$
 - 步骤二: 构造 $I_0 = CLOSURE(S' \rightarrow \cdot S)$, I_0 是 C 的初始项集
 - 步骤三: 对 C 中的每个项集 I_i 及每个文法符号 X_j , 将 $GOTO(I_i, X_j)$ 加入到 C 中
 - 重复步骤三, 直到没有新的项集可以加入

```
void items(G') {
    C = CLOSURE({S' -> .S});
    repeat
        for (C 中的每个项集 I)
            for (每个文法符号 X)
                if (GOTO(I, X) 非空且不在 C 中)
                    将 GOTO(I, X) 加入 C 中;
    until 在某一轮中没有新的项集被加入到 C 中;
}
```

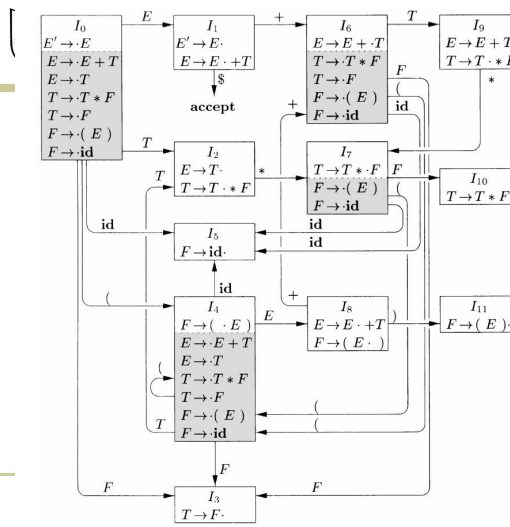


自底向上的语法分析

- 规范 LR(0) 项集族的构造示例:
 - 初始 $I_0 = closure(\{E' \rightarrow \cdot E\})$ 会包含下列项目:
 - $E' \rightarrow \cdot E$
 - $E \rightarrow \cdot E + T$
 - $E \rightarrow \cdot T$
 - $T \rightarrow \cdot T * F$
 - $T \rightarrow \cdot F$
 - $F \rightarrow \cdot (E)$
 - $F \rightarrow \cdot id$

说明: 因为 $\cdot$ 前面是 E (非终结符), 我们把 E 的产生式也加入; 在 $E \rightarrow \cdot T$ 中 $\cdot$ 前是 T , 我们继续把 T 的产生式加入; 同理把 F 的产生式加入, 直到 $closure$ 完成

- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$



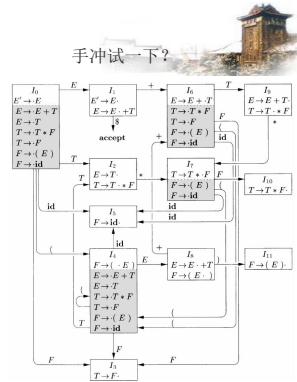
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T * F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$

项集规范族和 GOTO 函数的例子



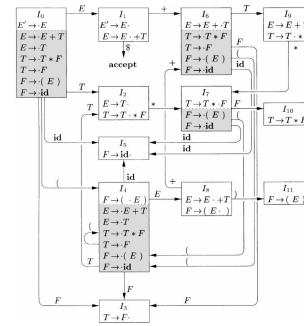
自底向上的语法分析

- 我们得到了如上的DFA，下一步，构建分
- 基本算法：
对于每个状态 i 和输入符号 a ：
移进 (Shift)
 - 如果某个项目 $[A \rightarrow \alpha \cdot a]$ 在状态 i
 - 那么 $ACTION[i, a] = "S_j"$ ，其中 $j = goto(i, a)$ 的
 规约 (Reduce)
 - 如果某个项目 $[A \rightarrow \alpha \cdot]$ 在状态 i (点在最右边)
 - 那么对所有 $a \in FOLLOW(A)$ ， $ACTION[i, a] = "R(A \rightarrow \alpha)"$
 接受 (Accept)
 - 如果有项目 $[E' \rightarrow E \cdot]$ 在状态 i ，并且输入符号是 $\$$ ，且 $goto(i, \$) = j$ ，则 $GOTO[i, \$] = j$
 - 那么 $ACTION[i, \$] = ACC$



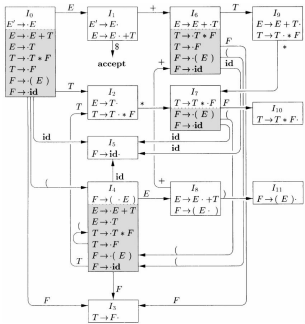
自底向上的语法分析

- 文法: (1) $E \rightarrow E+T$ (2) $E \rightarrow T$
(3) $T \rightarrow T * F$ (4) $T \rightarrow F$
(5) $F \rightarrow (E)$ (6) $F \rightarrow id$



自底向上的语法分析

- 文法: (1) $E \rightarrow E+T$ (2) $E \rightarrow T$
(3) $T \rightarrow T * F$ (4) $T \rightarrow F$
(5) $F \rightarrow (E)$ (6) $F \rightarrow id$

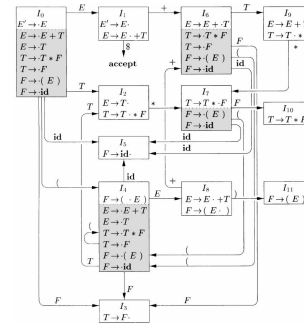


状态	id	(	)	+	*	\$	E	T	F
0	s5	s4					1	2	3
1				s6		acc			
2	r2	r2	r2	r2	s7/r2				
3	r4	r4	r4	r4	r4				
4	s5	s4				8	2	3	
5	r6	r6	r6	r6	r6				
6	s5	s4				9	3		
7	s5	s4				10			
8				s11	s6				
9	r1	r1	r1	r1	s7/r1	r1			
10	r3	r3	r3	r3	r3				
11	r5	r5	r5	r5	r5				



自底向上的语法分析

- 文法: (1) $E \rightarrow E+T$ (2) $E \rightarrow T$
(3) $T \rightarrow T * F$ (4) $T \rightarrow F$
(5) $F \rightarrow (E)$ (6) $F \rightarrow id$

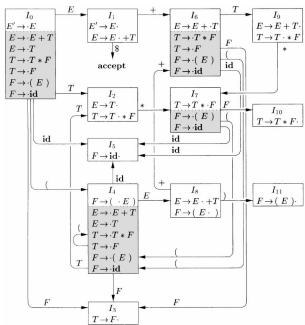


状态	id	(	)	+	*	\$	E	T	F
0	s5	s4					1	2	3
1				s6		acc			
2	r2	r2	r2	r2	s7/r2				
3	r4	r4	r4	r4	r4				
4	s5	s4				8	2	3	
5	r6	r6	r6	r6	r6				
6	s5	s4				9	3		
7	s5	s4				10			
8				s11	s6				
9	r1	r1	r1	r1	s7/r1	r1			
10	r3	r3	r3	r3	r3				
11	r5	r5	r5	r5	r5				



自底向上的语法分析

- 文法: (1) $E \rightarrow E+T$ (2) $E \rightarrow T$
(3) $T \rightarrow T * F$ (4) $T \rightarrow F$
(5) $F \rightarrow (E)$ (6) $F \rightarrow id$



在上表中出现 Shift/Reduce 冲突的格子为：
State 2，在符号 * 上：表格格子是 s7/r2: $E \rightarrow T$ (冲突)。
State 2 的 LR(0) 项集为：
 $E \rightarrow T \cdot$ (可以用 $E \rightarrow T$ 归约)。
 $T \rightarrow T \cdot * F$ (看到 * 时可以 shift，进入乘法右侧)。
因为 LR(0) 在没有 lookahead 的情况下会在所有终结符上进行归约，所以在 * 上就同时出现了 shift (为处理乘法) 与 reduce (把当前 T 归约为 E) 两种动作 —— 无法在 LR(0) 中决定，产生冲突。

State 9，在符号 * 上：表格格子是 s7/r1: $E \rightarrow E + T$ (冲突)。
State 9 的 LR(0) 项集为：
 $E \rightarrow E + T \cdot$ (可以用 $E \rightarrow E + T$ 归约)。
 $T \rightarrow T \cdot * F$ (看到 * 时可 shift)。
同理，既可以把 $E + T$ 归约为 E (r1)，又可能因为接下来看到 * 而需要做乘法结合 (shift)，LR(0) 无法区分，发生冲突。

状态	id	(	)	+	*	\$	E	T	F
0	s5	s4					1	2	3
1				s6		acc			
2	r2	r2	r2	r2	s7/r2				
3	r4	r4	r4	r4	r4				
4	s5	s4				8	2	3	
5	r6	r6	r6	r6	r6				
6	s5	s4				9	3		
7	s5	s4				10			
8				s11	s6				
9	r1	r1	r1	r1	s7/r1	r1			
10	r3	r3	r3	r3	r3				
11	r5	r5	r5	r5	r5				



自底向上的语法分析

- 假设文法符号串 γ 使 LR(0) 自动机从开始状态运行到状态 (项集) j
 - 如果 j 中有一个形如 $A \rightarrow \alpha \cdot$ 的项，那么
 - 在 γ 之后添加一些终结符号可以得到一个最右句型
 - α 是 γ 的后缀，且 $A \rightarrow \alpha$ 是这个句型的句柄
 - 表示可能找到了当前最右句型的句柄
 - 如果 j 中存在一个项 $B \rightarrow \alpha \cdot X \beta$ ，那么
 - 在 γ 之后添加 $X\beta$ ，然后再添加一个终结符号串可得到一个最右句型
 - 在这个句型中 $B \rightarrow \alpha X \beta$ 是句柄
 - 此时表示还没有找到句柄，需要移入



自底向上的语法分析



- LR(0)自动机的使用
 - 移入-归约时，LR(0)自动机被用于识别句型
 - 已经归约/移入得到的文法符号序列对应于LR(0)自动机的一条路径
 - 如果路径到达接受状态，表明栈上端的某个符号串可能是句柄
- 不需要每次用归约/移入得到的串来运行LR(0)自动机
 - 路径被放到栈中；且文法符号可以省略，因为由LR(0)状态可以确定相应文法符号
 - 在移入后，根据原来的栈顶状态即可知道新的状态
 - 在归约时，根据归约产生式的右部长度的右部长度弹出相应状态，仍然可以根据此时的栈顶状态计算得到新状态



自底向上的语法分析



- 一个关键的问题？！
- LR(0) 在做“规约”决策时不看任何前瞻符号
 - 也就是说一旦某个项目集里出现了完成项 $A \rightarrow \alpha \cdot$ ，LR(0) 会在所有可能的输入符号上把它当作可规约动作（除非该状态上还有移进动作，产生移进-规约冲突）
- 对于这个表达式文法，许多状态会同时包含：
 - 某个完成项（例如 $F \rightarrow id \cdot$ 或 $T \rightarrow F \cdot$ 等），以及
 - 另一个项目如 $T \rightarrow T \cdot * F$ 或 $E \rightarrow E \cdot + T$ （点在非终结符或终结符前）
- 因此，当分析器遇到某些输入（例如 $id + id * id$ 的中间位置），会产生移进-规约冲突或规约-规约冲突

LR(0) 无法凭借其有限状态做出正确、唯一的决策



自底向上的语法分析



- 回顾：
 - First函数：由某个非终结符能够推导出来的句子的句首符号的集合，就是这个非终结符的First集合
 - Follow函数：对于某个非终结符，紧跟在其后面的终结符的集合就是这个非终结符的Follow集



自底向上的语法分析



- 一个关键的问题？！
- LR(0) 在做“规约”决策时不看任何前瞻符号
 - 也就是说一旦某个项目集里出现了完成项 $A \rightarrow \alpha \cdot$ ，LR(0) 会在所有可能的输入符号上把它当作可规约动作（除非该状态上还有移进动作，产生移进-规约冲突）



自底向上的语法分析



- 讨论：LR(0) 的局限性
 - 有些文法会产生 移进-规约冲突 或 规约-规约冲突
 - LR(0) 过于严格，能处理的文法较少
 - 根本原因：
 - LR(0) 只看项目中的“点位置”，并不知道后面真正可能出现的符号
 - 因此在“能规约”和“能移进”的情形下，它无法做出唯一选择
- 改进方案是什么？
 - 更强的算法（SLR, LR(1), LALR）就是通过前瞻或 FOLLOW 来减少冲突



自底向上的语法分析



- SLR: Simple LR 算法的核心
 - 基本思想：
 - 在 LR(0) 项目集基础上，加上 FOLLOW 集来控制规约动作
 - 规约 $A \rightarrow \alpha$ 只在 输入符号 $\in \text{FOLLOW}(A)$ 时才执行
 - 优点：
 - 消除了大部分 LR(0) 的冲突
 - 构造过程和 LR(0) 类似，只需在生成 ACTION 表时加上 FOLLOW 集限制
 - 缺点：
 - 仍然可能存在不足（FOLLOW 集过大，可能引入不必要的冲突）



自底向上的语法分析

- SLR: Simple LR 算法的核心
- Follow集的计算:



自底向上的语法分析

- SLR: Simple LR 算法的核心
 - 如果不仅考虑First, 而且把Follow集也考虑在内, 就可以一定程度上解决reduce-shift conflict. 还是以上语法:
 - $F \rightarrow Y \cdot + B$
 - $F \rightarrow Y \cdot$
 - 如果 $FOLLOW(F) = \{a, b\}$, 那么当我们遇到a与b时, 就可以选择归约F; 当我们遇到+时, 就可以选择移进操作



自底向上的语法分析

- 输入: 一个增广文法G'
- 输出: G'的SLR语法分析表函数ACTION和GOTO
- 方法:
 - 1) 构造G' LR(0)项集规范族 $C = \{I_0, I_1, \dots, I_n\}$
 - 2) 对于状态i中的项:
 - $[A \rightarrow \alpha \cdot a \beta]$ 且 $GOTO[I_i, a] = I_j$, 那么 $ACTION[i, a] = \text{移入 } j$
 - $[A \rightarrow \alpha \cdot]$, 那么对于 $FOLLOW(A)$ 中的所有a, $ACTION[i, a] = \text{归约 } A \rightarrow \alpha$
 - $[S' \rightarrow S \cdot]$, 那么 $ACTION[i, \$] = \text{接受}$
 - 3) 状态i对于非终结符号A的转换规则:
 - 如果 $GOTO[I_i, A] = I_j$, 那么 $GOTO[i, A] = j$
 - 4) 规则2)和3)没有定义的所有条目设置为“报错”



自底向上的语法分析

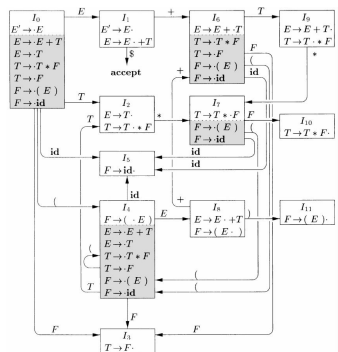
- SLR: Simple LR 算法的核心
 - (0) $E' \rightarrow E$
 - (1) $E \rightarrow E + T$
 - (2) $E \rightarrow T$
 - (3) $T \rightarrow T * F$
 - (4) $T \rightarrow F$
 - (5) $F \rightarrow (E)$
 - (6) $F \rightarrow id$
- FOLLOW 集结果:
 - $FOLLOW(E) = \{ \$,), + \}$
 - 说明: 来自 $E' \rightarrow E$ 得到 $\$$; $F \rightarrow (E)$ 得到 $)$; $E \rightarrow E + T$ 在 $+$ 后面有 T , 与 $FOLLOW(E)$ 无关; 但在 $E \rightarrow E + T$ 中, E 后面直接是 $+$, 因此 $+$ $\in FOLLOW(E)$.
 - $FOLLOW(T) = \{ \$,), + \}$
 - 说明: $E \rightarrow E + T$ 给 $FOLLOW(T)$ 包含 $FOLLOW(E)$; 且 T 在 $E \rightarrow E + T$ 末尾, $FOLLOW(T) \supseteq FOLLOW(E)$; 综合为 $\{ \$,), + \}$.
 - $FOLLOW(F) = \{ \$,), +, * \}$
 - 说明: $T \rightarrow T * F$ 给 $FOLLOW(F)$ (F 前面是 $*$) 但 F 在式尾, $FOLLOW(F) \supseteq FOLLOW(T)$; 同时在 $T \rightarrow T * F$ 中, F 后无符号, 故 $FOLLOW(F) \supseteq FOLLOW(T)$; 最终 $\{ \$,), +, * \}$.
- 在某状态出现 $[E \rightarrow T \cdot]$
 - 在 LR(0): 会在所有终结符上放 "reduce $E \rightarrow T$ "
 - 所以在 $*$ 上冲突 (因为 $[T \rightarrow T * F]$ 允许移进)
 - 在 SLR: 只在 $FOLLOW(E) = \{ +,), \$ \}$ 上放规约
 - 在 $*$ 上不会规约 $\rightarrow$ 冲突消除了

$FOLLOW(E) = \{ +,), \$ \}$
 $FOLLOW(T) = \{ +, *,), \$ \}$
 $FOLLOW(F) = \{ +, *,), \$ \}$



自底向上的语法分析

- SLR: Simple LR 算法的核心

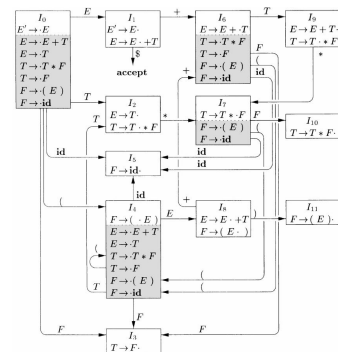


	(0) $E' \rightarrow E$											
	(1) $E \rightarrow E + T$											
	(2) $E \rightarrow T$											
	(3) $T \rightarrow T * F$											
	(4) $T \rightarrow F$											
	(5) $F \rightarrow (E)$											
	(6) $F \rightarrow id$											
	$FOLLOW(E) = \{ +,), \$ \}$											
	$FOLLOW(T) = \{ +, *,), \$ \}$											
	$FOLLOW(F) = \{ +, *,), \$ \}$											
状态	id	(	)	+	*	\$	E	T	F			
0	s5	s4						1	2	3		
1					s6					acc		
2	r2	r2	r2	r2	s7/r2	r2						
3	r4	r4	r4	r4	r4	r4						
4	s5	s4					8	2	3			
5	r6	r6	r6	r6	r6	r6						
6	s5	s4						9	3			
7	s5	s4							10			
8				s11	s6							
9	r1	r1	r1	r1	s7/r1	r1						
10	r3	r3	r3	r3	r3	r3						
11	r5	r5	r5	r5	r5	r5						



自底向上的语法分析

- SLR: Simple LR 算法的核心



	(0) $E' \rightarrow E$											
	(1) $E \rightarrow E + T$											
	(2) $E \rightarrow T$											
	(3) $T \rightarrow T * F$											
	(4) $T \rightarrow F$											
	(5) $F \rightarrow (E)$											
	(6) $F \rightarrow id$											
	$FOLLOW(E) = \{ +,), \$ \}$											
	$FOLLOW(T) = \{ +, *,), \$ \}$											
	$FOLLOW(F) = \{ +, *,), \$ \}$											
状态	id	(	)	+	*	\$	E	T	F			
0	s5	s4						1	2	3		
1					s6					acc		
2												
3												
4	s5	s4					8	2	3			
5												
6	s5	s4						9	3			
7	s5	s4							10			
8				s11	s6							
9												
10												
11												





- 根据上一算法构造的语法分析表，若表中各位置没有多个条目，则称为文法G的SLR(1)分析表。使用该分析表的分析器，称为G的SLR(1)语法分析器。G称为SLR(1)文法
- 若表中某位置存在多个条目（多个可选的动作，冲突），则该文法是非SLR(1)文法

总结：SLR(1) = 在 LR(0) 基础上 + FOLLOW 过滤

→ 简单高效，但能力有限；LR(1) 更强，但代价是状态数暴增。



- SLR语法分析器的弱点
 - SLR不能完全解决reduce-shift conflict.
 - 同样还是上面的语法：
 - $F \rightarrow Y \cdot + B$
 - $F \rightarrow Y \cdot$
 - 如果 $FOLLOW(F) = \{a, b, +\}$ ，那当我们遇到 + 符号时，就无法确定到底是选择移进操作得到 $F \rightarrow Y + \cdot B$ ，还是归约F。
 - SLR不能完全解决reduce-shift conflict. 这就是为什么我们要选择LR(1) / LALR(1)了



复习课（三） （语义分析）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn



语法制导定义的基本概念

- 计算机程序是由很多语法结构构成
- 相同的语法单元在不同的语法结构中可能包含不同的语义信息
- 使用上下文无关文法引导语言的翻译
 - CFG的非终结符号代表了语言的某个构造
 - 程序设计语言的构造由更小的构造组合而成
 - 一个构造的语义可以由小构造的含义**综合**而来
 - 比如：表达式 $x+y$ 的类型由 x 、 y 的类型和运算符 $+$ 决定
 - 也可以从附近的构造**继承**而来
 - 比如：声明`int x;`中 x 的类型由它左边的类型表达式决定



语法制导定义的基本概念

- 语法制导定义(SDD: Syntax-Directed Definition)
 - 将文法符号和某些属性相关联
 - 并通过**语义规则**来描述如何计算属性的值
 - $E \rightarrow E_1 + T$ $E.code = E_1.code \parallel T.code \parallel '+'$
 - 属性`code`代表中缀表达式的逆波兰表示(后缀表示法)，规则说明加法表达式的逆波兰表示由两个分量的逆波兰表示并置，然后加上 $+$ 得到
- 语法制导翻译(SDD: Syntax-Directed Translation)
 - 在产生式体中加入**语义动作**，并在适当的时候执行这些语义动作
 - $E \rightarrow E_1 + T$ `{print '+';}`



四、简答题（共1题，共15分）

给定一个表示四进制小数的文法 G ，其开始符号为 S ，并且包

含以下四个产生式。小数部分可以为空，即 `123` 是合法的四

进制小数。产生式如下：

$G(S):$ $S \rightarrow Q_1 \mid Q_1 Q_2$
 $Q_1 \rightarrow Q_1 \text{Num} \mid \text{Num}$
 $Q_2 \rightarrow \text{Num} Q_2 \mid \text{Num}$
 $\text{Num} \rightarrow 0 \mid 1 \mid 2 \mid 3$

1. (5分) 文法 G 中包含左递归，请消除左递归；

2. (5分) 请设计一个语法制导翻译方案来统计四进制小数中奇数数字(1和3)的个数。

例如 2.123 中奇数数字个数为 2；

3. (5分) 请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123

转换的二进制字符串为 "10.011011"。



语法制导定义的基本概念

- 背景：
 - 上下文无关文法只能描述语法
 - 需要结合语义规则来完成语义分析与中间代码生成
- 语法制导的定义 (SDD: Syntax-Directed Definition):
 - 一种形式系统，将语义规则附着到文法产生式上
 - 将文法符号和某些属性相关联
 - 并通过**语义规则**来描述如何计算属性的值

语法树 + 属性计算 = 语义分析



语法制导定义的基本概念

特性
表达方式
可读性
应用

SDD（属性定义）
抽象、数学化
高，便于教学与证明
理论建模，描述属性依赖

SDT（语义动作）
具体、嵌入代码
偏向实现，代码细节多
编译器工具（如 YACC/Bison）



语法制导定义的基本概念



- SDD是对上下文无关文法的推广
 - SDD = 上下文无关文法 + 属性 + 语义规则
 - 将每个文法符号和一个语义属性集合相关联
 - 将每个产生式和一组语义规则相关联，这些规则用于计算该产生式中各文法符号的属性值
- 为什么需要属性？
 - 语法分析只能保证程序符号串符合文法规则
 - 但程序的语义信息（如类型、值、作用域等）需要额外描述
 - 属性 (Attribute) 就是为了解决这个问题
 - 给文法符号附加额外信息
 - 在分析过程中逐步计算并传播这些信息



语法制导定义的基本概念



- 属性 (Attribute): 与文法符号相关联的值，用来存储语义信息
- 在属性文法 (Attributed Grammar) 中，属性之间可能存在依赖关系

$$E.val = E1.val + T.val$$

这里 $E.val$ 的计算依赖于 $E1.val$ 和 $T.val$

问题：我们如何确定属性计算的顺序？

引入 **依赖关系的图形表示** 来形式化表达属性之间的依赖



语法制导定义的基本概念



- 按照分析树中的分支对应的文法产生式，应用相应的语义规则计算属性值
- 计算顺序问题：
 - 如果某个结点 N 的属性 a 为 $f(N_1.b_1, N_2.b_2, \dots, N_k.b_k)$ ，那么我们需要先算出 $N_1.b_1, N_2.b_2, \dots, N_k.b_k$ 的值。
- 如果我们可以给各个属性值排出计算顺序，那么这个注释分析树就可以计算得到
 - S属性的SDD一定可以按照自底向上的方式求值
- 下面的SDD不能计算
 - $A \rightarrow B$ $A.s = B.i;$ $B.i = A.s + 1;$



语法制导定义的基本概念



- 属性 (Attribute): 与文法符号相关联的值，用来存储语义信息
- 在属性文法 (Attributed Grammar) 中，属性之间可能存在依赖关系

$$E.val = E1.val + T.val$$



语法制导定义的基本概念



- 实践中很少先构造语法分析树再进行SDD求值
- 但在分析树上求值有助于翻译方案的可视化，便于理解
- 注释语法分析树
 - 包含了各个结点的各属性值的语法分析树
- 步骤：
 - 对于任意的输入串，首先构造出相应的分析树。
 - 给各个结点（根据其文法符号）加上相应的属性值
 - 按照语义规则计算这些属性值即可



语法制导定义的基本概念



- 分为两类：
 - 综合属性 (Synthesized Attribute): 自下而上计算，从子结点传到父结点，常用于表达式的值、类型计算
 - 继承属性 (Inherited Attribute): 自上而下或兄弟结点之间传递，常用于符号表传递、上下文约束、类型环境
- 不允许结点的继承属性通过子结点上的属性来定义，但是允许结点的综合属性依赖于结点本身的继承属性
- 终结符号有综合属性（由词法分析获得），但是没有继承属性



语法制导定义的基本概念

综合属性示例:

文法: $E \rightarrow E + T \mid T$ $T \rightarrow \text{num}$

$E \rightarrow E1 + T$ { $E.\text{val} = E1.\text{val} + T.\text{val}$ }

$E \rightarrow T$ { $E.\text{val} = T.\text{val}$ }

$T \rightarrow \text{num}$ { $T.\text{val} = \text{num}.\text{lexval}$ }

val 是 **综合属性**, 计算方向是 **从下到上**



语法制导定义的基本概念

SDD中属性的求值顺序

- 在对SDD的求值过程中, 如果结点N的属性a依赖于结点 M_1 的属性 a_1 , M_2 的属性 a_2 , ...。那么我们必须先计算出 M_1 的属性, 才能计算N的属性a
- 使用依赖图来表示计算顺序
- 显然, 这些值的计算顺序应该形成一个偏序关系。如果依赖图中出现了环, 表示属性值无法计算



语法制导定义的基本概念

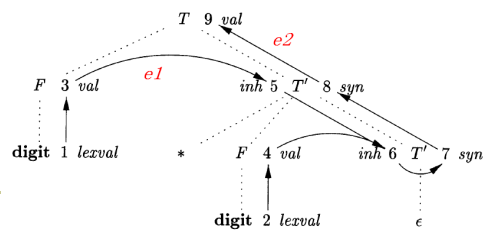
3*2的注释分析树:

$T \rightarrow FT'$ { $T.\text{val} = T'.\text{syn}; T'.\text{inh} = F.\text{val};$ }

边e1、e2

可能的计算顺序:

1.2.3.4.5.6.7.8.9



产生式	语义规则
1) $T \rightarrow FT'$	$T'.\text{inh} = F.\text{val}$ $T.\text{val} = T'.\text{syn}$
2) $T' \rightarrow * FT'_1$	$T'_1.\text{inh} = T'.\text{inh} \times F.\text{val}$ $T'.\text{syn} = T'_1.\text{syn}$
3) $T' \rightarrow \epsilon$	$T'.\text{syn} = T'.\text{inh}$
4) $F \rightarrow \text{digit}$	$F.\text{val} = \text{digit}.\text{lexval}$



语法制导定义的基本概念

继承属性示例:

文法: $L \rightarrow TL' \mid \epsilon$ $T \rightarrow \text{id}$

$T.\text{type}$ 由父节点传递

$\text{id}.\text{entry.type} = T.\text{type}$ (把类型传给符号表项)

$T.\text{type}$ 就是 **继承属性**, 它随文法结构向下传递



语法制导定义的基本概念

依赖的图形化表示: **依赖图**

- 描述了某棵特定的分析树上各个属性实例之间的信息流 (计算顺序)
- 从实例 a_1 到实例 a_2 的有向边表示计算 a_2 时需要 a_1 的值 (必须先计算 a_2 , 再计算 a_1)
- 对于标号为X的分析树结点N, 和X关联的每个属性a都对应依赖图的一个结点N.a
- 结点N对应的产生式的语义规则通过X.c计算了A.b的值, 且在分析树中X和A分别对应于 N_1 和 N_2 , 那么从 $N_1.c$ 到 $N_2.b$ 有一条边



语法制导定义的基本概念

各个属性的值需要按照依赖图的拓扑顺序计算

- 如果依赖图中存在环, 则属性计算无法进行

给定一个SDD, 很难判定是否存在一棵分析树, 其对应的依赖图包含环

怎样的SDD可以和以前所介绍的语法分析结合在一起?



语法制导定义的基本概念



- 特定类型的SDD一定不包含环，且有固定排序模式
 - S属性的SDD
 - L属性的SDD
- 对于这些类型的SDD，我们可以确定属性的计算顺序，且可以把不需要的属性（及分析树结点）抛弃以提高效率
- 这两类SDD可以很好地和我们已经研究过的语法分析相结合



语法制导定义的基本概念



- L属性的SDD，其中，每个属性
 - 要么是综合属性
 - 要么是继承属性，且产生式 $A \rightarrow X_1X_2 \cdots X_n$ 中计算 X_i .a的规则只能使用
 - A的继承属性
 - X_i 左边的文法符号 X_j 的继承属性或综合属性
 - X_i 自身的继承或综合属性，且这些属性之间的依赖关系不形成环



语法制导定义的基本概念



- S属性的SDD：每个属性都是综合属性
- 都是根据子构造的属性计算出父构造的属性
- 在依赖图中，总是通过子结点的属性值来计算父结点的属性值。可以和自顶向下、自底向上的语法分析过程一起计算
 - 自底向上
 - 在构造分析树的结点的同时计算相关的属性（此时其子结点的属性必然已经计算完毕）
 - 自顶向下
 - 递归下降分析中，可以在过程 $A()$ 的最后计算A的属性（此时A调用的其他过程（对应于子结构）已经调用完毕）



语法制导定义的基本概念



- 只包含综合属性的SDD称为S属性的SDD
 - 每个语义规则都根据产生式体中的属性值来计算头部非终结符号的属性值
- S属性的SDD可以和LR语法分析器一起实现
 - 栈中的状态可以附加相应的属性值
 - 在进行归约时，按照语义规则计算归约得到的符号的属性值
- 语义规则不应该有复杂的副作用
 - 要求副作用不影响其它属性的求值
 - 没有副作用的SDD称为属性文法



语法制导定义的基本概念



- S属性文法（只有综合属性）一定不会有环（因为信息只往上走）
- L属性文法（继承 + 综合，受限条件）也能保证不存在环路（因为继承只依赖父或左兄弟）
- 属性的继承/综合划分，帮助我们构造 S 属性文法与 L 属性文法，这两类文法天然避免了依赖图中的环

S文法与L文法是对属性依赖的限制，它们是“子集”，设计上就是为了保证可行的无环计算顺序



语法制导定义的基本概念



- L属性的SDD：每个属性
 - 要么是综合属性
 - 要么是继承属性，且产生式 $A \rightarrow X_1X_2 \cdots X_n$ 中计算 X_i .a的规则只能使用
 - A的继承属性
 - X_i 左边的文法符号 X_j 的继承属性或综合属性
 - X_i 自身的继承或综合属性，且这些属性之间的依赖关系不形成环
- 特点
 - 依赖图的边
 - 继承属性从左到右，从上到下
 - 综合属性从下到上
 - 在扫描过程中，计算一个属性值时，和它相关的依赖属性都已经计算完毕



语法制导定义的基本概念



- 语义规则 (Semantic Rules)
 - 定义：语义规则是抽象层次上的定义，用于描述属性如何从文法符号传播、计算和约束；它通常写在属性文法 (Attribute Grammar, AG) 的形式化体系中
- 语义动作 (Semantic Actions)
 - 定义：语义动作是实现层次上的指令，通常嵌入在文法产生式中的可执行代码，在语法分析过程中被触发执行

语义规则：逻辑层面的“应当如何计算属性”

语义动作：实现层面的“在编译器里怎么做”

语义规则 = 数学公式

语义动作 = 代码实现



语法制导定义的基本概念



- 具有受控副作用的语义规则
- 属性文法没有副作用，但增加了描述的复杂度
 - 比如语法分析时如果没有副作用，标识符表就必须作为属性传递
 - 可以把标识符表作为全局变量，然后通过副作用函数来添加新标识符



语法制导定义的基本概念



- 为什么不能随意放置副作用？

$D \rightarrow T \text{ id}$
 // 意图：把 id 加入符号表，类型就是 T.type
 动作：addType(id.name, T.type)

- 合法情形（受控）：addType 在 T.type 已计算完之后、并且在无法回溯或取消的上下文中执行
- 非法情形（不受控）：在存在回溯的解析器里（如带回溯的递归下降/PEG 回溯）直接执行 addType，若解析失败回溯，会导致符号表状态与解析状态不同步 → 需要回滚或缓冲



语法制导定义的基本概念



- 副作用 (Side effect)：语义规则执行时对外部运行时或全局状态的修改，例如：
 - 修改符号表 (insert、update)
 - 生成/输出中间代码或目标代码 (emit)
 - 写日志/报错 (print/error)
 - 修改全局计数器、序号等
- 核心问题：副作用如果位置不当或在不确定的求值顺序下执行，会导致不一致或难以重现的语义错误。



语法制导定义的基本概念



- 具有受控副作用的语义规则
- 属性文法没有副作用，但增加了描述的复杂度
 - 比如语法分析时如果没有副作用，标识符表就必须作为属性传递
 - 可以把标识符表作为全局变量，然后通过副作用函数来添加新标识符
- 受控的副作用需要满足的条件：
 - 先行条件已满足：动作执行时，它所依赖的所有属性/信息均已被正确计算。
 - 执行顺序确定：副作用的执行位置在语法/语义规则中明确，与解析策略（自顶向下或自底向上）一致。
 - 不会改变依赖图的合法性（不会引入新的循环依赖或破坏属性求值结果）。
- 若任一条件不满足，副作用就不受控，可能引发错乱（例如重复插入、提前生成错误的代码）



语法制导定义的基本概念



- 受控副作用的例子
- $L \rightarrow E \text{ n} \quad \text{print}(E.\text{val})$
 - 通过副作用打印出E的值
 - 总是在最后执行，而且不会影响其它属性的求值
- 变量声明的SDD中的副作用
 - addType将标识符的类型信息加入到标识符表中
 - 只要标识符不被重复声明，标识符的类型信息总是正确的

产生式	语义规则
1) $D \rightarrow T L$	$L.inh = T.type$
2) $T \rightarrow \text{int}$	$T.type = \text{integer}$
3) $T \rightarrow \text{float}$	$T.type = \text{float}$
4) $L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $\text{addType}(\text{id.entry}, L.inh)$
5) $L \rightarrow \text{id}$	$\text{addType}(\text{id.entry}, L.inh)$



抽象语法树与类型



- 抽象语法树的构造
- 基本类型和数组类型的L属性定义



抽象语法树与类型



抽象语法树（ Abstract Syntax Tree ）

VS.

具象语法树（ Concrete Syntax Tree ）



抽象语法树与类型



- 具象语法树（ Concrete Syntax Tree ）：按照文法规则推导得到的完整树，精确反映文法的层次结构
 - 特点：包含所有非终结符与终结符；结构冗余，贴近文法；强调推导过程
- 抽象语法树（ Abstract Syntax Tree ）：去除了文法中冗余的语法细节，只保留与程序语义相关的核心结构
 - 特点：节点表示语言的核心成分（如“加法”、“变量”）；忽略不必要的非终结符（如括号层次、文法辅助符号）；更接近于编译器内部的中间表示。



抽象语法树与类型



特性	具体语法树（CST）	抽象语法树（AST）
目的	展示文法推导过程	表达程序语义的核心结构
节点内容	所有非终结符和终结符	语义相关的构造，如运算符、标识符
冗余性	高，结构庞大	精简，去掉无用信息
应用场景	理论教学，文法验证	编译器前端，中间代码生成

CST = 全部语法细节（就像考试解题步骤）
 AST = 最终简化结果（就像答案公式）
 在编译器中，AST 是后续 语义分析、优化和代码生成的核心输入



抽象语法树与类型



- 语法树（CST）：层次复杂，包含大量冗余符号。
- 抽象语法树（AST）：精简，直接反映程序的语义结构。
- 在 AST 上进行属性计算，效率更高，逻辑更清晰。
- 应用场景：
 - 类型检查（type checking）
 - 作用域/符号表管理
 - 常量折叠、优化
 - 中间代码生成



抽象语法树与类型



- 抽象语法树
 - 每个结点代表一个语法结构；对应于一个运算符
 - 结点的每个子结点代表其子结构；对应于运算分量
 - 表示这些子结构按照特定方式组成了较大的结构
 - 可以忽略掉一些标点符号等非本质的东西
- 构造抽象语法树的SDD
- 语法树的表示方法
 - 每个结点用一个对象表示
 - 对象有多个域
 - 叶子结点中只存放词法值
 - 内部结点中存放了op值和参数（通常指向其它结点）



抽象语法树与类型



- 在 SDT 中语义动作不仅用于生成 AST，还可：
 - 在遇到声明语句 时：向符号表插入条目
 - 在遇到变量使用 时：查询符号表进行检查
- 在 AST 中每个 标识符结点 都需要绑定符号表中的条目
- 符号表与 AST 协同工作：
 - AST 管理结构（树形层次）
 - 符号表管理名字（作用域与属性）



抽象语法树与类型



- 示例：

```
int x;
int f(int y) {
    return x + y;
}
```

- AST 负责表达式树形结构
- 符号表记录：x:int（全局），y:int（局部），f:function
- **符号表是语法树之外的“语义支撑系统”**



抽象语法树与类型



- 符号表的基本概念
- 定义
 - 符号表是编译器中用于记录源程序中各种符号（变量、函数、类型等）的数据结构。
 - 本质上是一个映射表（名称 → 属性）
- 作用
 - 在语义分析阶段，保存标识符的语义信息
 - 在后续的 中间代码生成、优化、目标代码生成 阶段提供支持。



抽象语法树与类型



- 符号表的基本概念
- 记录的信息
 - 符号名称（Identifier）
 - 类型信息（Type）
 - 作用域（Scope）
 - 存储分配（Memory Location）
 - 其他属性（如函数参数列表、是否为常量等）



抽象语法树与类型



- 符号表的基本概念
- 符号表的用途
 - 检查语义合法性
 - 变量是否声明后使用？
 - 类型是否匹配？
 - 作用域管理
 - 支持嵌套作用域（如函数、代码块）
 - 支持代码生成
 - 为每个变量分配存储空间或寄存器
 - 优化辅助
 - 确定变量的生命周期，进行寄存器分配或常量传播



抽象语法树与类型



- 符号表的基本概念
- 常见实现方式
 - 哈希表（Hash Table）：查找效率高（O(1)）
 - 链表（Linked List）：适合作用域嵌套结构
 - 树（如平衡树）：适合需要有序遍历的情况
- 支持作用域的设计
 - 单表 + 作用域链：符号表栈（stack of tables）
 - 进入新作用域 → 新建符号表；退出作用域 → 弹出符号表



四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G，其开始符号为 S，并且包

含以下四个产生式。小数部分可以为空，即 123 是合法的四

进制小数。产生式如下：

G(S): $S \rightarrow Q_1 \mid Q_1 Q_2$
 $Q_1 \rightarrow Q_1 \text{Num} \mid \text{Num}$
 $Q_2 \rightarrow \text{Num} Q_2 \mid \text{Num}$
 $\text{Num} \rightarrow 0 \mid 1 \mid 2 \mid 3$

1.（5 分）文法 G 中包含左递归，请消除左递归；

2.（5 分）请设计一个语法制导翻译方案来统计四进制小数中奇数数字（1 和 3）的个数。

例如 2.123 中奇数数字个数为 2；

3.（5 分）请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123

转换的二进制字符串为“10.011011”。

得分	
评分人	



四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G，其开始符号为 S，并且包

含以下四个产生式。小数部分可以为空，即 123 是合法的四

进制小数。产生式如下：

G(S): $S \rightarrow Q_1 \mid Q_1 Q_2$
 $Q_1 \rightarrow Q_1 \text{Num} \mid \text{Num}$
 $Q_2 \rightarrow \text{Num} Q_2 \mid \text{Num}$
 $\text{Num} \rightarrow 0 \mid 1 \mid 2 \mid 3$

1.（5 分）文法 G 中包含左递归，请消除左递归；

2.（5 分）请设计一个语法制导翻译方案来统计四进制小数中奇数数字（1 和 3）的个数。

例如 2.123 中奇数数字个数为 2；

3.（5 分）请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123

转换的二进制字符串为“10.011011”。

得分	
评分人	

2. 统计四进制数字中的奇数数字的个数的语义规则如下：

$\text{Num} \rightarrow 1 \mid 3: \text{Num.count} = 1$

$\text{Num} \rightarrow 0 \mid 2: \text{Num.count} = 0$

$Q_1 \rightarrow \text{Num} Q_1': Q_1.\text{count} = \text{Num.count} + Q_1'.\text{count}$

$Q_1' \rightarrow \text{Num} Q_1': Q_1'.\text{count} = \text{Num.count} + Q_1'.\text{count}$

$Q_1' \rightarrow \epsilon: Q_1'.\text{count} = 0$

$Q_2 \rightarrow \text{Num} Q_2: Q_2.\text{count} = \text{Num.count} + Q_2.\text{count}$

$Q_2 \rightarrow \text{Num}: Q_2.\text{count} = \text{Num.count}$



四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G，其开始符号为 S，并且包

含以下四个产生式。小数部分可以为空，即 123 是合法的四

进制小数。产生式如下：

G(S): $S \rightarrow Q_1 \mid Q_1 Q_2$
 $Q_1 \rightarrow Q_1 \text{Num} \mid \text{Num}$
 $Q_2 \rightarrow \text{Num} Q_2 \mid \text{Num}$
 $\text{Num} \rightarrow 0 \mid 1 \mid 2 \mid 3$

1.（5 分）文法 G 中包含左递归，请消除左递归；

2.（5 分）请设计一个语法制导翻译方案来统计四进制小数中奇数数字（1 和 3）的个数。

例如 2.123 中奇数数字个数为 2；

3.（5 分）请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123

转换的二进制字符串为“10.011011”。

得分	
评分人	

1. 消除文法 G 的左递归，就是消除 Q_1 的左递归：

$Q_1 \rightarrow \text{Num} Q_1'$

$Q_1' \rightarrow \text{Num} Q_1' \mid \epsilon$

更新后的文法 G(S)如下：

G(S): $S \rightarrow Q_1 \mid Q_1 Q_2$

$Q_1 \rightarrow \text{Num} Q_1'$

$Q_1' \rightarrow \text{Num} Q_1' \mid \epsilon$

$Q_2 \rightarrow \text{Num} Q_2 \mid \text{Num}$

$\text{Num} \rightarrow 0 \mid 1 \mid 2 \mid 3$



四、简答题（共 1 题，共 15 分）

给定一个表示四进制小数的文法 G，其开始符号为 S，并且包

含以下四个产生式。小数部分可以为空，即 123 是合法的四

进制小数。产生式如下：

G(S): $S \rightarrow Q_1 \mid Q_1 Q_2$
 $Q_1 \rightarrow Q_1 \text{Num} \mid \text{Num}$
 $Q_2 \rightarrow \text{Num} Q_2 \mid \text{Num}$
 $\text{Num} \rightarrow 0 \mid 1 \mid 2 \mid 3$

1.（5 分）文法 G 中包含左递归，请消除左递归；

2.（5 分）请设计一个语法制导翻译方案来统计四进制小数中奇数数字（1 和 3）的个数。

例如 2.123 中奇数数字个数为 2；

3.（5 分）请设计一个语法制导翻译方案将四进制小数转换为二进制字符串。例如 2.123

转换的二进制字符串为“10.011011”。

得分	
评分人	

3. 将四进制小数转换为二进制字符串形式的语义规则如下：

$\text{Num} \rightarrow 0: \text{Num.bin} = "00"$

$\text{Num} \rightarrow 1: \text{Num.bin} = "01"$

$\text{Num} \rightarrow 2: \text{Num.bin} = "10"$

$\text{Num} \rightarrow 3: \text{Num.bin} = "11"$

$Q_1 \rightarrow \text{Num} Q_1': Q_1.\text{bin} = \text{Num.bin} + Q_1'.\text{bin}$

$Q_1' \rightarrow \text{Num} Q_1': Q_1'.\text{bin} = \text{Num.bin} + Q_1'.\text{bin}$

$Q_1' \rightarrow \epsilon: Q_1'.\text{bin} = ""$

$Q_2 \rightarrow \text{Num} Q_2: Q_2.\text{bin} = \text{Num.bin} + Q_2.\text{bin}$

$Q_2 \rightarrow \text{Num}: Q_2.\text{bin} = \text{Num.bin}$



题目 5 (语法制导定义与翻译 (10=5+5 分))

已知程序的文法 G[P]如下：

$P \rightarrow D$

$D \rightarrow D; D \mid \text{id}; T \mid \text{proc id}; D; S$

(1) 请写一个语法制导定义，输出程序中一共声明了多少

(2) 请写一个翻译模式，输出程序中每个变量 id 的嵌套深

解答：

(1)

产生式	语义规则
$P \rightarrow D$	$\text{Print}(D.c)$
$D \rightarrow D1; D2$	$D.c = D1.c + D2.c$
$D \rightarrow \text{id}; T$	$D.c = 1$
$D \rightarrow \text{proc id}; D1; S$	$D.c = D1.c + 1$

(2)

$P \rightarrow \{D.d = 1\} D$

$D \rightarrow \{D1.d = D.d\} D1; \{D2.d = D.d\} D2$

$D \rightarrow \text{id}; T \{\text{print}(D.d)\}$

$D \rightarrow \text{proc id}; \{D1.d = D.d + 1\} D1; S$

题目 5 (语法制导定义与翻译 (10=5+5 分))

已知程序的文法 G[P]如下：

$P \rightarrow D$

$D \rightarrow D; D \mid \text{id}; T \mid \text{proc id}; D; S$

(1) 请写一个语法制导定义，输出程序中一共声明了多少个 id。

(2) 请写一个翻译模式，输出程序中每个变量 id 的嵌套深度。



复习课（四） （中间代码表示）

冯洋

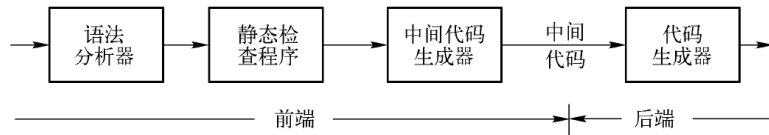
南京大学计算机学院

fengyang@nju.edu.cn



中间代码表示

- 中间表示（Intermediate Representation, IR）:编译器在前端与后端之间使用的、与具体源语言和目标机器都相对独立的程序表示形式，用于承载程序语义、驱动优化，并作为目标代码的输入



中间代码表示

- IR设计与分析的关键维度
 - 控制流表示
 - 结构化（如高层循环构造）vs 显式跳转与标签（基本块图）
 - 数据表示
 - 值模型：三地址临时 vs 栈操作数（堆栈式IR）
 - 赋值模型：普通赋值 vs 静态单赋值（SSA，单点写，多点读）
 - 内存与地址
 - 显式load/store vs 隐式求值
 - 别名与指针是否可见、是否区分地址和值（l-value/r-value）
 - 类型与语义信息
 - 强类型IR（带精确类型与对齐）vs 弱类型/无类型
 - 高层语义保留程度（异常、越界检查、GC屏障、并发原语）
 - 可分析性与可变换性
 - 是否便于构建CFG/DFG、做数据流分析、回填与短路求值



中间代码表示

五、简答题（共1题，共10分）

1.（5分）在图1中间代码翻译方案中添加处理如下控制流构造的规则（注意布尔表达式之前的否运算）：

$S \rightarrow \text{for}(S_1; !B; S_2) S_3$

2.（5分）结合图2布尔表达式的翻译方案，翻译

如下代码（第5到第7行）：

要求：画出语法分析树，并在每个节点上标注相应的属性，给出最终的中间代码。

```

(1) int i;
(2) int n = 100;
(3) int sum = 0;
(4)
(5) for (i = 0; i < n && sum < 1000; i = i + 1) {
(6)   sum = sum + i;
(7) }
  
```

得分	评分人

产生式	语义规则
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign}.code$
$S \rightarrow \text{if}(B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if}(B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code \parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while}(B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code \parallel label(B.true) \parallel S_1.code \parallel gen(begin.next)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_2.next) \parallel S_2.code$



中间代码表示

- 高层IR（高抽象、语义保留多）
 - 接近源语言结构：函数、循环、异常、对象、闭包等仍显式存在
 - 优点：便于高层语义相关优化（如内联、异常消除、对象去虚拟化）
 - 缺点：与目标机器距离远，难以直接选址与寄存器分配
- 中层IR（常见）
 - 典型：三地址码（TAC）、四元式、SSA形态的TAC
 - 权衡：足够接近机器以便分析，又保留必要结构信息
- 低层IR（低抽象、接近机器）
 - 显式寄存器/栈、分支、加载/存储、调用约定
 - 优点：易映射到目标指令；缺点：优化空间受限



中间代码表示

- 常见IR范式与代表
 - 树形IR：抽象语法树（AST）、语法树-语义注释
 - 适合“早期”高层变换，不适合低层优化
 - 三地址码（TAC）： $x = y \text{ op } z$; $\text{if } x \text{ goto } L$; $\text{param } x$; $\text{call } f, n$; $\text{return } x$
 - 表示方式：四元式（op, arg1, arg2, result）、三元式（op, arg1, arg2）
 - SSA（静态单赋值）形态的IR：在TAC基础上引入 Φ 函数
 - 优化友好：简化数据流问题，如活跃性、可用表达式、冗余消除
 - 堆栈式IR：操作数隐含在栈上（如JVM字节码）
 - 简单、紧凑，但数据依赖不显式，部分优化不便
 - LLVM IR/Sea-of-Nodes 等工业IR
 - 应用极其广泛



三地址码



- 定义与概述：三地址码是一种常见的中间表示形式，它将源程序转换为一系列简单的指令，每条指令最多涉及三个“地址”（操作数或结果）
- 典型格式为：**result = arg1 op arg2**，其中op是操作符，arg1和arg2是操作数，result是结果变量
 - “地址”可以是常量、变量、临时变量或标签
 - 它类似于低级汇编语言，但独立于具体机器架构，便于编译器后端处理



三地址码



- 定义与概述：三地址码是一种常见的中间表示形式，它将源程序转换为一系列简单的指令，每条指令最多涉及三个“地址”（操作数或结果）
- 在编译器中的位置：三地址码通常从抽象语法树（AST）生成，是前端（语法/语义分析）与后端（优化/目标代码生成）之间的桥梁
- 为什么叫“三地址”？因为大多数指令模拟三操作数机器的运算（如加法：**t1 = a + b**），相比更高级的IR（如树状表示），它更线性且易于翻译成机器码



三地址码



- 每条指令右侧最多有一个运算符
 - 一般情况可以写成 $x = y \text{ op } z$
- 允许的运算分量
 - 名字：源程序中的名字作为三地址代码的地址
 - 常量：源程序中出现或生成的常量
 - 编译器生成的临时变量



三地址码



三地址码实例

语句

do i = i + 1; while (a[i] < v);

```
L:  t1 = i + 1
    i = t1
    t2 = i * 8
    t3 = a [ t2 ]
    if t3 < v goto L
```

a) 符号标号

```
100: t1 = i + 1
101: i = t1
102: t2 = i * 8
103: t3 = a [ t2 ]
104: if t3 < v goto 100
```

b) 位置号



三地址码



- 在实现时，可以使用**四元式/三元式/间接三元式**来表示三地址指令
- 四元式：可以实现为纪录（或结构）
- 格式（字段）：**op arg1 arg2 result**
 - op: 运算符的内部编码
 - arg1, arg2, result是地址
 - $x = y + z$ + y z x
- 单目运算符不使用arg2
- param运算不使用arg2和result
- 条件转移/非条件转移将目标标号放在result字段



三地址码



四元式的实例

- 赋值语句： $a = b * -c + b * -c$

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

a) 三地址代码

	op	arg ₁	arg ₂	result
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
				...

b) 四元式



三地址码

- 三元式表示 triple op arg1 arg2
- 使用三元式的位置来引用三元式的运算结果
- $x[i]=y$ 需要拆分为两个三元式
 - 求 $x[i]$ 的地址, 然后再赋值
- $x=y$ op z 需要拆分为 (这里? 是编号)
 - (?) op y z
 - = x ?

- 问题: 在优化时经常需要移动/删除/添加三元式, 导致三元式的移动, 应该如何处理?



三地址码

- 间接三元式: 包含了一个指向三元式的指针的列表
- 我们可以对这个列表进行操作, 完成优化功能; 操作时不需要修改三元式中的参数

instruction	
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)
	...

	op	arg ₁	arg ₂
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
		...	



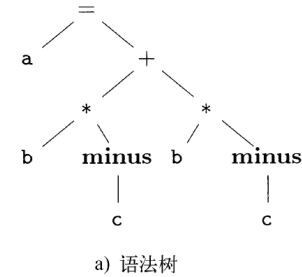
三地址码

- 三地址码的指令类型
 - 函数调用与参数: 处理过程调用
 - 格式: param x (传递参数); call func, n (调用函数, n参数个数); x = call func (带返回值)
 - 示例: param sum; call printf, 1
 - 数组/指针访问: 地址计算
 - 格式: $x = y[i]$ 或 $x[i] = y$; 需计算偏移, 如 $t1 = i * \text{width}$; $x = y + t1$
 - 示例: 对于数组 $a[5]$, $t1 = i * 4$; $t2 = a + t1$; $val = t2$ (假设int类型4字节)
 - 标签: 标记代码位置
 - 格式: L: (



三地址码

- 三元式的实例



b) 三元式

	op	arg ₁	arg ₂
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
		...	



三地址码

- 三地址码的指令类型
 - 算术/逻辑运算: 用于表达式计算
 - 格式: $x = y$ op z (op 如 +, -, *, /, &&, || 等)
 - 示例: $t1 = a + b$ (加法); $t2 = t1 * c$ (乘法, 使用临时变量链式处理)
 - 赋值与拷贝: 简单数据移动
 - 格式: $x = y$ (单地址) 或 $x = \text{unary_op } y$ (如 $x = -y$)
 - 示例: $\text{sum} = 0$ (初始化)
 - 控制流指令: 处理分支和循环
 - 无条件跳转: goto L (L为标签)
 - 条件跳转: if x relop y goto L (relop 如 <, >, ==)
 - 示例: if $t1 <= 10$ goto L1 (循环条件)



三地址码

- 随堂测试题目: $x = (a + b) * (c - d) + e$
写出这个式子的三元式、间接三元式、四元式和AST树



三地址码



- 随堂测试题目: $x = (a + b) * (c - d) + e$

写出这个式子的三元式、间接三元式、四元式和AST树

序号	op	arg1	arg2	result	
(1)	+	a	b	t1	$ \begin{array}{c} = \\ / \ \backslash \\ x \quad + \\ / \ \backslash \\ * \quad e \\ / \ \backslash \\ (+) \ (-) \\ / \ \backslash \ / \ \backslash \\ a \ b \ c \ d \end{array} $
(2)	-	c	d	t2	
(3)	*	t1	t2	t3	
(4)	+	t3	e	t4	
(5)	=	t4	-	x	



三地址码



- 随堂测试题目: $x = (a + b) * (c - d) + e$

写出这个式子的三元式、间接三元式、四元式和AST树

序号	op	arg1	arg2	序号	op	arg1	arg2
(1)	+	a	b	(1)	+	a	b
(2)	-	c	d	(2)	-	c	d
(3)	*	(1)	(2)	(3)	*	(0)	(1)
(4)	+	(3)	e	(4)	+	(2)	e
(5)	=	(4)	x	(5)	=	(3)	x

0 → (1)
 1 → (2)
 2 → (3)
 3 → (4)
 4 → (5)

(注意: 这里 (0) 和 (1) 表示索引表里的引用, 而不是直接的三元式编号, 便于优化和重排。)



静态单赋值



- 静态单赋值形式 (Static Single Assignment, SSA) 是编译器设计中用于优化的中间表示形式, 其核心规则要求每个变量仅被赋值一次, 通过添加下标区分变量不同版本
- 静态单赋值形式被广泛应用于中间代码表示, 使用SSA可以很容易地发现一些可优化项, 如死代码优化、常量传播等
- 优点:
 - 消除了变量重赋值带来的混淆
 - 数据流清晰, 每个使用点对应唯一的定义点
 - 大大简化编译器的优化 (如常量传播、死代码删除、寄存器分配)



静态单赋值



- 1981 年: 由 Cytron 等人在论文 Efficiently Computing Static Single Assignment Form and the Control Dependence Graph 中提出
 - 初衷: 为优化数据流分析, 引入“单赋值”思想, 但控制流合流点需要额外机制, 因此发明了 ϕ 函数
 - 后续: SSA 成为工业级编译器的核心技术 (LLVM、GCC、Java HotSpot、Rustc 都使用 SSA 或变体)



静态单赋值



- SSA中的所有赋值都是针对不同名的变量
- 对于同一个变量在不同路径中定值的情况, 可以使用 ϕ 函数来合并不同的定值

```

if (flag)
    x = -1;
else
    x = 1;

y = x*a
  
```



静态单赋值



- SSA中的所有赋值都是针对不同名的变量
- 对于同一个变量在不同路径中定值的情况, 可以使用 ϕ 函数来合并不同的定值

```

if (flag)
    x = -1;
else
    x = 1;

y = x*a

if (flag)
    x1=-1;
else
    x2 = 1;

x3=phi(x1,x2);
y = x3*a
  
```



静态单赋值

- SSA中的所有赋值都是针对不同名的变量
- 对于同一个变量在不同路径中定值的情况，可以使用 ϕ 函数来合并不同的定值

```
if (flag)
    x = -1;
else
    x = 1;

y = x*a
```

```
if (flag)
    x1=-1;
else
    x2 = 1;

x3= $\phi(x1, x2)$ ;
y = x3*a
```



静态单赋值

代码层实现策略

- (1) 插入并行赋值 (idealized semantics)：在 IR 层面，可以把 ϕ 看作所有赋值同时发生：


```
if from pred1: x3 := x1
if from pred2: x3 := x2
```
- (2) 寄存器重命名 / 移动指令：在真实机器代码生成时，常见做法是在合流点前，插入 `move` 指令，保证合流点有正确的版本：


```
L1:
    x1 = 1
    goto L3_pre

L2:
    x2 = 2
    goto L3_pre

L3_pre:
    x3 = (pred == L1 ? x1 : x2) ; 插入 move
    goto L3

L3:
    x3 =  $\phi(x1, x2)$ 
    y1 = x3 + 3
```



静态单赋值

ϕ 函数在循环中的功能

```
int factorial(int n) {
    int result = 1;
    while (n > 1) {
        result = result * n;
        n = n - 1;
    }
    return result;
}
```

初始化`result=1`，然后在`while`循环中迭代更新`result`和`n`，直到`n<=1`。这是一个典型的后测试循环（do-while语义，但用`while`实现），迭代变量`n`和`result`在循环中被更新。

```
# Entry
result = 1
goto L_header

# Header
L_header:
t1 = n > 1
if t1 goto L_body else goto L_exit

# Body
L_body:
t2 = result * n
result = t2
t3 = n - 1
n = t3
goto L_header

# Exit
L_exit:
return result
```

非SSA形式的三地址码

```
Entry: result = 1
      goto Header

Header: if (n > 1) goto Body
      else goto Exit

Body: result = result * n
      n = n - 1
      goto Header

Exit: return result
```



静态单赋值

ϕ 函数的本质

- 数学意义：一个“多路选择器”（类似 `switch`）。
- 语义：在控制流合流点，根据 控制路径 决定选哪个值。
- 形式： $x_{n+1} = \phi(x_1, x_2, \dots, x_n)$
- ϕ 不是 CPU 指令，而是编译器 IR 的“抽象伪指令”。在生成汇编代码时，**必须把它消除（ ϕ -elimination）**



静态单赋值

代码层实现策略

- (3) LLVM 中的实现
 - LLVM 在其“InsertCopiesForPHIs”阶段，会为每个 ϕ 在每个前驱生成一个 `move`（放在前驱的末尾或拆分后的插入块）。它使用与上面类似的并行复制分解算法，并分配临时 `virtual registers`。随后，`register allocator` 会尽量为不冲突的版本分配同一寄存器以消除复制（`coalescing`）。
 - GCC (GIMPLE) 也有类似流程：GIMPLE 使用 SSA 表示，最终生成 RTL 时会做 ϕ 展开并插入复制



静态单赋值

ϕ 函数在循环中的功能

```
int factorial(int n) {
    int result = 1;
    while (n > 1) {
        result = result * n;
        n = n - 1;
    }
    return result;
}
```

初始化`result=1`，然后在`while`循环中迭代更新`result`和`n`，直到`n<=1`。这是一个典型的后测试循环（do-while语义，但用`while`实现），迭代变量`n`和`result`在循环中被更新。

```
# Entry
result_0 = 1
n_0 = n # 假设输入参数n
goto L_header

# Header
L_header:
result_1 =  $\phi(result\_0, result\_2)$  # 从Entry取result_0, 从Body取result_2
n_1 =  $\phi(n\_0, n\_2)$  # 从Entry取n_0, 从Body取n_2
t1 = n_1 > 1
if t1 goto L_body else goto L_exit

# Body
L_body:
t2 = result_1 * n_1
result_2 = t2
t3 = n_1 - 1
n_2 = t3
goto L_header

# Exit
L_exit:
result_3 =  $\phi(result\_1)$  # 从Header取（无其他路径）
return result_3
```

在while循环中，迭代变量（如`n`和`result`）有“初始值”（从Entry）和“更新值”（从Body反馈）。 ϕ 在Header处合并它们：第一次迭代用初始值（`n_1 = n_0`），后续迭代用更新值（`n_1 = n_2`）

在SSA中，每个变量只赋值一次，使用下标表示版本（e.g., `result_0`, `result_1`）。 ϕ 函数置于合并点（Header块开头），形式为`x_i =  $\phi(x_j, x_k, \dots)$` ，根据前驱块动态选择值。
注意：在实际SSA（如LLVM IR）中， ϕ 函数指定前驱：e.g., `result_1 =  $\phi$  result_0, Entry, [result_2, Body]`，这里简化表示。

SSA表示



控制流语句翻译

- if-else语句, while语句
- 需要将语句的翻译和布尔表达式的翻译结合在一起
- 布尔表达式是被用作语句中改变控制流的条件表达式, 通常用来
 - 改变控制流。布尔表达式的值由程序到达的某个位置隐含地指出。
 - 计算逻辑值。可以使用带有逻辑运算符的三地址指令进行求值。
- 布尔表达式的使用意图要根据其语法上下文确定
 - 跟在关键字if后面的表达式用来改变控制流
 - 一个赋值语句右部的表达式用来计算一个逻辑值
 - 可以使用两个不同的非终结符号或其它方法来区分这两种使用



控制流语句翻译

- 布尔表达式的高效求值
 - B1 || B2, B1为真, 则不用求B2也能断定整个表达式为真
 - B1 && B2, B1为假, 则整个表达式肯定为假
 - 如果某些程序设计语言允许这种高效的求值方式, 则编译器可以优化布尔表达式的求值过程, 只要已经求值部分足以确定整个表达式的值就可以了。
- 短路(跳转)代码
 - 布尔运算符&&、||、!被翻译成跳转指令。由跳转位置隐含的指出布尔表达式的值。
 - if(x<100 || x>200 && x!=y) x=0;

```

if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2:  x = 0
L1:
    
```

图 6-34 跳转代码



控制流语句翻译

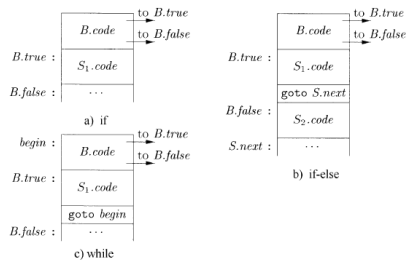
- 语句及文法

$$S \rightarrow \text{if } (B) S_1$$

$$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$$

$$S \rightarrow \text{while } (B) S_1$$

- B和S有综合属性code, 表示翻译得到的三地址代码。
- B的继承属性true和false, S的继承属性next, 表示跳转的位置。



控制流语句翻译

- 布尔表达式
 - 将布尔运算符作用在布尔变量或关系表达式上, 构成布尔表达式
 - 引入新的非终结符号B表示布尔表达式
 - 布尔运算符: &&、||、!
 - 关系表达式 E1 rel E2
 - 关系运算符: <、<=、=、!=、>、>=
 - 其中布尔运算符&&和||是左结合的, 优先级||最低, 其次是&&, !最高
 - 表示布尔表达式的文法

$$B \rightarrow B \mid B \mid B \&\& B \mid ! B \mid (B) \mid E \text{ rel } E \mid \text{true} \mid \text{false}$$



控制流语句翻译

- 语句及文法
 - B和S有综合属性code, 表示翻译得到的三地址代码。
 - B的继承属性true和false, S的继承属性next, 表示跳转的位置。
 - S.true是一个地址, 用于存放了B为真时控制流转向的指令标号, S.false同理
 - S.next也是一

$$S \rightarrow \text{if } (B) S_1$$

$$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$$

$$S \rightarrow \text{while } (B) S_1$$



控制流语句翻译

- 几个核心函数回顾:
 - newlabel 函数: 生成一个用于存放标号的临时变量L, 返回变量地址;
 - label(X)函数: 将下一条指令的标号复制给参数X;
 - 翻译S → if (B) S1, 创建B.true标号, 并指向S1的第一条指令。
 - 翻译S → if (B) S1 else S2, B为真时, 跳转到S1代码的第一条指令; 当B为假时跳转到S2代码的第一条指令。然后, 控制流从S1或S2转到紧跟在S的代码后面的三地址指令, 该指令由继承属性S.next指定。
 - while语句中有个begin局部变量
 -

产生式	语法规则
$P \rightarrow S$	$S.next = \text{newlabel}()$ $P.code = S.code \parallel \text{label}(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign.code}$
$S \rightarrow \text{if } (B) S_1$	$B.true = \text{newlabel}()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel \text{label}(B.true) \parallel S_1.code$
$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$	$B.true = \text{newlabel}()$ $B.false = \text{newlabel}()$ $S_1.next = S_2.next = S.next$ $S.code = B.code \parallel \text{label}(B.true) \parallel S_1.code \parallel \text{label}(B.false) \parallel S_2.code$
$S \rightarrow \text{while } (B) S_1$	$\text{begin} = \text{newlabel}()$ $B.true = \text{newlabel}()$ $B.false = S.next$ $S_1.next = \text{begin}$ $S.code = \text{label}(\text{begin}) \parallel B.code \parallel \text{label}(B.true) \parallel S_1.code \parallel \text{gen}(\text{'goto' } \text{begin})$
$S \rightarrow S_1 S_2$	$S_1.next = \text{newlabel}()$ $S_2.next = S.next$ $S.code = S_1.code \parallel \text{label}(S_1.next) \parallel S_2.code$



控制流语句翻译

■ 布尔表达式的控制流翻译及分析:

- 布尔表达式B被翻译成三地址指令，生成的条件或无条件转移指令反映B的值。
- B→E1 rel E2，直接翻译成三地址比较指令，跳转到正确位置。
- B→B1 || B2，如果B1为真，B一定为真，所以B1.true和B.true相同。因此B1.false指向B2的代码开始的位置。B2的真假出口分别等于B的真假出口。

产生式	语义规则
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel gen('if' E_1.addr \text{ rel } op E_2.addr \text{ goto } B.true)$ $\parallel gen('goto' B.false)$
$B \rightarrow \text{true}$	$B.code = gen('goto' B.true)$
$B \rightarrow \text{false}$	$B.code = gen('goto' B.false)$

○



控制流语句翻译

■ 布尔表达式及控制流语句翻译示例:

- 布尔表达式翻译, $a < b$
- if $a < b$ goto B.true
goto B.false

```

        if x < 100 goto L2
        goto L3
L3:     if x > 200 goto L4
        goto L1
L4:     if x != y goto L2
        goto L1
L2:     x = 0
L1:
    
```

控制流语句翻译 if ($x < 100 \parallel x > 200$
&& $x != y$) $x = 0$;



控制流语句翻译

■ 避免冗余的goto指令:

- 在上面的例子中goto L3是冗余的
- X>200翻译成 $\text{if } x > 200 \text{ goto } L_4$
goto L₁
L₄: ...
- 可以替换成 L₄: ...
 $\text{ifFalse } x > 200 \text{ goto } L_1$
- 减少了一条goto指令
- 引入一个特殊标号“fall”(穿越, fall through), 表示不要生成任何跳转指令。
- S→if (B) S1的新语义规则 $B.true = fall$
 $B.false = S_1.next = S.next$
 $S.code = B.code \parallel S_1.code$
- 对于if-else和while语句的规则也将B.true设为fall



控制流语句翻译

产生式	语义规则
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign.code}$
$S \rightarrow \text{if } (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' S.next)$ $\parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while } (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$



控制流语句翻译

■ 布尔表达式及控制流语句翻译示例:

- 布尔表达式翻译, $a < b$
- if $a < b$ goto B.true
goto B.false

```

        if x < 100 goto L2
        goto L3
L3:     if x > 200 goto L4
        goto L1
L4:     if x != y goto L2
        goto L1
L2:     x = 0
L1:
    
```

控制流语句翻译 if ($x < 100 \parallel x > 200$
&& $x != y$) $x = 0$;

思考, 是否有什么问题?



控制流语句翻译

■ 在布尔表达式中, && 与 || 需要 短路求值:

- B1 && B2: 若 B1 为假, 则直接判定为假, 不再计算 B2
- B1 || B2: 若 B1 为真, 则直接判定为真, 不再计算 B2

■ 翻译时:

- 通过 真假链 (true/false lists) 表示控制流
- 利用 fall-through (顺序进入下一条指令) 来避免生成多余跳转
- 穿越:
 - 对 &&: false 直接短路, true fall-through
 - 对 ||: true 直接短路, false fall-through



控制流语句翻译



- 核心问题：在SDT中，我们通过附加到文法产生式的语义动作（semantic actions）来生成中间代码。对于表达式（如 $a = b + c$ ），翻译是线性的（自底向上或自顶向下）。但对于控制流语句（如if-else、while），存在“未知跳转目标”的问题：
 - 条件分支：if条件真时跳到then部分，否则跳到else或下一语句。但在解析时，else或循环结束的地址未知
 - 循环：while的条件检查需跳回循环头或跳出循环，但循环体长度在解析时不确定



控制流语句翻译



- 回填：回填是一种延迟绑定技术，在SDT中用于处理控制流语句的跳转指令。具体来说：
 - 先生成不完整的三地址码跳转指令（如if t goto _ 或 goto _），其中“_”是占位符（实际实现中用链表或列表存储待回填位置）
 - 当解析到目标位置时，回溯并填充这些占位符为实际标签（labels，如L1、L2）



控制流语句翻译



- 布尔表达式的回填翻译
 - 布尔表达式用于语句的控制流时，它总是在取值true时和取值false时分别跳转到某个位置
 - 引入两个综合属性记录B的代码中跳转指令goto S.next, if ... goto S.next的位置，但是不生成跳转目标
 - truelist: 包含跳转指令（位置）的列表，这些指令在取值true时执行
 - falselist: 包含跳转指令（位置）的列表，这些指令在取值false时执行
 - 辅助函数
 - Makelist(i): 创建一个只包含i的列表
 - Merge(p1,p2): 将p1和p2指向的列表合并
 - Backpatch(p,i): 将i作为目标标号插入到p所指列表中的各指令中
 - 语义属性：在SDT文法中，为非终结符添加属性如{code, truelist, falselist, nextlist}



控制流语句翻译



- 标准SDT是“单遍”翻译（one-pass），但控制流引入“前向引用”（forward references），即跳转到尚未生成的代码位置。如果不处理，会导致代码生成中断或错误
- 解决方案：引入回填（backpatching）技术。它允许先生成带有“占位符”（placeholders）的跳转指令，然后在后续解析中“回填”实际地址或标签
- 回填的优势：保持SDT的效率，支持LALR(1)等单遍分析器；便于处理嵌套控制流



控制流语句翻译



- 回填：为布尔表达式和控制流语句生成目标代码的关键问题：某些跳转指令应该跳转到哪里
 - 例如：if (B) S
 - 按照短路代码的翻译方法，B的代码中有一些跳转指令在B为假时执行，这些跳转指令的目标应该跳过S对应的代码，生成这些指令时，S的代码尚未生成，因此目标不确定
 - 通过语句的继承属性next来传递。需要第二趟处理
 - 基本思想
 - 记录B的代码中跳转指令goto S.next, if ... goto S.next的位置，但是不生成跳转目标
 - 这些位置被记录到B的综合属性B.falseList中
 - 当S.next的值已知时（即S的代码生成完毕时），把S.nextList中的所有指令的目标都填上这个值
 - 回填技术
 - 生成跳转指令时暂时不指定跳转目标标号，而是使用列表记录这些不完整的指令
 - 等知道正确的目标时再填写目标标号
 - 每个列表中的指令都指向同一个目标



控制流语句翻译



- 我们看一个具体的例子！


```
if ((x < y) && (y < z) || (x == z))
    a = 1;
else
    a = 2;
```
- 翻译过程（带穿越）
 - 表达式 $(x < y) \&\& (y < z)$
 - 如果 $x < y$ 为假 $\rightarrow$ 整体为假（短路），跳转到 false
 - 如果 $x < y$ 为真 $\rightarrow$ fall-through，进入 $y < z$ 判断
 - 与 $(x == z)$ 的 $\parallel$
 - 如果 $(x < y \&\& y < z)$ 为真 $\rightarrow$ 整体为真（短路），跳转到 true
 - 否则继续判断 $(x == z)$



控制流语句翻译



■ 我们看一个具体的例子！

```
if ((x < y) && (y < z) || (x == z))
```

```
  a = 1;
```

```
else
```

```
  a = 2;
```

■ 翻译过程（带穿越）

- gen("if a rel b goto _") // quad i → truelist = {i}
- gen("goto _") // quad i+1 → falselist = {i+1}

对 B1 && B2:

1. 生成 B1 → 得到 B1.true, B1.false
2. M = nextquad(), backpatch(B1.true, M) (若 B1 为真, 跳到 B2)
3. 生成 B2 → B.true = B2.true, B.false = merge(B1.false, B2.false)

对 B1 || B2:

1. 生成 B1 → B1.true, B1.false
2. M = nextquad(), backpatch(B1.false, M) (若 B1 为假, 跳到 B2)
3. 生成 B2 → B.true = merge(B1.true, B2.true), B.false = B2.false



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))
```

```
  a = 1;
```

```
else
```

```
  a = 2;
```



■ 我们看一个具体的例子！

■ 完整思考流程:

- 初始化 nextquad = 1。我们按上面的模板生成代码并记录各个 list
- (1) 生成 x < y (记作 B1)
 - 1: if x < y goto _ // B1.true = {1}
 - 2: goto _ // B1.false = {2}
 - nextquad = 3
- (2) 要生成 B1 && B2, 先把 B1.true 指向 B2 开始位置 M = nextquad = 3
 - backpatch(B1.true, 3) // 设置 quad1 目标为 3
- (3) 生成 y < z (记作 B2), 从 quad 3 开始
 - 3: if y < z goto _ // B2.true = {3}
 - 4: goto _ // B2.false = {4}
 - nextquad = 5

现在 B_and = B1 && B2:

B_and.true = B2.true = {3}

B_and.false = merge(B1.false, B2.false) = {2,4}



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))
```

```
  a = 1;
```

```
else
```

```
  a = 2;
```



■ 我们看一个具体的例子！

■ 完整思考流程:

- 初始化 nextquad = 1。我们按上面的模板生成代码并记录各个 list
- (4) 处理 (B_and) || (x == z): 对 ||, 需要把左侧的 falselist 指向右侧开始 M2 = nextquad = 5
 - backpatch(B_and.false, 5) // 设置 quad2 和 quad4 的目标为 5
- (5) 生成右侧 x == z (记作 C), 从 quad 5 开始
 - 5: if x == z goto _ // C.true = {5}
 - 6: goto _ // C.false = {6}
 - nextquad = 7



控制流语句翻译



■ 我们看一个具体的例子！

```
if ((x < y) && (y < z) || (x == z))
```

```
  a = 1;
```

```
else
```

```
  a = 2;
```

■ 翻译过程（带穿越）

- 表达式 (x < y) && (y < z)
 - 如果 x < y 为假 → 整体为假（短路），跳转到 false
 - 如果 x < y 为真 → fall-through, 进入 y < z 判断
- 与 (x == z) 的 ||
 - 如果 (x < y && y < z) 为真 → 整体为真（短路），跳转到 true
 - 否则继续判断 (x == z)

(x < y)
true → fall-through 到 if y < z
false → 跳到 (x == z)

(y < z)
true → 直接跳到 then 部分
false → fall-through 到 (x == z)

(x == z)
true → 直接跳到 then 部分
false → 跳到 else 部分



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))
```

```
  a = 1;
```

```
else
```

```
  a = 2;
```



■ 我们看一个具体的例子！

■ 完整思考流程:

- 初始化 nextquad = 1。我们按上面的模板生成代码并记录各个 list
- (4) 处理 (B_and) || (x == z): 对 ||, 需要把左侧的 falselist 指向右侧开始 M2 = nextquad = 5
 - backpatch(B_and.false, 5) // 设置 quad2 和 quad4 的目标为 5

现在 B_and = B1 && B2:

B_and.true = B2.true = {3}

B_and.false = merge(B1.false, B2.false) = {2,4}



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))
```

```
  a = 1;
```

```
else
```

```
  a = 2;
```



■ 我们看一个具体的例子！

■ 完整思考流程:

- 初始化 nextquad = 1。我们按上面的模板生成代码并记录各个 list
- (6) 生成 then / else 代码并回填
 - 将 then 语句 (a = 1) 放在 nextquad = 7:
 - backpatch(overall.true, 7) // 把 quad3 和 quad5 的目标填为 7
 - 7: a = 1
 - 8: goto _ // after-then 跳过 else (占位, 稍后 backpatch 到 end)
 - nextquad = 9
 - 9: a = 2 // else 分支
 - nextquad = 10
- 回填 else (false) 目标及 then 后跳转:
 - backpatch(overall.false, 9) // 把 quad6 的目标设为 9 (else 起始)
 - backpatch({8}, 10) // 把 quad8 的目标设为 10 (then 后跳转到 end)
 - 最后 nextquad = 10 是程序续处 (end label)



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))
    a = 1;
else
    a = 2;
```



- 我们看一个具体的例子！
- 最终填好的三地址码

```
1: if x < y goto 3
2: goto 5
3: if y < z goto 7
4: goto 5
5: if x == z goto 7
6: goto 9
7: a = 1
8: goto 10
9: a = 2
10: ... // 后续代码
```

说明：
1 的 true 目标是 3（进入 $y < z$ ）；
2/4 都跳到 5（当 $x < y$ 或 $y < z$ 为假时，进入右侧 $x == z$ 的判断）；
3 和 5 都跳到 7（当任一判断为真时进入 then）；
6 跳到 9（若右侧 $x == z$ 为假则进入 else）；
8 跳到 10（then 执行完跳过 else）。



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))
    a = 1;
else
    a = 2;
```



- 我们看一个具体的例子！
- 最终填好的三地址码

```
1: if x < y goto 3
2: goto 5
3: if y < z goto 7
4: goto 5
5: if x == z goto 7
6: goto 9
7: a = 1
8: goto 10
9: a = 2
10: ... // 后续代码
```

说明：
1 的 true 目标是 3（进入 $y < z$ ）；
2/4 都跳到 5（当 $x < y$ 或 $y < z$ 为假时，进入右侧 $x == z$ 的判断）；
3 和 5 都跳到 7（当任一判断为真时进入 then）；
6 跳到 9（若右侧 $x == z$ 为假则进入 else）；
8 跳到 10（then 执行完跳过 else）。

思考：这个是不是最优解？



控制流语句翻译

- 引入：为什么需要“穿越”？
 - 在控制流语句（如 if-else、while、for）中，中间代码往往包含跳转指令
 - 这些跳转的目标位置（label）在生成代码时通常还未知，需要在整个语法分析完成后才能确定
 - 为了处理这种情况，我们引入 穿越（translation schemes with inherited attributes + backpatching），在语义动作中延迟确定目标地址



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))
    a = 1;
else
    a = 2;
```



- 我们看一个具体的例子！
- 最终填好的三地址码

```
1: if x < y goto 3
2: goto 5
3: if y < z goto 7
4: goto 5
5: if x == z goto 7
6: goto 9
7: a = 1
8: goto 10
9: a = 2
10: ... // 后续代码
```

说明：
1 的 true 目标是 3（进入 $y < z$ ）；
2/4 都跳到 5（当 $x < y$ 或 $y < z$ 为假时，进入右侧 $x == z$ 的判断）；
3 和 5 都跳到 7（当任一判断为真时进入 then）；
6 跳到 9（若右侧 $x == z$ 为假则进入 else）；
8 跳到 10（then 执行完跳过 else）。

思考：这个是不是最优解？



控制流语句翻译

- 总结！翻译过程（带穿越）

- 表达式 $(x < y) \&\& (y < z)$
 - 如果 $x < y$ 为假 → 整体为假（短路），跳转到 false
 - 如果 $x < y$ 为真 → fall-through，进入 $y < z$ 判断
- 与 $(x == z)$ 的 ||
 - 如果 $(x < y \&\& y < z)$ 为真 → 整体为真（短路），跳转到 true
 - 否则继续判断 $(x == z)$

(x < y)
true → fall-through 到 if y < z
false → 跳到 (x == z)

(y < z)
true → 直接跳到 then 部分
false → fall-through 到 (x == z)

(x == z)
true → 直接跳到 then 部分
false → 跳到 else 部分



控制流语句翻译

- “穿越”的基本思想
 - 穿越（traversal/translation scheme）：在语法制导翻译（SDT）中，利用继承属性，将未确定的跳转目标传递下去，直到生成目标点时再进行“回填”
 - 配合 回填（backpatching）：将之前生成的跳转指令列表与目标语句地址绑定



控制流语句翻译

- “穿越”的基本思想
 - 穿越 (traversal/translation scheme)：在语法制导翻译 (SDT) 中，利用继承属性，将未确定的跳转目标传递下去，直到生成目标点时再进行“回填”
 - 配合 回填 (backpatching)：将之前生成的跳转指令列表与目标语句地址绑定

换句话说：穿越 = 在语法树遍历过程中传递未完成的跳转信息



控制流语句翻译

- $B \rightarrow B_1 \&\& B_2$ 带“穿越”的语义规则：


```

B1.false = if (B.false = fall) newlabel() else B.false
B1.true = fall
B2.true = B.true
B2.false = B.false
B.code = if (B.false = fall) then B1.code || B2.code || label(B1.false) else
B1.code || B2.code
      
```

$B \rightarrow B_1 \&\& B_2$	<pre> B₁.true = newlabel() B₁.false = B.false B₂.true = B.true B₂.false = B.false B.code = B₁.code label(B₁.true) B₂.code </pre>
------------------------------	---



控制流语句翻译

- 布尔表达式有如下两个功能：
 - 改变控制流，跳转
 - 刚刚前面重点讨论，用非终结符号 **B** 表示此种功能的布尔表达式以示区别
 - 求值
 - 如 $x = \text{true}$, $x = a < b$
- 统一处理


```

S → id = E ; | if (E) S | while (E) S | S S
E → E || E | E && E | E rel E | E + E | (E) | id | true | false
      
```

 - 使用不同的代码生成函数处理表达式的两种角色。E.n 对应于抽象语法树上的表达式节点。两个函数：
 - jump, 对于出现在 $S \rightarrow \text{while}(E) S$ 中的 E, 调用 E.n.jump(t.f)
 - rvalue, 对于出现在 $S \rightarrow \text{id} = E$ 中的 E, 在节点 E.n 上调用 rvalue。如果 E 是算术表达式，按照算术表达式的翻译生成代码。如果 E 是布尔表达式，如 $E_1 \&\& E_2$ ，首先为 E 生成跳转代码，然后在跳转代码的真假出口分别将 true 或 false 赋给一个新的临时变量 t。



控制流语句翻译

- 利用“穿越”修改布尔表达式的语义规则：

```

test = E1.addr rel.op E2.addr
s = if B.true ≠ fall and B.false ≠ fall then
    gen('if' test 'goto' B.true) || gen('goto' B.false)
    else if B.true ≠ fall then gen('if' test 'goto' B.true)
    else if B.false ≠ fall then gen('ifFalse' test 'goto' B.false)
    else ''
B.code = E1.code || E2.code || s
      
```

图 6-39 $B \rightarrow E_1 \text{ rel } E_2$ 的语义规则

```

B1.true = if B.true ≠ fall then B.true else newlabel()
B1.false = fall
B2.true = B.true
B2.false = B.false
B.code = if B.true ≠ fall then B1.code || B2.code
    else B1.code || B2.code || label(B1.true)
      
```

图 6-40 $B \rightarrow B_1 || B_2$ 的语义规则

- 注意 B.true=fall 时，还得为 B₁.true new 一个 label

```

B1.true = B.true
B1.false = newlabel()
B2.true = B.true
B2.false = B.false
B.code = B1.code || label(B1.false) || B2.code
      
```



控制流语句翻译

- 使用标号 fall 的控制流语句翻译示例：

- if ($x < 100 || x > 200 \&\& x != y$) $x = 0$;

```

B1.true = if B.true ≠ fall then B.true else newlabel()
B1.false = fall
B2.true = B.true
B2.false = B.false
B.code = if B.true ≠ fall then B1.code || B2.code
    else B1.code || B2.code || label(B1.true)
      
```

$B \rightarrow B_1 || B_2$ 的语义规则

```

test = E1.addr rel.op E2.addr
s = if B.true ≠ fall and B.false ≠ fall then
    gen('if' test 'goto' B.true) || gen('goto' B.false)
    else if B.true ≠ fall then gen('if' test 'goto' B.true)
    else if B.false ≠ fall then gen('ifFalse' test 'goto' B.false)
    else ''
B.code = E1.code || E2.code || s
      
```

$B \rightarrow E_1 \text{ rel } E_2$ 的语义规则

```

B.true = fall
B.false = S1.next = S.next
S.code = B.code || S1.code
      
```

```

S.next = newlabel()
P.code = S.code || label(S.next)
      
```

```

if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2: x = 0
L1:
      
```



控制流语句翻译

- 示例
 - 赋值语句 $x = a < b \&\& c < d$

```

ifFalse a < b goto L1
ifFalse c < d goto L1
t = true
goto L2
L1: t = false
L2: x = t
      
```

图 6-42 通过计算一个临时变量的值来翻译一个布尔类型的赋值语句



控制流语句翻译

■ 布尔表达式的回填翻译

1) $B \rightarrow B_1 \parallel M B_2$	{ $backpatch(B_1.falselist, M.instr);$ $B.truelist = merge(B_1.truelist, B_2.truelist);$ $B.falselist = B_2.falselist; \}$
2) $B \rightarrow B_1 \&\& M B_2$	{ $backpatch(B_1.truelist, M.instr);$ $B.truelist = B_2.truelist;$ $B.falselist = merge(B_1.falselist, B_2.falselist); \}$
3) $B \rightarrow ! B_1$	{ $B.truelist = B_1.falselist;$ $B.falselist = B_1.truelist; \}$
4) $B \rightarrow (B_1)$	{ $B.truelist = B_1.truelist;$ $B.falselist = B_1.falselist; \}$
5) $B \rightarrow E_1 \text{ rel } E_2$	{ $B.truelist = makelist(nextinstr);$ $B.falselist = makelist(nextinstr + 1);$ $gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto -');$ $gen('goto -');$ }
6) $B \rightarrow \text{true}$	{ $B.truelist = makelist(nextinstr);$ $gen('goto -');$ }
7) $B \rightarrow \text{false}$	{ $B.falselist = makelist(nextinstr);$ $gen('goto -');$ }
8) $M \rightarrow \epsilon$	{ $M.instr = nextinstr; \}$



控制流语句翻译

■ 布尔表达式的回填翻译

$$B \rightarrow B_1 \parallel B_2 \quad \left\{ \begin{array}{l} B_1.true = B.true \\ B_1.false = newlabel() \\ B_2.true = B.true \\ B_2.false = B.false \\ B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code \end{array} \right.$$

$$B \rightarrow B_1 \parallel M B_2 \quad \left\{ \begin{array}{l} backpatch(B_1.falselist, M.instr); \\ B.truelist = merge(B_1.truelist, B_2.truelist); \\ B.falselist = B_2.falselist; \end{array} \right.$$



控制流语句翻译

■ 布尔表达式的回填翻译

$$B \rightarrow ! B_1 \quad \left\{ \begin{array}{l} B_1.true = B.false \\ B_1.false = B.true \\ B.code = B_1.code \end{array} \right.$$

$$B \rightarrow ! B_1 \quad \left\{ \begin{array}{l} B.truelist = B_1.falselist; \\ B.falselist = B_1.truelist; \end{array} \right.$$

$$B \rightarrow (B_1) \quad \left\{ \begin{array}{l} B.truelist = B_1.truelist; \\ B.falselist = B_1.falselist; \end{array} \right.$$



控制流语句翻译

■ 布尔表达式的回填翻译

```
backpatch(list p, label L) {
  for each position i in p {
    code[i] = "goto " + L; // 回填
  }
}
```

1) $B \rightarrow B_1 \parallel M B_2$	{ $backpatch(B_1.falselist, M.instr);$ $B.truelist = merge(B_1.truelist, B_2.truelist);$ $B.falselist = B_2.falselist; \}$
2) $B \rightarrow B_1 \&\& M B_2$	{ $backpatch(B_1.truelist, M.instr);$ $B.truelist = B_2.truelist;$ $B.falselist = merge(B_1.falselist, B_2.falselist); \}$
3) $B \rightarrow ! B_1$	{ $B.truelist = B_1.falselist;$ $B.falselist = B_1.truelist; \}$
4) $B \rightarrow (B_1)$	{ $B.truelist = B_1.truelist;$ $B.falselist = B_1.falselist; \}$
5) $B \rightarrow E_1 \text{ rel } E_2$	{ $B.truelist = makelist(nextinstr);$ $B.falselist = makelist(nextinstr + 1);$ $gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto -');$ $gen('goto -');$ }
6) $B \rightarrow \text{true}$	{ $B.truelist = makelist(nextinstr);$ $gen('goto -');$ }
7) $B \rightarrow \text{false}$	{ $B.falselist = makelist(nextinstr);$ $gen('goto -');$ }
8) $M \rightarrow \epsilon$	{ $M.instr = nextinstr; \}$



控制流语句翻译

■ 布尔表达式的回填翻译

$$B \rightarrow B_1 \&\& B_2 \quad \left\{ \begin{array}{l} B_1.true = newlabel() \\ B_1.false = B.false \\ B_2.true = B.true \\ B_2.false = B.false \\ B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code \end{array} \right.$$

$$B \rightarrow B_1 \&\& M B_2 \quad \left\{ \begin{array}{l} backpatch(B_1.truelist, M.instr); \\ B.truelist = B_2.truelist; \\ B.falselist = merge(B_1.falselist, B_2.falselist); \end{array} \right.$$



控制流语句翻译

■ 布尔表达式的回填翻译

$$B \rightarrow E_1 \text{ rel } E_2 \quad \left\{ \begin{array}{l} B.code = E_1.code \parallel E_2.code \\ \parallel gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto' B.true) \\ \parallel gen('goto' B.false) \end{array} \right.$$

$$B \rightarrow E_1 \text{ rel } E_2 \quad \left\{ \begin{array}{l} B.truelist = makelist(nextinstr); \\ B.falselist = makelist(nextinstr + 1); \\ emit('if' E_1.addr \text{ rel.op } E_2.addr 'goto -'); \\ emit('goto -'); \end{array} \right.$$



控制流语句翻译

■ 布尔表达式的回填翻译

$$\begin{array}{l|l} B \rightarrow \text{true} & B.\text{code} = \text{gen}('goto' B.\text{true}) \\ B \rightarrow \text{false} & B.\text{code} = \text{gen}('goto' B.\text{false}) \end{array}$$

$$B \rightarrow \text{true} \quad \{ B.\text{truelist} = \text{makelist}(\text{nextinstr}); \text{emit}('goto _'); \}$$

$$B \rightarrow \text{false} \quad \{ B.\text{falselist} = \text{makelist}(\text{nextinstr}); \text{emit}('goto _'); \}$$

$$M \rightarrow \epsilon \quad \{ M.\text{instr} = \text{nextinstr}; \}$$



控制流语句翻译

■ 回填和非回填方法的比较

$$B \rightarrow E_1 \text{ rel } E_2 \quad \begin{array}{l} B.\text{code} = E_1.\text{code} \parallel E_2.\text{code} \\ \parallel \text{gen}('if' E_1.\text{addr} \text{ rel.op } E_2.\text{addr} 'goto' B.\text{true}) \\ \parallel \text{gen}('goto' B.\text{false}) \end{array}$$

$$5) \quad B \rightarrow E_1 \text{ rel } E_2 \quad \{ B.\text{truelist} = \text{makelist}(\text{nextinstr}); B.\text{falselist} = \text{makelist}(\text{nextinstr} + 1); \text{gen}('if' E_1.\text{addr} \text{ rel.op } E_2.\text{addr} 'goto _'); \text{gen}('goto _'); \}$$

- 回填时生成指令坯，然后加入相应的list
- 原来跳转到B.true的指令，现在被加入到B.truelist中



控制流语句翻译

■ 控制转移语句的回填

$$\begin{array}{l} \circ S \rightarrow \text{if} (B) S \quad | \\ \quad \text{if} (B) S \text{ else } S \quad | \\ \quad \text{while} (B) S \quad | \\ \quad \{ L \} \quad | A \\ L \rightarrow L S \quad | S \end{array}$$

○ 语句的综合属性: nextlist

- nextlist中的跳转指令的目标应该是S执行完毕之后紧接着执行的下一条指令的位置
- 考虑S是while语句、if语句的子语句时，分别应该跳转到哪里



控制流语句翻译

■ 回填和非回填方法的比较

$$B \rightarrow B_1 \parallel B_2 \quad \begin{array}{l} B_1.\text{true} = B.\text{true} \\ B_1.\text{false} = \text{newlabel}() \\ B_2.\text{true} = B.\text{true} \\ B_2.\text{false} = B.\text{false} \\ B.\text{code} = B_1.\text{code} \parallel \text{label}(B_1.\text{false}) \parallel B_2.\text{code} \end{array}$$

$$1) \quad B \rightarrow B_1 \parallel M B_2 \quad \{ \text{backpatch}(B_1.\text{falselist}, M.\text{instr}); B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist}); B.\text{falselist} = B_2.\text{falselist}; \}$$

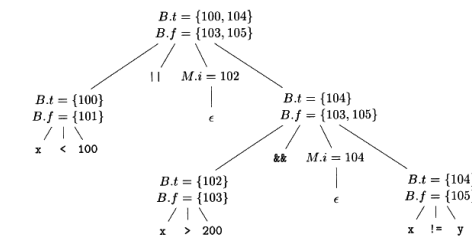
- ... 应为相应的list的赋值或者merge
- 生成label的地方，在回填方案中使用M来记录相应的代码位置，M.instr需对应label的标号
- 原方案生成的指令goto B1.false，现在生成了goto M.instr



控制流语句翻译

■ 布尔表达式的回填例子

- $x < 100 \parallel x > 200 \&\& x \neq y$



```

100: if x < 100 goto -
101: goto -
102: if x > 200 goto 104
103: goto -
104: if x != y goto -
105: goto -

```

a) 将 104 回填到指令 102 中之后

```

100: if x < 100 goto -
101: goto 102
102: if x > 200 goto 104
103: goto -
104: if x != y goto -
105: goto -

```

b) 将 102 回填到指令 101 中之后

```

if x < 100 goto L2
goto L3
L3: if x > 200 goto L4
goto L1
L4: if x != y goto L2
goto L1
L2: x = 0
L1:

```



控制流语句翻译

■ 控制转移语句的回填

$$1) \quad S \rightarrow \text{if} (B) M S_1 \quad \{ \text{backpatch}(B.\text{truelist}, M.\text{instr}); S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist}); \}$$

$$2) \quad S \rightarrow \text{if} (B) M_1 S_1 \text{ else } M_2 S_2 \quad \{ \text{backpatch}(B.\text{truelist}, M_1.\text{instr}); \text{backpatch}(B.\text{falselist}, M_2.\text{instr}); \text{temp} = \text{merge}(S_1.\text{nextlist}, S_2.\text{nextlist}); S.\text{nextlist} = \text{merge}(\text{temp}, S.\text{nextlist}); \}$$

$$6) \quad M \rightarrow \epsilon \quad \{ M.\text{instr} = \text{nextinstr}; \}$$

$$7) \quad N \rightarrow \epsilon \quad \{ N.\text{nextlist} = \text{makelist}(\text{nextinstr}); \text{gen}('goto _'); \}$$

- M的作用就是用M.instr记录下下一个指令的位置

- 规则1中记录了then分支的代码起始位置;
- 规则2中，分别记录了then分支和else分支的起始位置;

- N的作用是生成goto指令坯，N.nextlist只包含这个指令的位置



控制流语句翻译



■ 控制转移语句的回填

3) $S \rightarrow \text{while } M_1 (B) M_2 S_1$

```
{ backpatch( $S_1.nextlist$ ,  $M_1.instr$ );
  backpatch( $B.truelist$ ,  $M_2.instr$ );
   $S.nextlist = B.falselist$ ;
  gen('goto'  $M_1.instr$ ); }
```

4) $S \rightarrow \{ L \}$ $\{ S.nextlist = L.nextlist; \}$

5) $S \rightarrow A ;$ $\{ S.nextlist = \text{null}; \}$

8) $L \rightarrow L_1 M S$ $\{ \text{backpatch}(\mathcal{L}_1.nextlist, M.instr);$
 $L.nextlist = S.nextlist; \}$

9) $L \rightarrow S$ $\{ L.nextlist = S.nextlist; \}$



控制流语句翻译



■ 控制转移语句的回填

$S \rightarrow \text{while } (B) S_1$

```
begin = newlabel()
B.true = newlabel()
B.false = S.next
S1.next = begin
S.code = label(begin) || B.code
         || label(B.true) || S1.code
         || gen('goto' begin)
```

$S \rightarrow \text{while } M_1 (B) M_2 S_1$

```
{ backpatch( $S_1.nextlist$ ,  $M_1.instr$ );
  backpatch( $B.truelist$ ,  $M_2.instr$ );
   $S.nextlist = B.falselist$ ;
  emit('goto'  $M_1.instr$ ); }
```



控制流语句翻译



■ 看一个do while回填的例子

- 翻译步骤（带 nextquad）



控制流语句翻译



■ 控制转移语句的回填

$S \rightarrow \text{if } (B) S_1$

```
B.true = newlabel()
B.false = S1.next = S.next
S.code = B.code || label(B.true) || S1.code
```

$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$

```
B.true = newlabel()
B.false = newlabel()
S1.next = S2.next = S.next
S.code = B.code
         || label(B.true) || S1.code
         || gen('goto' S.next)
         || label(B.false) || S2.code
```

$S \rightarrow \text{if } (B) M S_1 \{ \text{backpatch}(\mathcal{B}.truelist, M.instr);$
 $S.nextlist = \text{merge}(\mathcal{B}.falselist, S_1.nextlist); \}$

$S \rightarrow \text{if } (B) M_1 S_1 N \text{ else } M_2 S_2$

```
{ backpatch( $B.truelist$ ,  $M_1.instr$ );
  backpatch( $B.falselist$ ,  $M_2.instr$ );
  temp = merge( $S_1.nextlist$ ,  $N.nextlist$ );
  S.nextlist = merge(temp,  $S_2.nextlist$ ); }
```



控制流语句翻译 $P \rightarrow S$



■ 控制转移语句的回填

$S \rightarrow \text{assign}$

```
S.next = newlabel()
P.code = S.code || label(S.next)
S.code = assign.code
```

$S \rightarrow S_1 S_2$

```
S1.next = newlabel()
S2.next = S.next
S.code = S1.code || label(S1.next) || S2.code
```

$S \rightarrow \{ L \}$ $\{ S.nextlist = L.nextlist; \}$

$S \rightarrow A ;$ $\{ S.nextlist = \text{null}; \}$

$M \rightarrow \epsilon$ $\{ M.instr = \text{nextinstr}; \}$

$N \rightarrow \epsilon$ $\{ N.nextlist = \text{makelist}(\text{nextinstr});$
 $\text{emit}(\text{'goto' .}); \}$

$L \rightarrow L_1 M S$ $\{ \text{backpatch}(\mathcal{L}_1.nextlist, M.instr);$
 $L.nextlist = S.nextlist; \}$

$L \rightarrow S$ $\{ L.nextlist = S.nextlist; \}$



控制流语句翻译



■ 看一个do while回填的例子

```
do {
  a = a + 1;
} while (a < 10 && b > 0);
```

- 条件 $a < 10 \ \&\& \ b > 0$ 需要 短路求值
- 先判断 $a < 10$ ，若为假，直接退出循环；若为真，再判断 $b > 0$

$S \rightarrow \text{while } M_1 (B) M_2 S_1$

```
{ backpatch( $S_1.nextlist$ ,  $M_1.instr$ );
  backpatch( $B.truelist$ ,  $M_2.instr$ );
   $S.nextlist = B.falselist$ ;
  emit('goto'  $M_1.instr$ ); }
```



控制流语句翻译

■ 看一个do while回填的例子

```
do {
  a = a + 1;
} while (a < 10 && b > 0);
```

- 条件 $a < 10 \ \&\& \ b > 0$ 需要 短路求值
- 先判断 $a < 10$, 若为假, 直接退出循环; 若为真, 再判断 $b > 0$
- 翻译步骤 (带 nextquad)

$S \rightarrow \text{while } M_1 (B) M_2 S_1$

```
{ backpatch( $S_1.nextlist$ ,  $M_1.instr$ );
  backpatch( $B.truelist$ ,  $M_2.instr$ );
   $S.nextlist = B.falselist$ ;
  emit('goto'  $M_1.instr$ ); }
```



控制流语句翻译

```
do {
  a = a + 1;
} while (a < 10 && b > 0);
```

■ 看一个do while回填的例子

- 翻译步骤 (带 nextquad)
- (3) 生成布尔条件 $a < 10 \ \&\& \ b > 0$ ➤ (4) 回填
 - 先处理 $a < 10$: $\square \text{backpatch}(B1.true, 5)$ (真分支跳到判断 $b > 0$)
 - 3: if $a < 10$ goto 5 // 真的情况继续判断 $b > 0$ $\square \text{backpatch}(B2.true, M=1)$ ($b > 0$ 成立时, 跳回循环体起点)
 - 4: goto _ // 假的情况退出
 - 此时: $\square B.false = \{4, 6\} \rightarrow$ 填为循环出口。
 - $B1.true = \{3\}$
 - $B1.false = \{4\}$
 - 再处理 $b > 0$ (从 nextquad=5 开始):
 - 5: if $b > 0$ goto _
 - 6: goto _



Break、Continue的处理

- 虽然break、continue在语法上是一个独立的句子, 但是它的代码和外围语句相关
- 方法: (break语句)
 - 跟踪外围语句S,
 - 生成一个跳转指令坏
 - 将这个指令坏的位置加入到S的nextlist中
- 跟踪的方法
 - 在符号表中设置break条目, 令其指向外围语句
 - 在符号表中设置指向S的nextlist的指针, 然后把这个指令坏的位置直接加入到nextlist中



控制流语句翻译

```
do {
  a = a + 1;
} while (a < 10 && b > 0);
```

■ 看一个do while回填的例子

- 翻译步骤 (带 nextquad)
- 假设 nextquad = 1 开始
- (1) 标记循环体起点
 - $M = \text{nextquad} = 1$
- (2) 生成循环体
 - 1: $t1 = a + 1$
 - 2: $a = t1$
 - nextquad = 3
- (3) 生成布尔条件 $a < 10 \ \&\& \ b > 0$

$S \rightarrow \text{while } M_1 (B) M_2 S_1$

```
{ backpatch( $S_1.nextlist$ ,  $M_1.instr$ );
  backpatch( $B.truelist$ ,  $M_2.instr$ );
   $S.nextlist = B.falselist$ ;
  emit('goto'  $M_1.instr$ ); }
```



控制流语句翻译

```
do {
  a = a + 1;
} while (a < 10 && b > 0);
```

■ 看一个do while回填的例子

- 翻译步骤 (带 nextquad)
 - (3) 生成布尔条件 $a < 10 \ \&\& \ b > 0$ ➤ (4) 回填
 - 先处理 $a < 10$: $\square \text{backpatch}(B1.true, 5)$ (真分支跳到判断 $b > 0$)
 - 3: if $a < 10$ goto 5 // 真的情况继续判断 $b > 0$ $\square \text{backpatch}(B2.true, M=1)$ ($b > 0$ 成立时, 跳回循环体起点)
 - 4: goto _ // 假的情况退出
 - 此时: $\square B.false = \{4, 6\} \rightarrow$ 填为循环出口。
 - $B1.true = \{3\}$
 - $B1.false = \{4\}$
 - 再处理 $b > 0$ (从 nextquad=5 开始):
 - 5: if $b > 0$ goto _
 - 6: goto _
- ```
1: t1 = a + 1
2: a = t1
3: if a < 10 goto 5
4: goto 7 // false branch of a<10 → exit
5: if b > 0 goto 1
6: goto 7 // false branch of b>0 → exit
7: ... // 后续语句
```



## 中间代码表示

### 五、简答题 (共 1 题, 共 10 分)

1. (5 分) 在图 1 中间代码翻译方案中添加处理如下控制流构造

造的规则 (注意布尔表达式之前的否运算):

$S \rightarrow \text{for } (S_1; !B; S_2) S_3$

2. (5 分) 结合图 2 布尔表达式的翻译方案, 翻译

如下代码 (第 5 到第 7 行):

要求: 画出语法分析树, 并在每个节点上标注相

应的属性, 给出最终的中间代码。

```
(1) int i;
(2) int n = 100;
(3) int sum = 0;
(4)
(5) for (i = 0; i < n && sum < 1000; i = i + 1) {
(6) sum = sum + i;
(7) }
```

| 得分  |  |
|-----|--|
| 评分人 |  |

| 产生式                                                  | 语义规则                                                                                                                                                                                                                              |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P \rightarrow S$                                    | $S.next = newlabel()$<br>$P.code = S.code \ \&\& \ label(S.next)$                                                                                                                                                                 |
| $S \rightarrow \text{assign}$                        | $S.code = \text{assign}.code$                                                                                                                                                                                                     |
| $S \rightarrow \text{if } (B) S_1$                   | $B.true = newlabel()$<br>$B.false = S_1.next = S.next$<br>$S.code = B.code \ \&\& \ label(B.true) \ \&\& \ S_1.code$                                                                                                              |
| $S \rightarrow \text{if } (B) S_1 \text{ else } S_2$ | $B.true = newlabel()$<br>$B.false = newlabel()$<br>$S_1.next = S_2.next = S.next$<br>$S.code = B.code$<br>$\ \&\& \ label(B.true) \ \&\& \ S_1.code$<br>$\ \&\& \ label(B.false) \ \&\& \ S_2.code$                               |
| $S \rightarrow \text{while } (B) S_1$                | $begin = newlabel()$<br>$B.true = newlabel()$<br>$B.false = S.next$<br>$S_1.next = begin$<br>$S.code = label(begin) \ \&\& \ B.code$<br>$\ \&\& \ label(B.true) \ \&\& \ S_1.code$<br>$\ \&\& \ label(B.false) \ \&\& \ S_2.code$ |
| $S \rightarrow S_1 S_2$                              | $S_1.next = newlabel()$<br>$S_2.next = S.next$<br>$S.code = S_1.code \ \&\& \ label(S_2.next) \ \&\& \ S_2.code$                                                                                                                  |



# 中间代码表示

## 五、简答题（共 1 题，共 10 分）

1.（5 分）在图 1 中间代码翻译方案中添加处理如下控制流构造的规则（注意布尔表达式之前的否运算）；  
 $S \rightarrow \text{for}(S_1; !B; S_2) S_3$

2.（5 分）结合图 2 布尔表达式的翻译方案，翻译如下代码（第 5 到第 7 行）：

要求：画出语法分析树，并在每个节点上标注相应的属性，给出最终的中间代码。

```
11) int i;
12) int n = 100;
13) int sum = 0;
14)
15) for (i = 0; i < n && sum < 1000; i = i + 1) {
16) sum = sum + i;
17) }
```

| P-图                                                |  |
|----------------------------------------------------|--|
| $P \rightarrow S$                                  |  |
| $S \rightarrow \text{assign}$                      |  |
| $S \rightarrow \text{if}(B) S_1$                   |  |
| $S \rightarrow \text{if}(B) S_1 \text{ else } S_2$ |  |
| $S \rightarrow \text{while}(B) S_1$                |  |
| $S \rightarrow S_1 S_2$                            |  |

## 参考答案:

1.

begin = new\_label()

B.false = new\_label()

B.true = S.next()

S1.next = begin

S2.next = begin

S3.next = newlabel()

S.code = S1.code || label(begin) || B.code  
|| label(B.false) || S3.code ||  
label(S3.next)  
|| S2.code || gen(‘goto’ begin)



# 中间代码表示

## 五、简答题（共 1 题，共 10 分）

1.（5 分）在图 1 中间代码翻译方案中添加处理如下控制流构造的规则（注意布尔表达式之前的否运算）；  
 $S \rightarrow \text{for}(S_1; !B; S_2) S_3$

2.（5 分）结合图 2 布尔表达式的翻译方案，翻译如下代码（第 5 到第 7 行）：

要求：画出语法分析树，并在每个节点上标注相应的属性，给出最终的中间代码。

```
11) int i;
12) int n = 100;
13) int sum = 0;
14)
15) for (i = 0; i < n && sum < 1000; i = i + 1) {
16) sum = sum + i;
17) }
```

| P-图                                                |  |
|----------------------------------------------------|--|
| $P \rightarrow S$                                  |  |
| $S \rightarrow \text{assign}$                      |  |
| $S \rightarrow \text{if}(B) S_1$                   |  |
| $S \rightarrow \text{if}(B) S_1 \text{ else } S_2$ |  |
| $S \rightarrow \text{while}(B) S_1$                |  |
| $S \rightarrow S_1 S_2$                            |  |

2.

i = 0

L1: if i < 0 goto L2

goto L5

L2: if sum < 1000 goto L3

goto L5

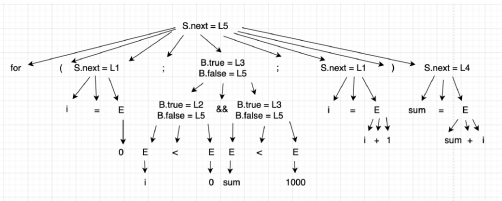
L3: sum = sum + i

goto L4

L4: i = i + 1

goto L1

L5:



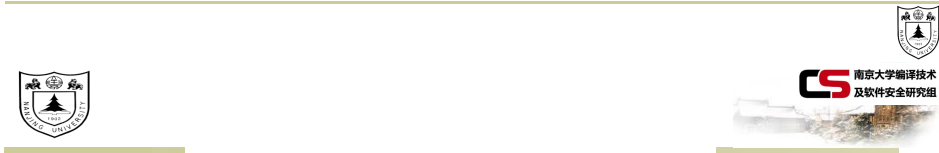


# 复习课（五）

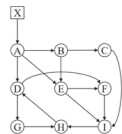
## （运行时环境）

冯洋

南京大学计算机学院  
fengyang@nju.edu.cn



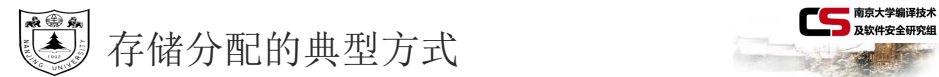
七、简答题（共1题，共10分）  
现有如下图所示的对象网络：



1. 从X指向A的指针被删除时，对象的引用计数会发生哪些变化，画出对应的指针被删除后的对象网络（保留下来的对象请在字母外画上实线圈，被删除对象不作处理，保留的指针画上箭头，被删除的指针不作处理）。



|     |  |
|-----|--|
| 得分  |  |
| 评分人 |  |

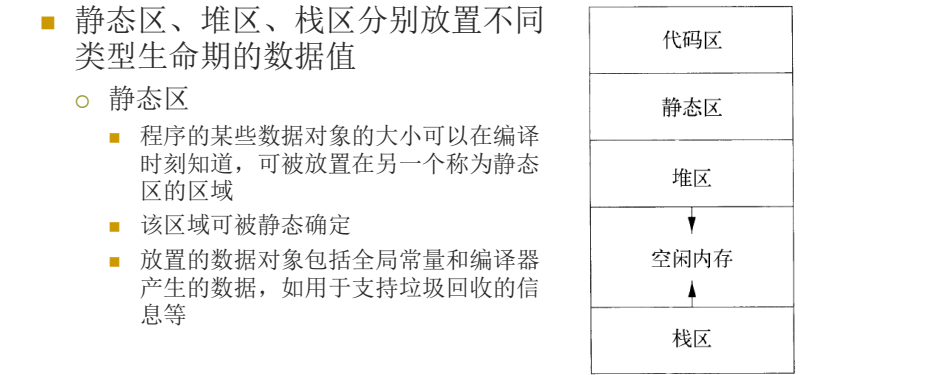
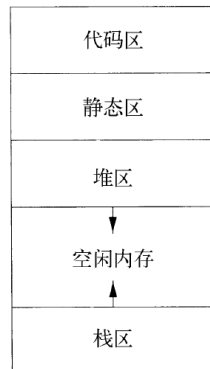


## 存储分配的典型方式

- 静态区、堆区、栈区分别放置不同类型生命期的数据值

### 静态区

- 程序的某些数据对象的大小可以在编译时刻知道，可被放置在另一个称为静态区的区域
- 该区域可被静态确定
- 放置的数据对象包括全局常量和编译器产生的数据，如用于支持垃圾回收的信息等



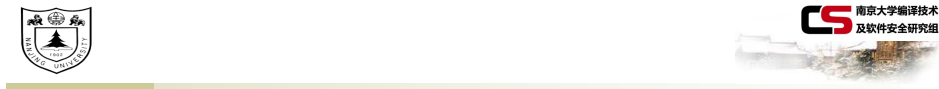
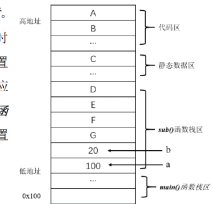
- 请考虑以下C语言代码。我们假设BP是指向栈底的指针，SP是指向栈顶的指针，C语言栈帧的结构如右图所示。本题中，栈帧布局地址增长遵循课本规定，栈是由低地址向高地址增长，其他部分则是由高地址向低地址增长。

- (1) A: \_\_\_\_\_
- (2) B: \_\_\_\_\_
- (3) C: \_\_\_\_\_
- (4) D: \_\_\_\_\_
- (5) E: \_\_\_\_\_
- (6) F: \_\_\_\_\_
- (7) G: \_\_\_\_\_
- (8) SP: \_\_\_\_\_
- (9) BP: \_\_\_\_\_

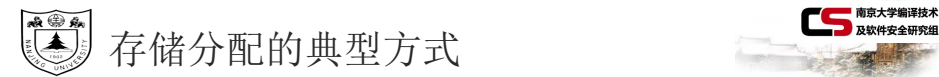
```
(1) int sub(int a, int b);
(2) int m = 0;
(3) void main(){
(4) int i = 100;
(5) int j = 20;
(6) m = sub(i, j);
(7) printf("m=%d", m);
(8) }
(9)
(10) int sub(int a, int b){
(11) int res = 0;
(12) res = a - b;
(13) return res;
(14) }
```

| 局部变量 | 高地址 |
|------|-----|
| BP的值 |     |
| 返回地址 |     |
| 返回值  |     |
| 实参   | 低地址 |

假设程序运行完第12行，即res计算完毕时。请在如下栈帧结构图中，分别写出此时A.B.C.D.E.F.G 对应位置的内容，若对应位置可以推断具体值（值可以为null），请写明对应变量名及其值，并在图中通过箭头表明sub()函数的SP和BP的位置（假设SP和BP的位置在res被定义时已经确定）。



- 2. 从A指向B的指针被删除时，引用计数会发生哪些变化，画出对应的指针被删除后的对象网络。

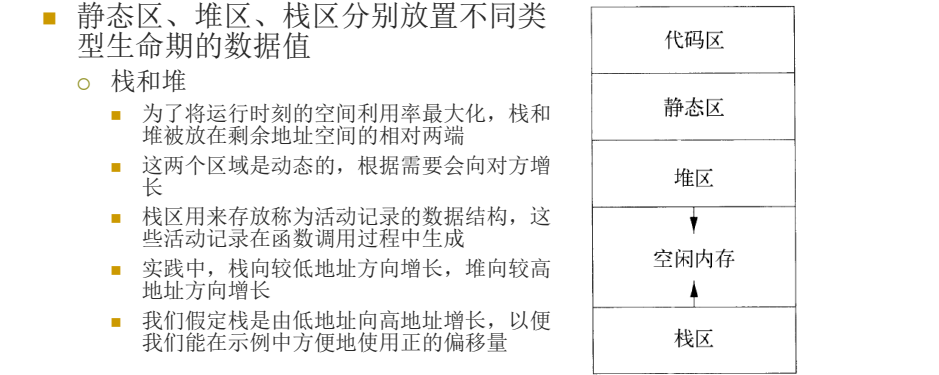
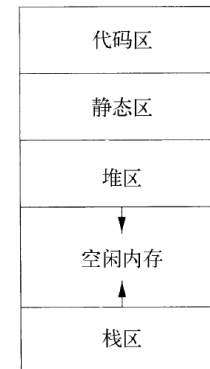


## 存储分配的典型方式

- 静态区、堆区、栈区分别放置不同类型生命期的数据值

### 栈和堆

- 为了将运行时刻的空间利用率最大化，栈和堆被放在剩余地址空间的相对两端
- 这两个区域是动态的，根据需要会向对方增长
- 栈区用来存放称为活动记录的数据结构，这些活动记录在函数调用过程中生成
- 实践中，栈向较低地址方向增长，堆向较高地址方向增长
- 我们假定栈是由低地址向高地址增长，以便我们能在示例中方便地使用正的偏移量



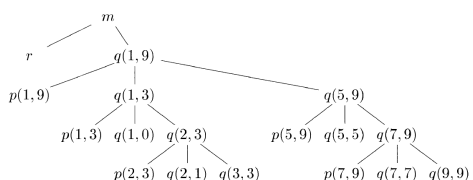


## 活动树的例子（1）

### ■ 程序

#### ○ 活动树与程序行为对应关系

- 过程调用的序列和活动树的前序遍历相对应。
- 过程返回的序列和活动树的后序遍历相对应。



```

enter main()
 enter readArray()
 leave readArray()
 enter quicksort(1,9)
 enter partition(1,9)
 leave partition(1,9)
 enter quicksort(1,3)
 ...
 leave quicksort(1,3)
 enter quicksort(5,9)
 ...
 leave quicksort(5,9)
 leave quicksort(1,9)
leave main()

```



## 调用/返回代码序列的要求

### ■ 数据方面

- 能够把参数正确地传递给被调用者
- 能够把返回值传递给调用者

### ■ 控制方面

- 能够正确转到被调用过程的代码开始位置
  - 能够正确转回调用者的调用位置（的下一条指令）
- 调用代码序列和活动记录的布局相关



## 活动记录

### ■ 过程调用和返回由控制栈进行管理

- 每个活跃的活动对应于栈中的一个活动记录
- 活动记录按照活动的开始时间，从栈底到栈顶排列
  - 活动树的根位于栈底
  - 栈中全部活动记录的序列对应于在活动树中到达当前控制所在的活动结点的路径

|         |
|---------|
| 实在参数    |
| 返回值     |
| 控制链     |
| 访问链     |
| 保存的机器状态 |
| 局部数据    |
| 临时变量    |



## 一个简单的例子

### ■ 两个函数

### ■ 一个全局变量m

### ■ 若干局部变量i, j, c, a...

### ■ 一次函数调用

```

int fun(int a, int b);
int m=10;
int main()
{
 int i = 4;
 int j = 5;
 m = fun(i,j);
 return 0;
}

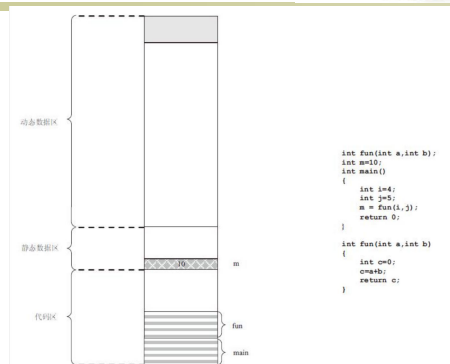
int fun(int a, int b)
{
 int c = 0;
 c = a + b;
 return c;
}

```



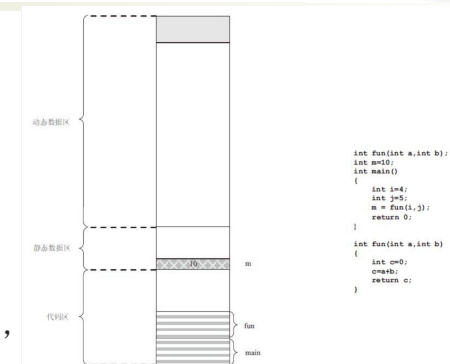
## 一个简单的例子

- 整个程序放到了内存中的样子如右图。共有三个区域：代码区，静态数据区，动态数据区



## 一个简单的例子

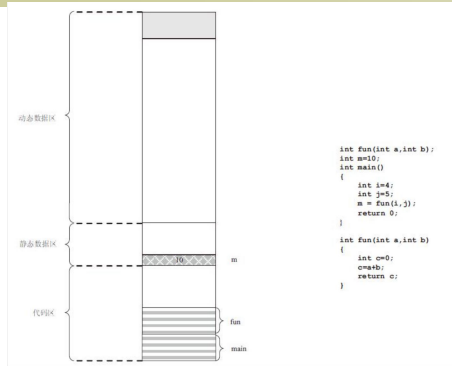
- 静态数据区：可以在编译时刻知道大小的数据对象，包括全局常量和编译器产生的数据（比如支持垃圾回收的信息等）
- 动态数据区：无法在编译时刻知道大小的数据对象
- 代码区：因为目标代码在编译时刻已经知道具体的大小，代码区也是一种静态区





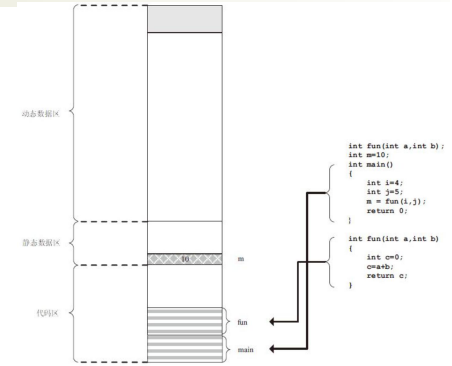
## 一个简单的例子

- 静态数据区：可以在编译时刻知道大小的数据对象，包括全局常量和编译器产生的数据（比如支持垃圾回收的信息等）
- 动态数据区：无法在编译时刻知道大小的数据对象
- 代码区：因为目标代码在编译时刻已经知道具体的大小，代码区也是一种静态区



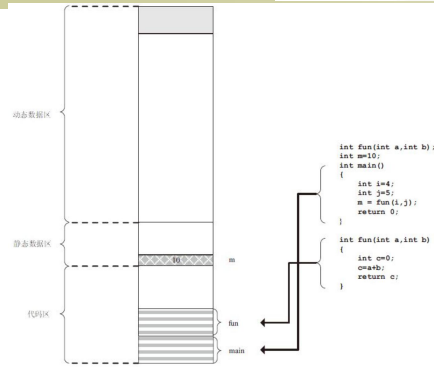
## 一个简单的例子

- 全局变量m的数值装载在静态数据区中
- 函数fun和main的代码装载在代码区中
- 程序开始执行前，动态数据区中没有数据



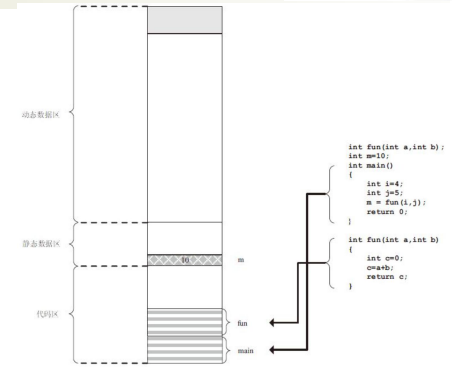
## 一个简单的例子

- 全局变量m的数值装载在静态数据区中
- 函数fun和main的代码装载在代码区中
- 程序开始执行前，动态数据区中没有数据；只有在程序开始执行后，在指令的驱动下，这一区域才会产生数据，压栈和清栈的工作就是在这一区域完成的



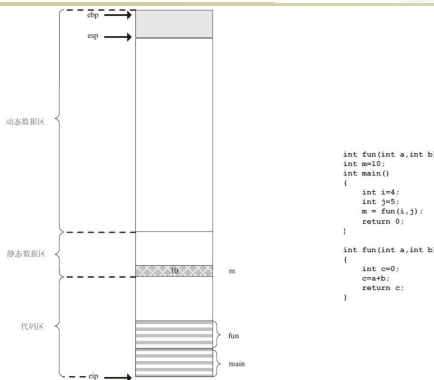
## 一个简单的例子

- 程序执行的本质，就是代码区的指令不断执行
  - 驱使动态数据区和静态数据区产生数据变化



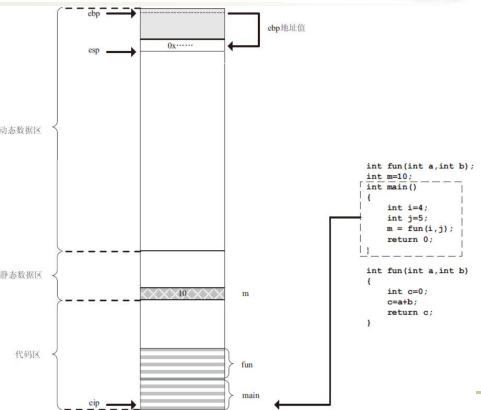
## 一个简单的例子

- 我们假定CPU中有三个寄存器，分别是eip、ebp、esp。其中eip永远指向代码区要执行的下一条指令
  - ebp与esp的位置在程序加载时由内核设置
  - eip的位置一开始指向main函数
- ebp和esp用来管理栈空间
  - ebp指向栈底，esp指向栈顶，代码区中的程序执行，函数调用、返回，均存在不断的压栈与清栈
  - 栈底画在栈顶的上方



## 一个简单的例子

- 程序开始执行main函数第一条指令，eip自动指向下一条指令。第一条指令的执行，则使得ebp的地址值被保存到栈里面
- 保存的目的是本程序执行完毕后，ebp还能返回到现在的位置，复原现在的栈。ebp保存之后，esp则自动向栈顶移动，它永远指向栈顶

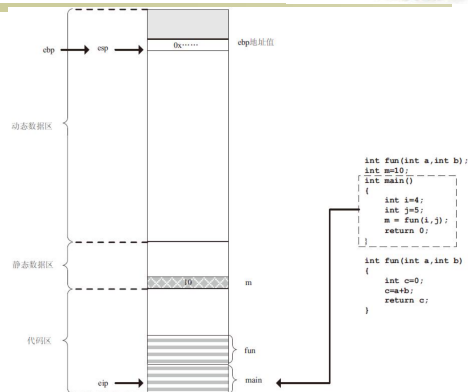






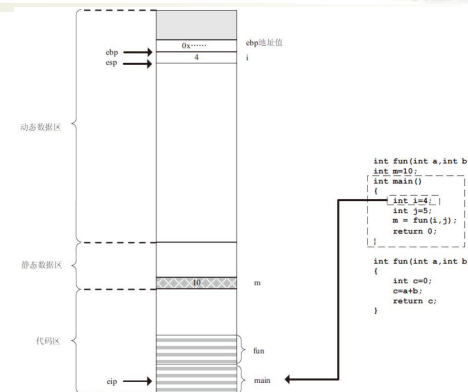
## 一个简单的例子

- 在对返回地址保存之后，开始为main构建栈，main的栈构建情况如右图所示
- 注意，此事ebp原来指向的值已经被存起来了，所以esp就自动向栈顶移动，同时，ebp需要用来指向main的栈底
- 此时由于没有数据在栈中，ebp和esp的位置其实是重叠的



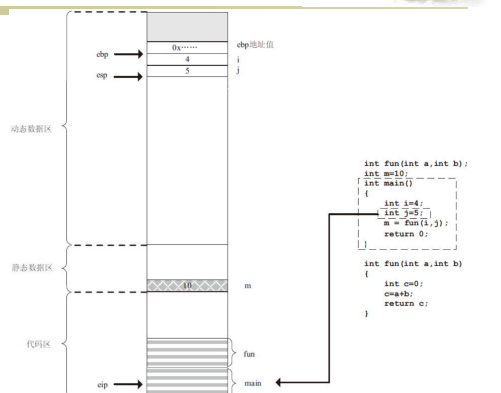
## 一个简单的例子

- 程序继续执行，第一个语句，int i = 4 被执行
- 注意到，对main来说，i 是一个局部变量，那么，其初始值4，被压入栈中
- 特别注意栈顶指针esp的变化



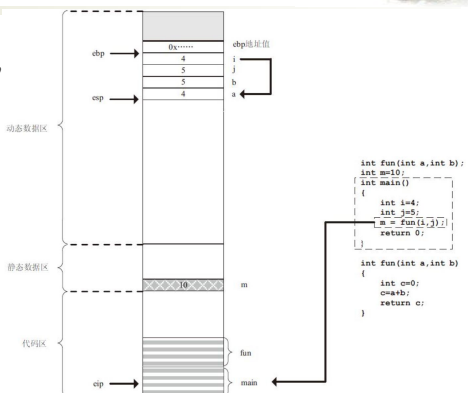
## 一个简单的例子

- 同样的道理，int j = 5被执行，5被压入到栈



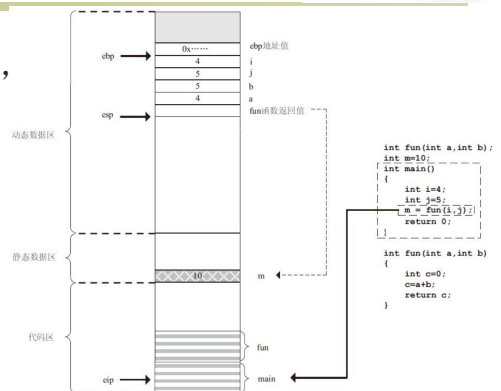
## 一个简单的例子

- 下一步，程序执行到调用函数fun的指令。特别注意，此处，我们对fun的调用，首先需要完成的，是传参指令
- 特别注意，在C语言中，参数入栈顺序与传参书写顺序正好相反。参数b先入栈，b的数值是局部变量j的数值5。因此，情况如右所示



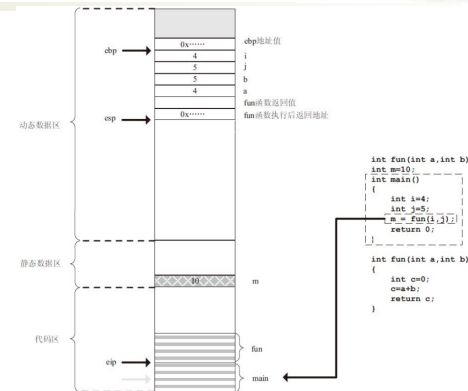
## 一个简单的例子

- 程序继续执行，那么应该压入的是fun函数的返回值，后面返回值应该是传递给m



## 一个简单的例子

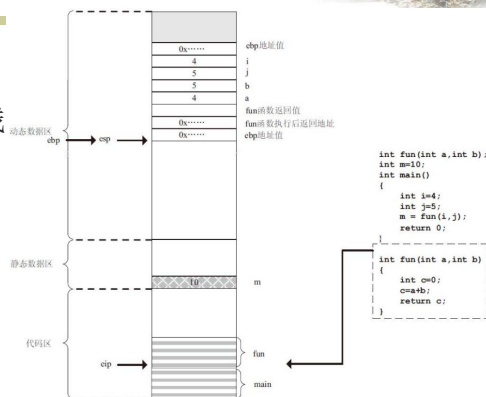
- 在保存好fun函数执行完毕的返回地址后，eip跳转到fun的代码区进行执行，如右图所示
- 后续则需要构建fun的栈





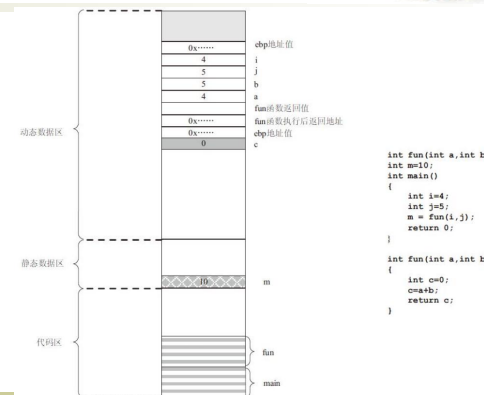
## 一个简单的例子

- 与前面我们调用main函数相似，ebp与esp再次指向同一个位置，此时fun的栈是空的



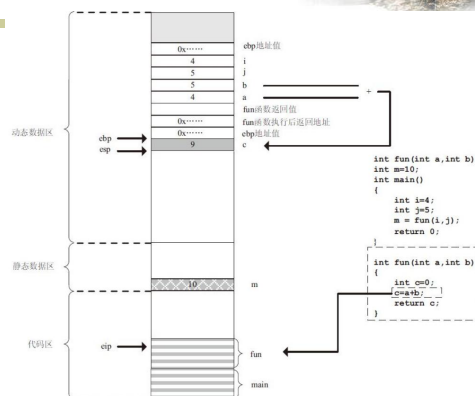
## 一个简单的例子

- 与前面的情况类似，c会被压入fun的栈中
- 后续是若干运算指令



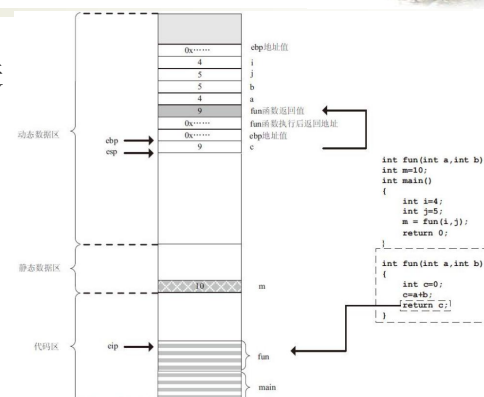
## 一个简单的例子

- 在执行a+b的过程中，我们可以看到，在运算指令的执行过程中，可以通过一系列的偏移计算，得到运算指令需要的数值
- 运算指令执行完毕后，赋值给局部变量c



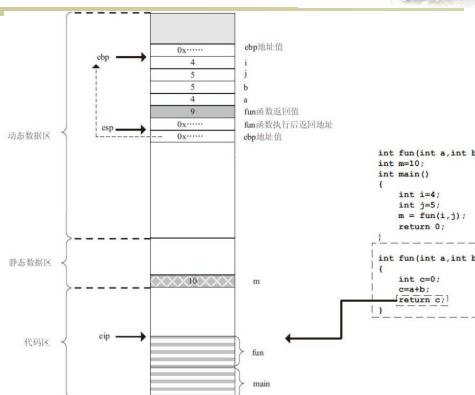
## 一个简单的例子

- 局部变量c的值作为返回值，对fun函数的返回值进行赋值



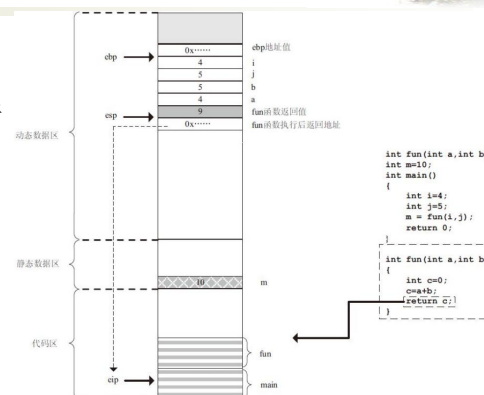
## 一个简单的例子

- fun函数执行完毕，要恢复main函数调用fun函数的现场
- 这一现场包括两个部分
  - 一部分是main函数的栈要恢复，包括栈顶和栈底
  - 另一部分是要找到fun函数执行后的返回地址，然后再跳转到那里去继续执行
  - ebp的恢复相对简单，之前在此处有存储ebp的地址值，在消栈操作中，只需要取出该值赋值给ebp指针即可



## 一个简单的例子

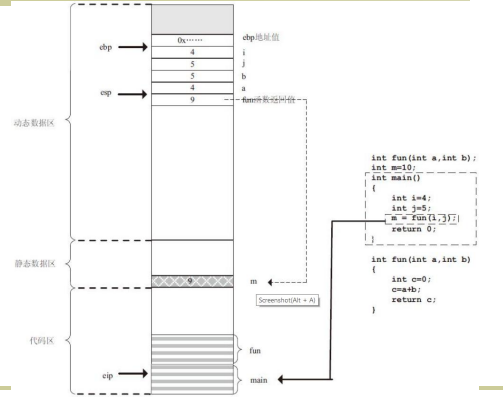
- 在ebp恢复后，esp自动退栈。
- 退栈后指向函数fun执行后的返回地址，然后执行ret指令
  - 该指令将esp此时的值反馈给eip，eip则可以找到main中的下一条指令，继续执行





## 一个简单的例子

- eip找到正确的后续执行指令之后，即将返回值赋值给正确的局部变量，即m。如右图所示



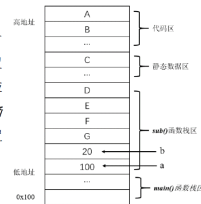
- 请考虑以下C语言代码。我们假设BP是指向栈底的指针，SP是指向栈顶的指针，C语言栈帧的结构如右图所示。本题中，栈帧布局地址增长遵循课本规定，栈是由低地址向高地址增长，其他部分则是由高地址向低地址增长。

- (1) A: \_\_\_\_\_
- (2) B: \_\_\_\_\_
- (3) C: \_\_\_\_\_
- (4) D: \_\_\_\_\_
- (5) E: \_\_\_\_\_
- (6) F: \_\_\_\_\_
- (7) G: \_\_\_\_\_
- (8) SP: \_\_\_\_\_
- (9) BP: \_\_\_\_\_

```
(1) int sub (int a, int b);
(2) int m = 0;
(3) void main(){
(4) int i = 100;
(5) int j = 20;
(6) m = sub(i, j);
(7) printf("m=%d", m)
(8) }
(9)
(10) int sub (int a, int b){
(11) int res = 0;
(12) res = a - b;
(13) return res;
(14) }
```

假设程序运行完第 12 行，即 *res* 计算完毕时，请在如下的栈帧结构图中，分别写出此时 A,B,C,D,E,F,G 对应位置的内容，若对应位置可以推断具体值（值可以为 null），请写明对应变量名及其值，并在图中通过箭头表明 *sub()* 函数的 SP 和 BP 的位置（假设 SP 和 BP 的位置在 *res* 被定义时已经确定）。

| 局部变量 | 高地址 |
|------|-----|
| BP的值 |     |
| 返回地址 |     |
| 返回值  |     |
| 实参   | 低地址 |



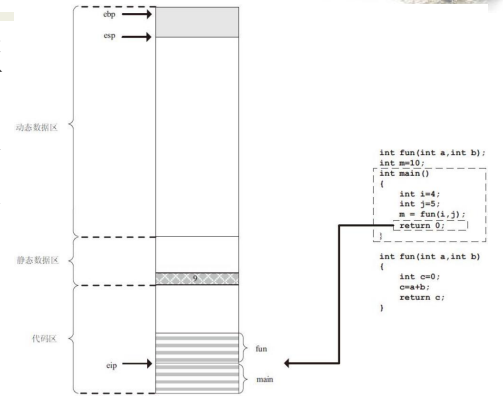
## 堆管理

- 堆空间
  - 用于存放生命周期不确定、或生存到被明确删除为止的数据对象
  - 例如：new生成的对象可以生存到被delete为止
  - malloc申请的空间生存到被free为止
- 存储管理器
  - 分配/回收堆区空间的子系统
    - 分配：它尽可能使用堆中的空闲空间来满足分配请求。如没有请求大小的空间块可供分配，试图从操作系统中获得连续的虚拟内存来增加堆区的存储空间。如无可利用空间，通知应用
    - 回收：通常不会将内存返回给操作系统，而是用于后续数据对象分配
  - 根据语言而定
    - C、C++需要手动回收空间
    - Java可以自动回收空间（垃圾收集）



## 一个简单的例子

- 同理，在执行完main相关的ret指令之后，也需要对main函数的栈进行清除，清楚后的结果如右图所示
- 至此，整个程序执行完毕



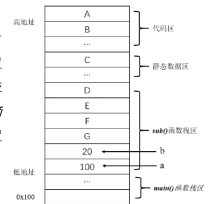
- 请考虑以下C语言代码。我们假设BP是指向栈底的指针，SP是指向栈顶的指针，C语言栈帧的结构如右图所示。本题中，栈帧布局地址增长遵循课本规定，栈是由低地址向高地址增长，其他部分则是由高地址向低地址增长。

- (1) A: 代码 int i = 100
- (2) B: 代码 int j = 20
- (3) C: m = 0
- (4) D: 局部变量 res = 80
- (5) E: BP
- (6) F: 返回地址
- (7) G: 返回值 = null
- (8) SP: D 的上方
- (9) BP: a 下方

```
(1) int sub (int a, int b);
(2) int m = 0;
(3) void main(){
(4) int i = 100;
(5) int j = 20;
(6) m = sub(i, j);
(7) printf("m=%d", m)
(8) }
(9)
(10) int sub (int a, int b){
(11) int res = 0;
(12) res = a - b;
(13) return res;
(14) }
```

假设程序运行完第 12 行，即 *res* 计算完毕时，请在如下的栈帧结构图中，分别写出此时 A,B,C,D,E,F,G 对应位置的内容，若对应位置可以推断具体值（值可以为 null），请写明对应变量名及其值，并在图中通过箭头表明 *sub()* 函数的 SP 和 BP 的位置（假设 SP 和 BP 的位置在 *res* 被定义时已经确定）。

| 局部变量 | 高地址 |
|------|-----|
| BP的值 |     |
| 返回地址 |     |
| 返回值  |     |
| 实参   | 低地址 |



## 存储管理器

- 基本功能
  - 分配：为每个内存请求分配一段连续的、适当大小的堆空间
    - 从空闲的堆空间分配，如果失败，则从操作系统中获取内存、增加堆空间
  - 回收：把被回收的空间返回空闲空间缓冲池，以满足其他内存需求
- 评价存储管理器的特性
  - 空间效率：使程序需要的堆空间最小，即减小碎片
    - 通过使存储碎片达到最少而得到
  - 程序效率：充分运用内存系统的层次，提高效率
    - 关注对象在存储中的放置方法，更好地利用空间，并且有希望使程序运行得更快
  - 低开销：使分配/收回内存的操作尽可能高效



## 存储管理器

- 处理器通常具有少量寄存器，寄存器中的内容由软件控制
  - 有一层或多层高速缓存，通常用静态RAM制造，主存通常用动态RAM实现
  - 在一次内存访问中，首先在最近的存储中寻找数据，如找不到，到下一层寻找
- 数据以连续存储块的方式进行传输
  - 在主存和高速缓存之间的数据是按照被称为高速缓存线的块进行传输的
    - 高速缓存线的长度通常在32-256字节之间
    - 在虚拟内存和主内存之间的数据以“页”的内存块进行传输的大小通常在4-64KB之间

| 典型的大小     |          | 典型的访问时间    |
|-----------|----------|------------|
| > 2GB     | 虚拟内存（磁盘） | 3~15 ms    |
| 256MB~2GB | 物理内存     | 100~150 ns |
| 128KB~4MB | 二级高速缓存   | 40~60 ns   |
| 16~64KB   | 一级高速缓存   | 5~10 ns    |
| 32个机器字    | 寄存器（处理器） | 1 ns       |



## 堆空间分配方法

- Best-Fit
  - 总是将请求的内存分配在满足请求的最小的窗口中
  - 好处：可以将大的窗口保留下来，应对更大的请求
- First-Fit
  - 总是将对象放置在第一个能够容纳请求的窗口中
  - 放置对象时花费时间较少，但是总体性能较差
  - 但是first-fit的分配方法通常具有较好的数据局部性
    - 同一时间段内的生成的对象经常被分配在连续的空间内



## 可达性

- 直观地讲，可达性就是指一个存储块可以被程序访问到
- 根集：不需要指针解引用就可以直接访问的数据
  - Java：静态成员、栈中变量
- 可达性
  - 根集的成员都是可达的
  - 对于任意一个对象，如果指向它的一个指针被保存在可达对象的某字段中、或数组元素中，那么这个对象也是可达的
- 性质
  - 一旦一个对象变得不可达，它就不会再变成可达的



## 堆空间的碎片问题

- 随着程序分配/回收内存，堆区逐渐被割裂成为若干空闲存储块（窗口，hole）和已用存储块的交错
- 分配一块内存时，通常是把一个窗口的一部分分配出去，其余部分成为更小的块
- 回收时，被释放的存储块被放回缓冲池。通常要把连续的窗口接合成为更大的窗口



## 垃圾回收

- 垃圾
  - 狭义：不能被引用（不可达）的数据
  - 广义：不需要再被引用的数据
  - 基于类型信息，可知道该对象有多大，及该对象的哪些分量包含指向其他对象的引用
    - 假定对对象的引用总是指向该对象的起始位置
    - 把一个用户程序称为增变者，它会修改堆中的对象集合
    - 增变者从存储管理器处获取空间，创建对象，还可引入和消除对已有对象的引用。当增变者不能“到达”某些对象时，这些对象就变成了“垃圾”
- 垃圾回收：自动回收不可达数据的机制，解除了程序员的负担
- 使用的语言
  - Java、Perl、ML、Modula-3、Prolog、Smalltalk

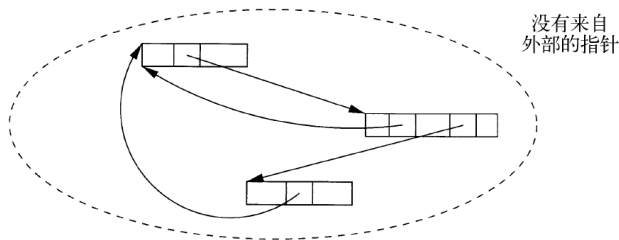
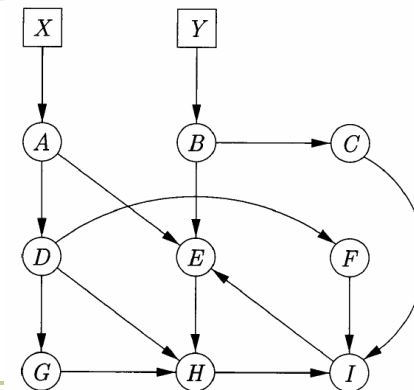


## 垃圾回收方法分类

- 跟踪相关操作，捕获对象变得不可达的时刻，回收对象占用的空间
- 在需要时，标记出所有可达对象、回收其它对象

- 每个对象有一个用于存放引用计数的字段，并按照如下方式维护
  - 对象分配：引用计数设为1
  - 参数传递：引用计数加1
  - 引用赋值： $u=v$ ：u指向的对象引用减1、v指向的对象引用加1
  - 过程返回：局部变量指向对象的引用计数减1
- 如果一个对象的引用计数为0，在删除对象之前，此对象中各个指针所指对象的引用计数减1
- 回收器有缺陷，可能引起内存泄漏
  - 不能回收不可达的循环数据结构
- 开销较大、但是不会引起停顿

- 考虑如下操作:
  - $y=x$
  - $y$ 是当前函数 $f$ 的局部变量,且 $f$ 返回
- 修改计数后总是先考虑是否释放
- 释放一个对象之前总是先处理对象内部的指针



- 功能:
  - 不在垃圾产生时就回收，而是周期性运行
  - 寻找不可达对象并收回它们的空间
  - 通常是空闲空间被耗尽或空闲空间数量低于某个阈值时启动
- 标记-清扫式垃圾回收
- 标记-清扫式垃圾回收的优化
- 标记并压缩垃圾回收

- 一种直接的、全面停顿的算法
- 分成两个阶段
  - 标记：从根集开始，跟踪并标记出所有可达对象
  - 清扫：遍历整个堆区，释放不可达对象
    - 找出所有不可达的对象，将它们放入空闲空间列表
- 如果我们把数据对象看作顶点，引用看作有向边，那么标记的过程实际上是从根集开始的图遍历的过程

- 该算法主要包括两步
  - mark, 从 root 开始进行树遍历, 每个访问的对象标注为「使用中」
  - sweep, 扫描整个内存区域, 对于标注为「使用中」的对象去掉该标志, 对于没有该标注的对象直接回收掉





## 标记-清扫式垃圾回收

- 输入：一个由对象组成的根集，一个堆和一个被称为Free的包含了堆中所有未分配存储块的空闲空间列表。所有空间块都用边界标记进行标识，指明它们的空闲 / 已用状态和大小。
- 输出：在删除了所有垃圾之后的经过修改的Free列表



## 标记-清扫式垃圾回收

- 该算法的缺点有
  - 在进行 GC 期间，整个系统会被挂起（暂停，Stop-the-world），所以在一些实现中，会采用各种措施来减少这个暂停时间
  - heap 容易出现碎片。实现中一般会进行 move 或 compact。（需要说明一点，所有 heap 回收机制都会这个问题）
  - 在 GC 工作一段时间后，heap 中连续地址上存在 age 不同的对象，这非常不利于引用的本地化（locality of reference）
  - 回收时间与 heap 大小成正比

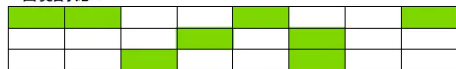


## 标记-清扫垃圾回收算法

回收前状态：



回收后状态：



存活对象    可回收    未使用



## 标记-清扫垃圾回收算法

```

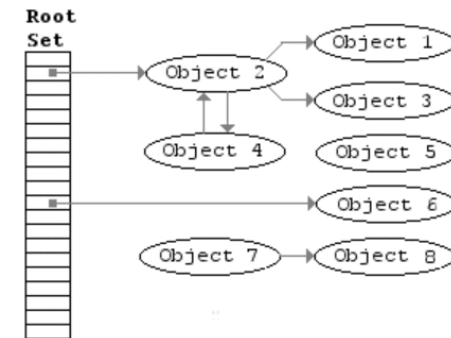
/* 标记阶段 */
1) 把被根集引用的每个对象的 reached 位设置为 1，并把它加入到 Unscanned 列表中；
2) while (Unscanned ≠ ∅) {
3) 从 Unscanned 列表中删除某个对象 o；
4) for (在 o 中引用的每个对象 o') {
5) if (o' 尚未被访问到，即它的 reached 位为 0) {
6) 将 o' 的 reached 位设置为 1；
7) 将 o' 放到 Unscanned 中；
8) }
9) }
10) }
/* 清扫阶段 */
8) Free = ∅；
9) for (堆区中的每个内存块 o) {
10) if (o 未被访问到，即它的 reached 位为 0) 将 o 加入到 Free 中；
11) else 将 o 的 reached 位设置为 0；
12) }

```

因为语言是强类型的，所以垃圾回收机制可以知道每个数据对象的类型，以及这个对象有哪些字段是指针

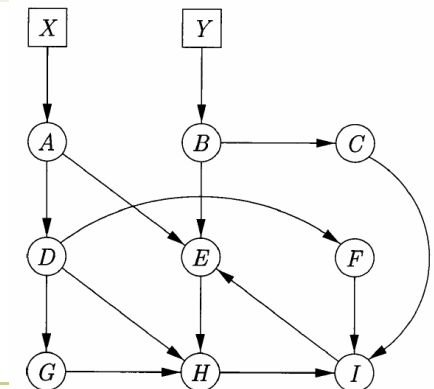


## 标记-清扫垃圾回收算法



## 例子

- 假设 x 是全局变量，y 是当前函数活动的局部变量
- 当前活动返回之后，进行标记清扫
  - A, D, E, F, I, G, H
  - B, C 不可达

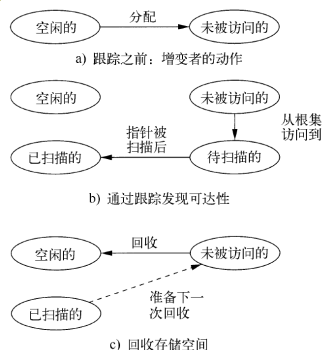






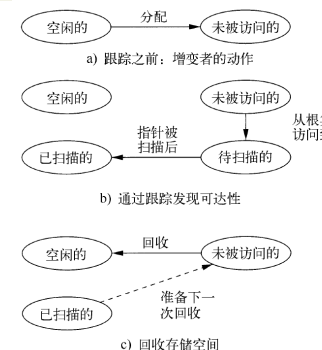
## 基本抽象分类

- 所有基于跟踪的算法都计算可达对象集合，然后取这个集合的补集
- 使用方式如下：
  - 程序（增变者）运行并发出分配请求
  - 垃圾回收器通过跟踪揭示可达性
  - 垃圾回收器收回不可达对象的存储空间



## 基本抽象分类

- 每个存储块处于四种状态之一
  - 空闲：可被分配，不存放可达对象
  - 未被访问：默认不可达，未确定可达性前，都标记为未被访问
  - 待扫描：存储块可达，但块中指针还没被扫描，则处于待扫描状态
  - 已扫描：只包含指向其他已扫描或待扫描对象的引用
- 对存储块的操作会改变存储块的状态
  - 应用程序分配
  - 垃圾回收器访问、扫描
  - 收回



## 标记-清扫垃圾回收算法的优化

- 基本算法需要扫描整个堆，代价很大
- 优化
  - 用一个列表记录所有已经分配的对象
  - 不可达对象等于已分配对象减去可达对象
    - 已分配对象和可达对象之间的差集
- 好处
  - 只需要扫描这个列表就可以完成清扫
- 坏处
  - 需要维护这个列表



## 压缩并标记垃圾回收

- 存在多种进行重新定位的回收器，不同之处在于它们是在本地进行重新定位还是在重新定位前预留了空间
  - 本节介绍的压缩并标记垃圾回收器在本地压缩对象，属于本地重新定位
  - 拷贝回收器，把对象从内存的一个区域移到另一个区域
    - 保留额外的空间用于重新定位可使得一发现可达对象就立刻移动它
- 对可达对象进行重定位可以消除存储碎片
  - 把可达对象移动到堆区的一端，另一端则是空闲空间
  - 空闲空间合并成单一块，分配内存时高效率
    - 可达对象占用的空间要大大小于空闲空间，标记出所有“窗口”后可以将所有可达对象重新定位到堆区的一端，使堆区的所有空闲空间成为一个块



## 压缩并标记垃圾回收

- 整个过程分成三个步骤
  - 标记阶段，参考之前的标记过程
  - 计算新位置
    - 算法扫描堆区中的已分配内存段，并为每个可达对象计算新的地址
    - 新地址从堆的最低端开始分配，可达对象之间没有空闲存储窗口
    - 每个对象的新地址记录在一个名为NewLocation的结构中
  - 移动并设置新的引用
    - 算法将对象拷贝到它们的新地址，更新对象中的所有引用
    - 使之指向新地址，新地址可在NewLocation中找到



## 压缩并标记垃圾回收

```

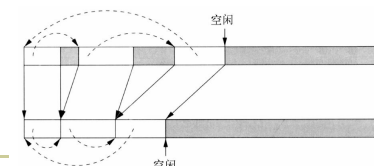
/* 标记 */
1) Unscanned = 根集引用的对象的集合;
2) while (Unscanned ≠ ∅) {
3) 从 Unscanned 中移除对象 o;
4) for (在 o 中引用的每个对象 o') {
5) if (o' 是未访问的) {
6) 将 o' 标记为已访问的;
7) 将 o' 加入到列表 Unscanned 中;
 }
 }
 }
/* 计算新的位置 */
8) free = 堆区的开始位置;
9) for (从低端开始, 遍历堆区中的每个存储块 o) {
10) if (o 是已访问的) {
11) NewLocation(o) = free;
12) free = free + sizeof(o);
 }
 }

```

```

/* 重新设置引用目标并移动已被访问的对象 */
13) for (从低端开始, 堆区中的每个存储块 o) {
14) if (o 是已访问的) {
15) for (o 中的每个引用 o.r) {
16) o.r = NewLocation(o.r);
17) }
18) }
19) r = NewLocation(o);
}

```



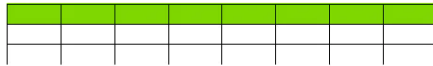


## 压缩并标记垃圾回收

回收前状态：



回收后状态：



存活对象

可回收

未使用



## 拷贝回收器

- 拷贝回收算法虽然能解决内存碎片问题，但是需要多次遍历heap空间，这会导致较大性能损耗
- 拷贝回收算法采用空间换时间的方式来提升性能
  - 输入：一个由对象组成的根集，一个包含了From半空间和To半空间的堆区，其中From半空间包含了已分配对象，To半空间全部是空闲的
  - 输出：最后，To半空间保存已分配的对象。free指针指明了To半空间中剩余空闲空间的开始位置。From半空间此时全部空闲
  - 方法：在From半空间找出可达对象，访问它们时立刻把它们拷贝到To半空间
- 这类GC并不会真正去“回收”不可到达对象，而是会把所有可到达对象移动到一个区域，heap中剩余的空间就是可用的了（因为这里面都是垃圾）



## 拷贝回收器

- 输入：一个由对象组成的根集，一个包含了From半空间和To半空间的堆区，其中From半空间包含了已分配对象，To半空间全部是空闲的
- 输出：最后，To半空间保存已分配的对象。free指针指明了To半空间中剩余空闲空间的开始位置。From半空间此时全部空闲
- 方法：
  - 算法在From半空间找出可达对象，访问它们时立刻把它们拷贝到To半空间



## 拷贝回收器

- 预先保留了可将对象移入的空间，解除了跟踪和发现空闲空间间的依赖关系
- 堆空间被分为两个半空间
  - 增变者在某个半空间内分配存储，当充满这个半空间时，开始垃圾回收
  - 回收时，可达对象被拷贝到另一个半空间
  - 回收完成后，两个半空间角色对调
  - 增变者可继续运行，并在半空间B中分配对象
  - 下一轮垃圾回收将把可达对象移动到A



## 拷贝回收器

- 典型的代表半空间拷贝回收器（semispace collector）。其工作过程是这样的
  - heap被分成2份相邻的空间（semispace）：fromspace与tospace
  - 在程序运行时，只有fromspace会被使用（分配新对象）
  - 在fromspace没有足够空间容纳新对象时，程序会被挂起，然后把fromspace的可达对象拷贝到tospace
  - 在拷贝完成时，之前的2个空间交换身份，tospace成了新一轮的fromspace



## 拷贝回收器

```

1) CopyingCollector () {
2) for (From空间中的所有对象o) NewLocation(o) = NULL;
3) unscanned = free = To空间的开始地址;
4) for (根集中的每个引用r)
5) 将r替换为 LookupNewLocations(r);
6) while (unscanned != free) {
7) o = 在unscanned所指位置上的对象;
8) for (o中的每个引用 o.r)
9) o.r = LookupNewLocation(o.r);
10) unscanned = unscanned + sizeof(o);
 }

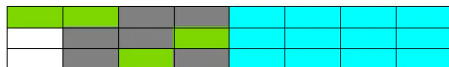
 /* 如果一个对象已经被移动过了，查找这个对象的新位置 */
 /* 否则将对象设置为待扫描状态 */
11) LookupNewLocation(o) {
12) if (NewLocation(o) = NULL) {
13) NewLocation(o) = free;
14) free = free + sizeof(o);
15) 将对象o拷贝到 NewLocation(o);
 }
16) return NewLocation(o);
 }

```

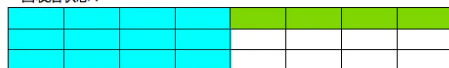


## 拷贝回收器

回收前状态：



回收后状态：



存活对象

可回收

未使用

保留区域



## 分代回收

### ■ 采用该策略的基本原因：

- 因为较年轻的世代往往包含较多的垃圾，也就更频繁地被回收
- 编程实践中，极少出现较老世代对较新世代的引用。因此，我们只需要记录该种情况，并定期采用其他策略进行处理即可



2. 从 A 指向 B 的指针被删除时，引用计数会发生哪些变化，画出对应的指针被删除后的对象网络。

X

A B C

D E F

G H I



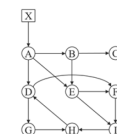
## 分代回收

- 针对很多变量存在时间较短的特点设计
- 基本思想：将堆区进行切割成块并标号，垃圾回收技术每次在较低标号的块上进行
- 具体执行方案：
  - 新对象在编号较低的堆块上创建
  - 每次在0号堆块被填满时刻，就对0号堆块进行垃圾回收；回收完毕后，将i号堆块往后移动到i+1号堆块中
  - 维护一个记忆集，用以存储较高号的堆块对较低号堆块中对象的引用



七、简答题（共1题，共10分）

现有如下图所示的对象网络：



|     |  |
|-----|--|
| 得分  |  |
| 评分人 |  |

1. 从 X 指向 A 的指针被删除时，对象的引用计数会发生哪些变化，画出对应的指针被删除后的对象网络（保留下来的对象请在字母外画上实线圈，被删除对象不作处理，保留的指针画上箭头，被删除的指针不作处理）。

X

A B C

D E F

G H I

X

A B C

D E F

G H I

X

A B C

D E F

G H I



# 复习课（六） （代码生成与寄存器分配）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn



## 代码生成器中的问题



### ■ 指令选择

- 代码生成器需把IR程序映射成为可以在目标机上运行的代码序列。
- 完成映射的复杂性由如下因素决定：
  - 1. IR层次
  - 2. 指令集体系结构本身特性
  - 3. 要达到代码质量
- 如每个形如 $x=y+z$ 的三地址语句，可被翻译为如下
  - LD R0, y
  - ADD R0, R0, z
  - ST x, R0
- 大多数机器上，一个给定的IR程序可用很多种不同的代码序列来实现
- 如目标机有一个“加一”指令，则 $a=a+1$ 可用一个指令INC a实现



## 代码生成器中的问题



### ■ 指令选择

$a = a + 1$

```
LD R0, a // R0 = a
ADD R0, R0, #1 // R0 = R0 + 1
ST a, R0 // a = R0
```



INC a

得考虑  
指令代价



八、简答题（共1题，共10分）

假设有两个可用的寄存器R1和R2，使用简单代码生成算法。

为如下三地址代码生成机器码。

$t = a * b$   
 $u = x + t$   
 $x = u$

- 给出每条三地址代码生成机器码后的寄存器描述符和内存描述符（请在表格左边空白部分写出三地址代码，并在表格中补充写入对应的描述符内容）。
- 结束时将寄存器中的变量写入内存中。

附机器代码格式：

和数指令：LD R, var // 将内存空间中变量var的值存入寄存器R中。

存数指令：ST var, R // 将寄存器R中的值存入变量var的内存空间中。

双目运算指令：OP dst, src1, src2 // 使用OP对寄存器src1和src2中的值进行计算，结果存入寄存器dst中（OP包括ADD, SUB, MUL, DIV）。

初始状态：

|    |    |   |   |   |   |   |
|----|----|---|---|---|---|---|
| R1 | R2 | x | a | b | t | u |
|    |    | x | a | b | t | u |

(1)  $t = a * b$

|    |    |   |   |   |    |   |
|----|----|---|---|---|----|---|
| R1 | R2 | x | a | b | t  | u |
|    |    | x | a | b | R1 | u |

(2)  $u = x + t$

|    |    |   |   |   |    |    |
|----|----|---|---|---|----|----|
| R1 | R2 | x | a | b | t  | u  |
| t  | u  | x | a | b | R1 | R2 |

(3)  $x = u$

|    |    |   |   |   |    |   |
|----|----|---|---|---|----|---|
| R1 | R2 | x | a | b | t  | u |
| t  |    |   | a | b | R1 |   |

(4)

|    |      |       |   |   |       |       |
|----|------|-------|---|---|-------|-------|
| R1 | R2   | x     | a | b | t     | u     |
| t  | u, x | x, R2 | a | b | t, R1 | u, R2 |



## 代码生成器中的问题



### ■ 指令选择

$x = y + z$

```
LD R0, y // R0 = y (load y into register R0)
ADD R0, R0, z // R0 = R0 + z (add z to R0)
ST x, R0 // x = R0 (store R0 into x)
```

$a = b + c$   
 $d = a + e$

```
LD R0, b // R0 = b
ADD R0, R0, c // R0 = R0 + c
ST a, R0 // a = R0
LD R0, a // R0 = a
ADD R0, R0, e // R0 = R0 + e
ST d, R0 // d = R0
```

如果变量a  
不再被使用  
了呢？

冗余的指令

思考，是不是一定冗余？



## 代码生成器中的问题



### ■ 目标语言

#### ■ 三地址机器的模型

- 寄存器的使用常被分解为两个子问题：
  - 1. 寄存器的分配
  - 2. 寄存器的指派
- 我们的目标计算机是一个三地址机器的模型
- 具有加载和保存操作，计算操作，跳转操作，条件跳转
- 这个计算机的内存按字节寻址，具有n个通用寄存器R0, ..., Rn-1
- 一个完成的汇编语言有几十到上百个指令
- 我们使用一个有限的指令集合
- 假设所有运算分量都是整数
- 大部分指令含一个运算符，然后是一个目标地址，最后是一个源运算分量列表
- 指令之前可能有一个标号



- 三地址机器的模型
- 指令集
  - 加载: LD dst, addr
    - 把地址addr中的内容加载到dst所指寄存器
  - 保存: ST x, r
    - 把寄存器r中的内容保存到x中
  - 计算: OP dst, src1, src2
    - 把src1和src2中的值运算后将结果存放到dst中
  - 无条件跳转: BR L
    - 控制流转向标号L的指令
  - 条件跳转: Bcond r, L
    - 对r中的值进行测试, 如果为真则转向L



- 如何将IR中的名字转换为目标代码中的地址
  - 不同的区域中的名字采用不同的寻址方式
- 如何为过程调用和返回生成代码
  - 静态分配
  - 栈式分配



- 划分基本块的算法
  - 输入: 三地址指令序列
  - 输出: 基本块的列表
  - 方法:
    - 确定基本块的首指令:
      - 第一个三地址指令
      - 任意一个条件或无条件转移指令的目标指令
      - 紧跟在一个条件/无条件转移指令之后的指令
    - 确定基本块:
      - 每个首指令对应于一个基本块: 从首指令开始到下一个首指令



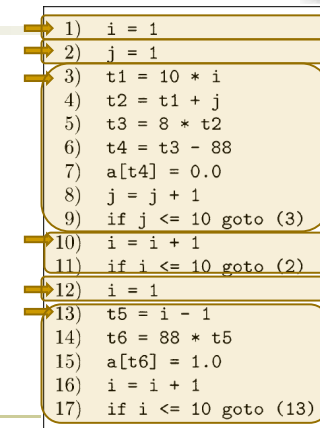
- 程序和指令的代价
  - 不同的目的有不同的度量
    - 最短编译时间、目标程序大小、运行时间、能耗
  - 假设: 每个指令有固定的代价, 设定为1加上运算分量寻址模式的代价
    - LD R0, R1; 代价为1
    - LD R0, M; 代价是2
    - LD R1, \*100(R2); 代价为2
  - 为给定源程序生成最优目标程序
    - 不可判定: 不可判定问题是可计算性理论和计算复杂性理论中定义的一类决定性问题, 此类问题无法总是用单一算法得出正确的是/否的答案。



- 中间代码的流图表示法
  - 中间代码划分成为基本块(basic block)
    - 控制流只能从第一个指令进入
    - 除基本块的最后一个指令外, 控制流不会跳转/停机
  - 结点: 基本块
  - 边: 指明了基本块的执行顺序
- 流图可以作为优化的基础
  - 指出了基本块之间的控制流
  - 可以根据流图了解到一个值是否会被使用等信息



- 基本块划分的例子
  - 第一个指令
    - 1
  - 跳转指令的目标
    - 3、2、13
  - 跳转指令的下一条指令
    - 10、12
  - 基本块:
    - 1-1; 2-2; 3-9; 10-11;
    - 12-12; 13-17







## 基本块和流图

### ■ 练习

- (1)  $b = 1$
- (2)  $n = 10$
- (3)  $d = l + n$
- (4)  $c = 2$
- (5)  $t1 = 4 * a$
- (6)  $t2 = l + n$
- (7)  $c = c + b$
- (8) if  $c < n$  goto (6)
- (9)  $a = a + b$
- (10) if  $a < n$  goto (4)
- (11)  $c = a - b$



## 基本块和流图

### ■ 后续使用信息

- 变量值的使用 (use) 与活跃 (live)
  - 假设三地址语句  $i$  给  $x$  赋了一个值, 如果在后续的另一语句  $j$  的一个运算分量为  $x$ , 且从  $i$  开始有一条没有对  $x$  进行赋值的路径能够到达  $j$ , 那么  $j$  就使用了  $i$  处计算得到的  $x$  的值
  - 我们可以说称  $x$  在语句  $i$  后的程序点上 **活跃**
  - 即在程序执行完语句  $i$  的时刻,  $x$  中存放的值将被后面的语句使用
  - 不活跃是指变量中存放的值不会被使用, 而不是变量不会被使用



## 基本块和流图

### ■ 例子

- 输入: 基本块  $B$ , 开始时  $B$  的所有非临时变量都是活跃的
- 输出: 各语句  $i$  上的变量的活跃性、后续使用信息
  - 在8之前的程序点上,  $i, j, a$  仍然活跃; 且  $j$  在8上被使用
  - 在7之前的程序点上,  $i, j, a, t4$  活跃; 且  $t4$  被7使用
  - 在6之前的程序点上,  $i, j, a, t3$  活跃,  $t4$  不活跃,  $t3$  被6使用
  - 在5之前的程序点上,  $i, j, a, t2$  活跃,  $t3$  不活跃
  - 在4之前的程序点上, ...

- 3)  $t1 = 10 * i$
- 4)  $t2 = t1 + j$
- 5)  $t3 = 8 * t2$
- 6)  $t4 = t3 - 88$
- 7)  $a[t4] = 0.0$
- 8)  $j = j + 1$



## 基本块和流图

### ■ 后续使用信息

- 可用于优化

#### ■ 寄存器指派

如果当前某个变量存放于一个寄存器中, 且之后不会再被使用, 那么这个寄存器可被分派给别的变量

$a = b + c$   
 $d = a + e$

LD R0, b  
ADD R1, R0, c  
ST a, R1  
  
LD R1, a  
ADD R2, R1, e  
ST d, R2



LD R0, b // R0 = b  
ADD R0, R0, c // R0 = R0 + c  
ST a, R0 // a = R0  
  
LD R0, a // R0 = a  
ADD R0, R0, e // R0 = R0 + e  
ST d, R0 // d = R0



## 基本块和流图

### ■ 确定基本块中的活跃性、后续使用

- 输入: 基本块  $B$ , 开始时  $B$  的所有非临时变量都是活跃的
- 输出: 各语句  $i$  上的变量的活跃性、后续使用信息
- 方法:
  - 从  $B$  的最后一个语句开始反向扫描
  - 对于每个语句  $i: x = y + z$ 
    - 从  $B$  的最后一个语句开始反向扫描
    - 对于每个语句  $i: x = y + z$ 
      1. 令语句  $i$  和  $x, y, z$  的当前活跃性信息/使用信息关联
      2. 设置  $x$  为不活跃、无后续使用
      3. 设置  $y$  和  $z$  为活跃, 并指明它们的下一次使用为语句  $i$
  - 注意: 2和3的顺序



## 基本块和流图

### ■ 流图的构造

- 两个顶点  $B$  和  $C$  之间有一条有向边当且仅当基本块  $C$  的第一个指令可能在  $B$  的最后一个指令之后执行
  - 从  $B$  的结尾指令是一条跳转到  $C$  的开头的条件/无条件语句
  - 在原来的序列中,  $C$  紧跟在  $B$  之后, 且  $B$  的结尾不是无条件跳转语句
- 称  $B$  是  $C$  的前驱,  $C$  是  $B$  的后继
- 入口和出口结点
  - 流图中额外添加的边, 不与中间代码 (基本块) 对应
  - 入口到第一条指令有一条边
  - 从任何可能最后执行的基本块到出口有一条边

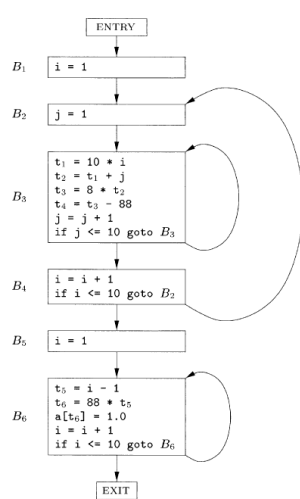




## 基本块和流图

### ■ 流图的构造

- 因跳转而生成的边
  - B3 → B3
  - B4 → B2
  - B6 → B6
- 因为顺序而生成的边
  - 其它



## 基本块和流图

### ■ 循环

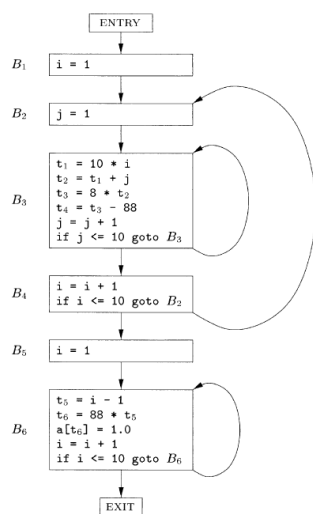
- 程序的大部分运行时间花费在循环上
  - 循环是识别的重点
- 循环的定义
  - 循环L是一个结点集合
  - 存在一个循环入口（loop entry）结点，是唯一的、前驱可以在循环L之外的结点，到达其余结点的路径必然先经过这个入口结点
  - 其余结点都存在到达入口结点的非空路径，且路径都在L中



## 基本块和流图

### ■ 循环的例子

- 循环
  - {B3}
  - {B6}
  - {B2, B3, B4}
- 对于 {B2, B3, B4} 的解释
  - B2为入口结点
  - B1, B5, B6不在循环内的理由
    - 到达B1可不经B2
    - B5, B6没有到达B2的结点



## 基本块的优化

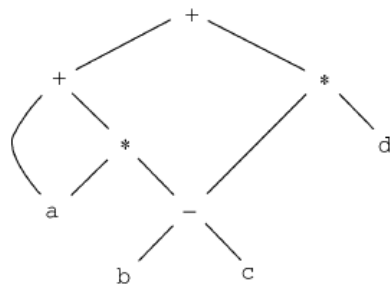
### ■ 针对基本块的局部优化

- 针对基本块的优化可以有很好的效果
- 基本块中各个指令要么都执行，要么都不执行



## 基本块的优化

### ■ 回顾：表达式的DAG



## 基本块的优化

### ■ 基本块可以用DAG表示

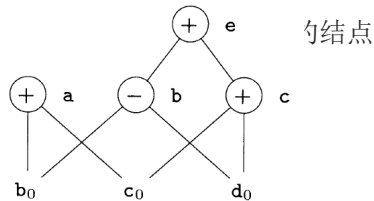
- 每个变量有对应的DAG的结点，代表初始值
- 每个语句s有一个相关的结点N，代表计算得到的值
  - N的子结点对应于（其运算分量当前值的）其它语句
  - 结点N的标号是s的运算符
  - N和一组变量关联，表示s是在此基本块内最晚对它们定值的语句
- 输出结点：结点对应的变量在基本块出口处活跃
- 从DAG，我们可以知道各个变量最后的值和初始值的关系



## 基本块的优化

### ■ 消除死代码

- 在DAG上消除没有附加活跃变量的根结点（没有父结点的结点），即消除死代码
- 如果图中c、e不是活跃变量



死代码=? 不可达代码



## 基本块的优化

### ■ 数组引用的DAG表示

- 从数组取值的运算 $x=a[i]$ 对应于 $=[]$ 的结点
  - $x$ 作为这个结点的标号之一
  - 该节点的左右子节点分别代表数组初始值和下标
- 对数组赋值（例如 $a[j]=y$ ）的运算对应于 $[]=$ 的结点，没有关联的变量、且杀死所有依赖于 $a$ 的变量
  - 三个子节点分别表示 $a_0$ 、 $j$ 和 $y$
- 数组赋值隐含的意思：将数组作为整体考虑，对数组元素赋值即改变整个数组



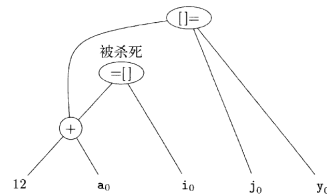
## 基本块的优化

### ■ 数组引用的DAG表示实例

- 设 $a$ 是数组， $b$ 是指针

- $b=12+a$
- $x=b[i]$
- $b[j]=y$

- 注意 $a$ 是被杀死的结点的孙结点
- 一个结点被杀死，意味着它不能被复用
  - 考虑再有指令 $m=b[i]$ ?



## 基本块的优化

### ■ 数组引用实例

- $x=a[i]$
  - $a[j]=y$
  - $z=a[i]$
- $a[i]$ 是公共子表达式吗?**

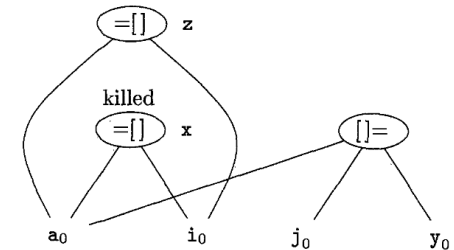
- 注意： $a[j]$ 可能改变 $a[i]$ 的值，因此不能和普通的运算符一样构造相应的结点



## 基本块的优化

### ■ 数组引用的DAG表示实例

- 基本块
  - $x=a[i]$
  - $a[j]=y$
  - $z=a[i]$



## 基本块的优化

- 根据三地址指令序列生成机器指令，假设
  - 每个三地址指令只有一个对应的机器指令
  - 有一组寄存器用于计算基本块内部的值
- 主要目标
  - 尽量减少加载和保存指令，即最大限度利用寄存器
- 重点考虑寄存器的使用方法
  - 执行运算时，运算分量必须放在寄存器中
  - 用于临时变量
  - 存放全局的值
  - 进行运行时管理（比如：栈顶指针）



## 基本块的优化

- DAG 是如何指导指令的选择的？一个例子
- 我们定义一系列的规则，用于支持指令的选择和排列

| Patterns                    | Instructions                                                                                                                                                       |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\odot_x$ a leaf node       | LD R, x // R is a new register                                                                                                                                     |
| $\oplus^x$ an operator node | ADD R <sub>n</sub> , R <sub>o1</sub> , R <sub>o2</sub><br>ST x, R <sub>n</sub><br>// R <sub>n</sub> is a new register;<br>// R <sub>o</sub> are previous registers |



## 基本块的优化

- $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$  是如何指导指令的选择的？一个例子



## 基本块的优化

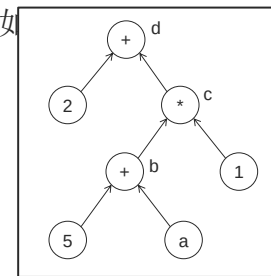
- DAG 是如何指导指令的选择的？一个例子
- 我们定义一系列的规则，用于支持指令的选择和排列

| Patterns                    | Instructions                                                                                                                                                       |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\odot_x$ a leaf node       | LD R, x // R is a new register                                                                                                                                     |
| $\oplus^x$ an operator node | ADD R <sub>n</sub> , R <sub>o1</sub> , R <sub>o2</sub><br>ST x, R <sub>n</sub><br>// R <sub>n</sub> is a new register;<br>// R <sub>o</sub> are previous registers |



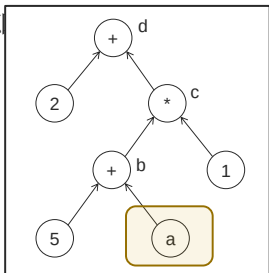
## 基本块的优化

- $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$  是如何指导指令的选择的？一个例子



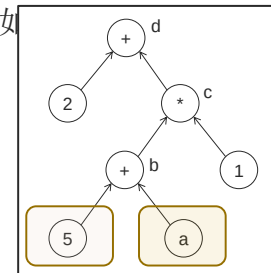
## 基本块的优化

- $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$  是如何指导指令的选择的？一个例子



## 基本块的优化

- $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$  是如何指导指令的选择的？一个例子

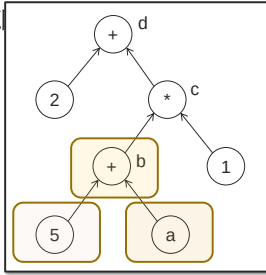


LD R1, 5



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



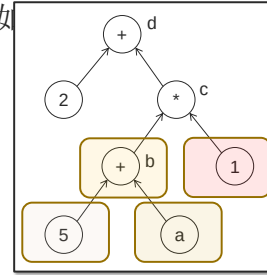
的? 一个例子

|     |            |
|-----|------------|
| LD  | R1, 5      |
| ADD | R2, R0, R1 |
| ST  | b R2       |



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



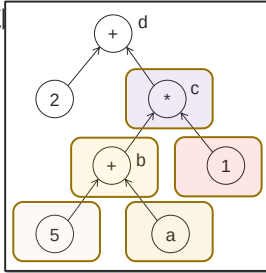
的? 一个例子

|     |            |
|-----|------------|
| LD  | R1, 5      |
| ADD | R2, R0, R1 |
| ST  | b R2       |
| LD  | R3, 1      |



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



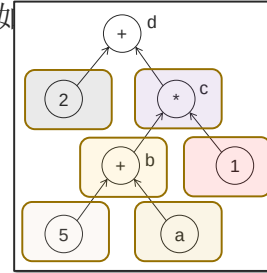
的? 一个例子

|     |            |
|-----|------------|
| LD  | R1, 5      |
| ADD | R2, R0, R1 |
| ST  | b R2       |
| LD  | R3, 1      |
| MUL | R4, R2, R3 |
| ST  | c R4       |



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



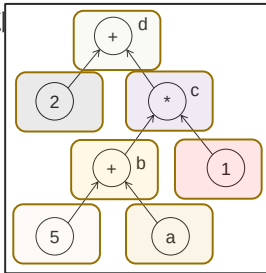
的? 一个例子

|     |            |
|-----|------------|
| LD  | R1, 5      |
| ADD | R2, R0, R1 |
| ST  | b R2       |
| LD  | R3, 1      |
| MUL | R4, R2, R3 |
| ST  | c R4       |
| LD  | R5, 2      |



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



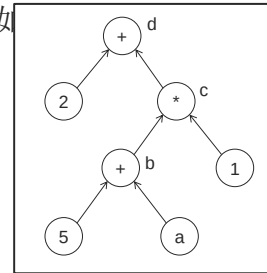
的? 一个例子

|     |            |
|-----|------------|
| LD  | R1, 5      |
| ADD | R2, R0, R1 |
| ST  | b R2       |
| LD  | R3, 1      |
| MUL | R4, R2, R3 |
| ST  | c R4       |
| LD  | R5, 2      |
| ADD | R6, R4, R5 |
| ST  | d, R6      |



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



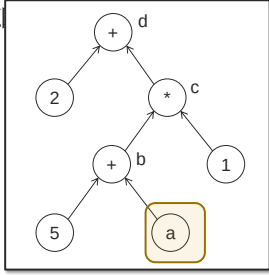
的? 一个例子

|     |            |
|-----|------------|
| LD  | R1, 5      |
| ADD | R2, R0, R1 |
| ST  | b R2       |
| LD  | R3, 1      |
| MUL | R4, R2, R3 |
| ST  | c R4       |
| LD  | R5, 2      |
| ADD | R6, R4, R5 |
| ST  | d, R6      |



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



的? 一个例子

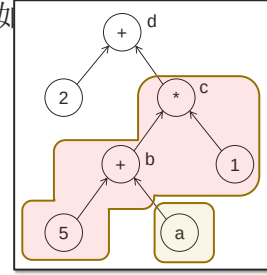
|     |     |        |
|-----|-----|--------|
| LD  | R1, | 5      |
| ADD | R2, | R0, R1 |
| ST  | b,  | R2     |
| LD  | R3, | 1      |
| MUL | R4, | R2, R3 |
| ST  | c,  | R4     |
| LD  | R5, | 2      |
| ADD | R6, | R4, R5 |
| ST  | d,  | R6     |

LD R0, a



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



的? 一个例子

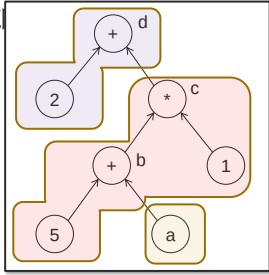
|     |     |        |
|-----|-----|--------|
| LD  | R1, | 5      |
| ADD | R2, | R0, R1 |
| ST  | b,  | R2     |
| LD  | R3, | 1      |
| MUL | R4, | R2, R3 |
| ST  | c,  | R4     |
| LD  | R5, | 2      |
| ADD | R6, | R4, R5 |
| ST  | d,  | R6     |

|     |     |       |
|-----|-----|-------|
| LD  | R0, | a     |
| ADD | R1, | R0, 5 |
| ST  | b,  | R1    |
| ST  | c,  | R1    |



## 基本块的优化

■  $b = a + 5$   
 $c = b * 1$   
 $d = 2 + c$



的? 一个例子

|     |     |        |
|-----|-----|--------|
| LD  | R1, | 5      |
| ADD | R2, | R0, R1 |
| ST  | b,  | R2     |
| LD  | R3, | 1      |
| MUL | R4, | R2, R3 |
| ST  | c,  | R4     |
| LD  | R5, | 2      |
| ADD | R6, | R4, R5 |
| ST  | d,  | R6     |

|     |     |       |
|-----|-----|-------|
| LD  | R0, | a     |
| ADD | R1, | R0, 5 |
| ST  | b,  | R1    |
| ST  | c,  | R1    |
| ADD | R2, | R1, 2 |
| ST  | d,  | R2    |



## 寄存器分配

### 实际情况

- $a = b + c$ 
  - LD R0, b
  - LD R1, c
  - ADD R2, R0, R1
  - ST a, R2
- 隐含的假设
  - 无限多寄存器
  - 不知道变量是否在寄存器中
  - 所有变量以后都有用

预先判断需要哪些加载指令把必须的运算分量加载进寄存器

有限的寄存器，加载时预判选用哪个寄存器

如果必要才把结果存入内存



## 寄存器分配

- 在一个基本块中的寄存器分配
- **MAXLIVE**: 每条指令处存活值的最大数量
  - 若  $\text{MAXLIVE} \leq k \rightarrow$  巨简单（无需溢出）！
  - 若  $\text{MAXLIVE} > k \rightarrow$  必须将部分值溢出到内存



## 寄存器分配

- 在一个基本块中的寄存器分配

|    |     |     |       |                                         |
|----|-----|-----|-------|-----------------------------------------|
| 1. | LD  | R1  | #1028 | // R1 <- 1028                           |
| 2. | LD  | R2  | *R1   | // R2 <- contents(R1), assume y         |
| 3. | MUL | R3  | R1 R2 | // R3 <- 1028 * y                       |
| 4. | LD  | R4  | x     | // R4 <- x                              |
| 5. | SUB | R5  | R4 R2 | // R5 <- x-y                            |
| 6. | LD  | R6  | z     | // R6 <- z                              |
| 7. | MUL | R7  | R5 R6 | // R7 <- z * (x-y)                      |
| 8. | SUB | R8  | R7 R3 | // R8 <- z * (x-y) - 1028 * y           |
| 9. | ST  | *R1 | R8    | // contents(R1) <- z * (x-y) - 1028 * y |



## 寄存器分配



```
1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y
```



## 寄存器分配



```
1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y
```



## 寄存器分配



```
1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y
```



## 寄存器分配



```
1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y
```



## 寄存器分配



```
1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y
```



## 寄存器分配



```
1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y
```





## 寄存器分配



```

1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y

```



## 寄存器分配



```

1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y

```



## 寄存器分配



```

1. LD R1 #1028 // R1
2. LD R2 *R1 // R1 R2
3. MUL R3 R1 R2 // R1 R2 R3
4. LD R4 x // R1 R2 R3 R4
5. SUB R5 R4 R2 // R1 R3 R5
6. LD R6 z // R1 R3 R5 R6
7. MUL R7 R5 R6 // R1 R3 R7
8. SUB R8 R7 R3 // R1 R8
9. ST *R1 R8 //

```



## 寄存器分配



```

1. LD R1 #1028 // R1 <- 1028
2. LD R2 *R1 // R2 <- contents(R1), assume y
3. MUL R3 R1 R2 // R3 <- 1028 * y
4. LD R4 x // R4 <- x
5. SUB R5 R4 R2 // R5 <- x-y
6. LD R6 z // R6 <- z
7. MUL R7 R5 R6 // R7 <- z * (x-y)
8. SUB R8 R7 R3 // R8 <- z * (x-y) - 1028 * y
9. ST *R1 R8 // contents(R1) <- z * (x-y) - 1028 * y

```



## 寄存器分配



```

1. LD R1 #1028 // R1
2. LD R2 *R1 // R1 R2
3. MUL R3 R1 R2 // R1 R2 R3
4. LD R4 x // R1 R2 R3 R4
5. SUB R5 R4 R2 // R1 R3 R5
6. LD R6 z // R1 R3 R5 R6
7. MUL R7 R5 R6 // R1 R3 R7
8. SUB R8 R7 R3 // R1 R8
9. ST *R1 R8 //

```



## 寄存器分配



```

1. LD R1 #1028 // R1
2. LD R2 *R1 // R1 R2
3. MUL R3 R1 R2 // R1 R2 R3
4. LD R4 x // R1 R2 R3 R4
5. SUB R5 R4 R2 // R1 R3 R5
6. LD R6 z // R1 R3 R5 R6
7. MUL R7 R5 R6 // R1 R3 R7
8. SUB R8 R7 R3 // R1 R8
9. ST *R1 R8 //

```

MAXLIVE = 4



## 寄存器分配

|        |     |       |    |                |
|--------|-----|-------|----|----------------|
| 1. LD  | R1  | #1028 |    | // R1          |
| 2. LD  | R2  | *R1   |    | // R1 R2       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |
| 5. SUB | R5  | R4    | R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     |    | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5    | R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7    | R3 | // R1 R8       |
| 9. ST  | *R1 | R8    |    | //             |

MAXLIVE = 4

4 个寄存器就够了! ?



## 寄存器分配

|        |     |       |    |                |
|--------|-----|-------|----|----------------|
| 1. LD  | R1  | #1028 |    | // R1          |
| 2. LD  | R2  | *R1   |    | // R1 R2       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |
| 5. SUB | R5  | R4    | R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     |    | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5    | R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7    | R3 | // R1 R8       |
| 9. ST  | *R1 | R8    |    | //             |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配

|        |     |       |    |                |
|--------|-----|-------|----|----------------|
| 1. LD  | R1  | #1028 |    | // R1          |
| 2. LD  | R2  | *R1   |    | // R1 R2       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |
| 5. SUB | R5  | R4    | R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     |    | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5    | R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7    | R3 | // R1 R8       |
| 9. ST  | *R1 | R8    |    | //             |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配

|        |     |       |    |                |
|--------|-----|-------|----|----------------|
| 1. LD  | R1  | #1028 |    | // R1          |
| 2. LD  | R2  | *R1   |    | // R1 R2       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |
| 5. SUB | R2  | R4    | R2 | // R1 R3 R2    |
| 6. LD  | R6  | z     |    | // R1 R3 R2 R6 |
| 7. MUL | R7  | R2    | R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7    | R3 | // R1 R8       |
| 9. ST  | *R1 | R8    |    | //             |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配

|        |     |       |    |                |
|--------|-----|-------|----|----------------|
| 1. LD  | R1  | #1028 |    | // R1          |
| 2. LD  | R2  | *R1   |    | // R1 R2       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |
| 5. SUB | R2  | R4    | R2 | // R1 R3 R2    |
| 6. LD  | R6  | z     |    | // R1 R3 R2 R6 |
| 7. MUL | R7  | R2    | R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7    | R3 | // R1 R8       |
| 9. ST  | *R1 | R8    |    | //             |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配

|        |     |       |    |                |
|--------|-----|-------|----|----------------|
| 1. LD  | R1  | #1028 |    | // R1          |
| 2. LD  | R2  | *R1   |    | // R1 R2       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |
| 5. SUB | R2  | R4    | R2 | // R1 R3 R2    |
| 6. LD  | R6  | z     |    | // R1 R3 R2 R6 |
| 7. MUL | R7  | R2    | R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7    | R3 | // R1 R8       |
| 9. ST  | *R1 | R8    |    | //             |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配



|        |     |       |    |                |       |
|--------|-----|-------|----|----------------|-------|
| 1. LD  | R1  | #1028 |    | // R1          |       |
| 2. LD  | R2  | *R1   |    | // R1 R2       |       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |       |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |       |
| 5. SUB | R2  | R4    | R2 | // R1 R3       | R2    |
| 6. LD  | R4  | z     |    | // R1 R3       | R2 R4 |
| 7. MUL | R7  | R2    | R4 | // R1 R3       | R7    |
| 8. SUB | R8  | R7    | R3 | // R1          | R8    |
| 9. ST  | *R1 | R8    |    | //             |       |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配



|        |     |       |    |                |       |
|--------|-----|-------|----|----------------|-------|
| 1. LD  | R1  | #1028 |    | // R1          |       |
| 2. LD  | R2  | *R1   |    | // R1 R2       |       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |       |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |       |
| 5. SUB | R2  | R4    | R2 | // R1 R3       | R2    |
| 6. LD  | R4  | z     |    | // R1 R3       | R2 R4 |
| 7. MUL | R7  | R2    | R4 | // R1 R3       | R7    |
| 8. SUB | R8  | R7    | R3 | // R1          | R8    |
| 9. ST  | *R1 | R8    |    | //             |       |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配



|        |     |       |    |                |       |
|--------|-----|-------|----|----------------|-------|
| 1. LD  | R1  | #1028 |    | // R1          |       |
| 2. LD  | R2  | *R1   |    | // R1 R2       |       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |       |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |       |
| 5. SUB | R2  | R4    | R2 | // R1 R3       | R2    |
| 6. LD  | R4  | z     |    | // R1 R3       | R2 R4 |
| 7. MUL | R2  | R2    | R4 | // R1 R3       | R2    |
| 8. SUB | R8  | R2    | R3 | // R1          | R8    |
| 9. ST  | *R1 | R8    |    | //             |       |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配



|        |     |       |    |                |       |
|--------|-----|-------|----|----------------|-------|
| 1. LD  | R1  | #1028 |    | // R1          |       |
| 2. LD  | R2  | *R1   |    | // R1 R2       |       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |       |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |       |
| 5. SUB | R2  | R4    | R2 | // R1 R3       | R2    |
| 6. LD  | R4  | z     |    | // R1 R3       | R2 R4 |
| 7. MUL | R2  | R2    | R4 | // R1 R3       | R2    |
| 8. SUB | R8  | R2    | R3 | // R1          | R8    |
| 9. ST  | *R1 | R8    |    | //             |       |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配



|        |     |       |    |                |       |
|--------|-----|-------|----|----------------|-------|
| 1. LD  | R1  | #1028 |    | // R1          |       |
| 2. LD  | R2  | *R1   |    | // R1 R2       |       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |       |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |       |
| 5. SUB | R2  | R4    | R2 | // R1 R3       | R2    |
| 6. LD  | R4  | z     |    | // R1 R3       | R2 R4 |
| 7. MUL | R2  | R2    | R4 | // R1 R3       | R2    |
| 8. SUB | R2  | R2    | R3 | // R1          | R2    |
| 9. ST  | *R1 | R2    |    | //             |       |

MAXLIVE = 4

if  $k \geq 4$ , e.g.,  $k = 4$



## 寄存器分配



|        |     |       |    |                |       |
|--------|-----|-------|----|----------------|-------|
| 1. LD  | R1  | #1028 |    | // R1          |       |
| 2. LD  | R2  | *R1   |    | // R1 R2       |       |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    |       |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |       |
| 5. SUB | R5  | R4    | R2 | // R1 R3       | R5    |
| 6. LD  | R6  | z     |    | // R1 R3       | R5 R6 |
| 7. MUL | R7  | R5    | R6 | // R1 R3       | R7    |
| 8. SUB | R8  | R7    | R3 | // R1          | R8    |
| 9. ST  | *R1 | R8    |    | //             |       |

MAXLIVE = 4

if  $k < 4$ , e.g.,  $k = 3$



## 寄存器分配

|        |     |       |                |
|--------|-----|-------|----------------|
| 1. LD  | R1  | #1028 | // R1          |
| 2. LD  | R2  | *R1   | // R1 R2       |
| 3. MUL | R3  | R1 R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     | // R1 R2 R3 R4 |
| 5. SUB | R5  | R4 R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5 R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7 R3 | // R1 R8       |
| 9. ST  | *R1 | R8    | //             |

MAXLIVE = 4

if  $k < 4$ , e.g.,  $k = 3$



## 寄存器分配

|        |     |       |                |
|--------|-----|-------|----------------|
| 1. LD  | R1  | #1028 | // R1          |
| 2. LD  | R2  | *R1   | // R1 R2       |
| 3. MUL | R3  | R1 R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     | // R1 R2 R3 R4 |
| 5. SUB | R5  | R4 R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5 R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7 R3 | // R1 R8       |
| 9. ST  | *R1 | R8    | //             |

MAXLIVE = 4

if  $k < 4$ , e.g.,  $k = 3$



## 寄存器分配

在一个基本块中的寄存器分配

|        |     |       |                |
|--------|-----|-------|----------------|
| 1. LD  | R1  | #1028 | // R1          |
| 2. LD  | R2  | *R1   | // R1 R2       |
| 3. MUL | R3  | R1 R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     | // R1 R2 R3 R4 |
| 5. SUB | R5  | R4 R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5 R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7 R3 | // R1 R8       |
| 9. ST  | *R1 | R8    | //             |

MAXLIVE = 4

if  $k < 4$ , e.g.,  $k = 3$



## 寄存器分配

|        |     |       |                |
|--------|-----|-------|----------------|
| 1. LD  | R1  | #1028 | // R1          |
| 2. LD  | R2  | *R1   | // R1 R2       |
| 3. MUL | R3  | R1 R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     | // R1 R2 R3 R4 |
| 5. SUB | R5  | R4 R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5 R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7 R3 | // R1 R8       |
| 9. ST  | *R1 | R8    | //             |

MAXLIVE = 4

if  $k < 4$ , e.g.,  $k = 3$



## 寄存器分配

|        |     |       |                |
|--------|-----|-------|----------------|
| 1. LD  | R1  | #1028 | // R1          |
| 2. LD  | R2  | *R1   | // R1 R2       |
| 3. MUL | R3  | R1 R2 | // R1 R2 R3    |
| 4. LD  | R4  | x     | // R1 R2 R3 R4 |
| 5. SUB | R5  | R4 R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5 R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7 R3 | // R1 R8       |
| 9. ST  | *R1 | R8    | //             |

MAXLIVE = 4

if  $k < 4$ , e.g.,  $k = 3$

重新加载 R3, LD R3 v



## 寄存器分配

|        |     |       |                |
|--------|-----|-------|----------------|
| 1. LD  | R1  | #1028 | // R1          |
| 2. LD  | R2  | *R1   | // R1 R2       |
| 3. MUL | R3  | R1 R2 | // R1 R2 R3    |
| 4. LD  | R3  | x     | // R1 R2 R3    |
| 5. SUB | R5  | R3 R2 | // R1 R3 R5    |
| 6. LD  | R6  | z     | // R1 R3 R5 R6 |
| 7. MUL | R7  | R5 R6 | // R1 R3 R7    |
| 8. SUB | R8  | R7 R3 | // R1 R8       |
| 9. ST  | *R1 | R8    | //             |

MAXLIVE = 4

if  $k < 4$ , e.g.,  $k = 3$

重新加载 R3, LD R3 v



## 寄存器分配

|        |     |       |    |                |                            |  |  |  |
|--------|-----|-------|----|----------------|----------------------------|--|--|--|
| 1. LD  | R1  | #1028 |    | // R1          |                            |  |  |  |
| 2. LD  | R2  | *R1   |    | // R1 R2       |                            |  |  |  |
| 3. MUL | R3  | R1    | R2 | // R1 R2 R3    | 需要考虑将 R3 中的内容进行转存, ST v R3 |  |  |  |
| 4. LD  | R4  | x     |    | // R1 R2 R3 R4 |                            |  |  |  |
| 5. SUB | R5  | R4    | R2 | // R1 R3 R5    |                            |  |  |  |
| 6. LD  | R6  | z     |    | // R1 R3 R5 R6 |                            |  |  |  |
| 7. MUL | R7  | R5    | R6 | // R1 R3 R7    |                            |  |  |  |
| 8. SUB | R8  | R7    | R3 | // R1 R8       |                            |  |  |  |
| 9. ST  | *R1 | R8    |    | //             |                            |  |  |  |

MAXLIVE = 4

if  $k < 4$ , e.g.,  $k = 3$



## 寄存器分配

- 记录各个值对应的位置
  - 寄存器描述符: 跟踪各个寄存器都存放了哪些变量的当前值
  - 地址描述符: 某个变量的当前值存放在哪个或哪些位置(包括内存位置和寄存器)



## 寄存器分配

- 运算语句:  $x = y + z$ 
  - 调用  $\text{getReg}(x=y+z)$ , 为  $x, y, z$  选择寄存器  $R_x, R_y, R_z$
  - 查  $R_y$  的寄存器描述符, 如果  $y$  不在  $R_y$  中则生成指令
    - LD  $R_y, y'$  ( $y'$  表示存放  $y$  值的当前位置)
  - 类似地, 确定是否生成 LD  $R_z, z'$
  - 生成指令 ADD  $R_x, R_y, R_z$
- 赋值语句:  $x = y$ 
  - $\text{getReg}(x=y)$  总是为  $x$  和  $y$  选择相同的寄存器
  - 如果  $y$  不在  $R_y$  中, 生成机器指令 LD  $R_y, y$
- 基本块的收尾
  - 如果变量  $x$  在出口处活跃, 且  $x$  现在不在内存, 那么生成指令 ST  $x, R_x$



## 寄存器分配

- 依次考虑各三地址指令, 尽可能把值保留在寄存器中, 以减少寄存器/内存之间的数据交换
- 为一个三地址指令生成机器指令时
  - 只有当运算分量不在寄存器中时, 才从内存载入
  - 尽量保证只有当寄存器中的值不被使用时, 才把它覆盖掉



## 寄存器分配

- 代码生成算法框架
  - 遍历三地址指令
    - 使用  $\text{getReg}()$  函数选择寄存器
    - 生成指令
- 重要子函数:  $\text{getReg}(I)$ 
  - 根据寄存器描述符和地址描述符、数据流信息, 为三地址指令  $I$  选择最佳的寄存器
  - 得到的机器指令的质量依赖于  $\text{getReg}$  函数选取寄存器的算法



## 寄存器分配

- 代码生成同时更新寄存器和地址描述符
- 处理普通指令时生成 LD  $R, x$ 
  - $R$  的寄存器描述符: 只包含  $x$
  - $x$  的地址描述符:  $R$  作为新位置加入到  $x$  的位置集合中
  - 从任何不同于  $x$  的变量的地址描述符中删除  $R$
- ST  $x, R$ 
  - 修改  $x$  的地址描述符, 包含自己的内存位置



## 寄存器分配



- ADD Rx, Ry, Rz
  - Rx的寄存器描述符只包含x
  - x的地址描述符只包含Rx（不包含x的内存位置！）
  - 从任何不同于x的变量的地址描述符中删除Rx。
- 处理x=y时，
  - 如果生成LD Ry y，按照第一个规则处理；
  - 把x加入到Ry的寄存器描述符中（Ry存放了x和y的当前值）；
  - 修改x的地址描述符，使它只包含Ry



## 寄存器分配



- a、b、c、d在出口处活跃
- t、u、v是局部临时变量

```

t = a - b
u = a - c
v = t + u
a = d
d = v + u

```



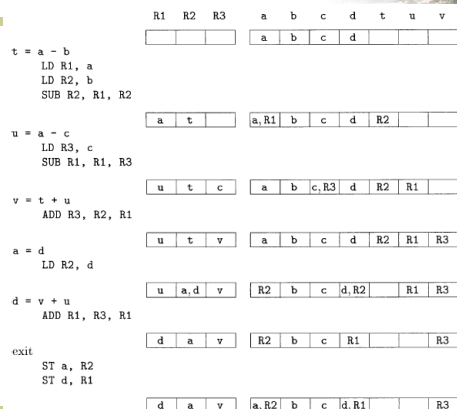
## 寄存器分配



```

t = a - b
u = a - c
v = t + u
a = d
d = v + u

```



生成的指令以及寄存器和地址描述符的改变过程



## 寄存器分配



- getReg的目标：减少LD/ST指令
- 为运算分量y分配寄存器（如果不在寄存器中，且没有空闲寄存器）
  - 寻找一个寄存器R，假设已经有某个变量v在R中了（R的寄存器描述符中包含了v）
    - 如果v的地址描述符表明还可以在别的地方找到v，DONE
    - v就是x（即结果），且x不是运算分量之一，DONE
    - 如果v在此之后不会被使用，DONE
    - 生成保存指令ST v R（溢出操作）并修改v的地址描述符，如果R中存放了多个变量的值，那么需要生成多条ST指令（选尽可能少ST的）；

- 关键：借用了v所在的寄存器，不能随意把v冲掉



## 寄存器分配



- getReg的目标：减少LD/ST指令
- 任务：为运算分量和结果分配寄存器
- 为  $x = y + z$  指令选择寄存器
- 为运算分量y分配寄存器
  - 如果已经在某个寄存器中，不需要进行处理，选择这个寄存器
  - 如果不在寄存器中，且有空闲寄存器，选择一个空闲寄存器
  - 如果不在寄存器中，且没有空闲寄存器？



## 寄存器分配



- 为x选择寄存器Rx（基本方法和把y从内存LD时一样）
- 区别
  - 只存放x的值的寄存器总是可接受的，因为x计算出了一个新的值
  - 如果y在指令I之后不再使用，且Ry仅仅保存了y的值，那么Ry同时也可以作为Rx
- 处理x=y时，我们总是让Rx=Ry





## 寄存器分配

- 看一个示例习题： $d := (a-b)+(a-c)+(a-c)$ 的三地址代码序列：



## 寄存器分配

- 看一个示例习题： $d := (a-b)+(a-c)+(a-c)$ 的三地址代码序列：

$t_1 := a - b$   
 $t_2 := a - c$   
 $t_3 := t_1 + t_2$   
 $d := t_3 + t_2$

假设只有两个寄存器，做寄存器分配？



## 寄存器分配

| statements         | code generated                      | register descriptor | address descriptor         |
|--------------------|-------------------------------------|---------------------|----------------------------|
| $t_1 := a - b$     | LD R0,a<br>LD R1,b<br>SUB R1,R0,R1  | R0含a<br>R1含 $t_1$   | a in R0<br>$t_1$ in R1     |
| $t_2 := a - c$     | ST t1,R1<br>LD R1,c<br>SUB R0,R0,R1 | R1含c<br>R0含 $t_2$   | c in R1<br>$t_2$ in R0     |
| $t_3 := t_1 + t_2$ | LD R1,t1<br>ADD R1,R1,R0            | R1含 $t_3$           | $t_3$ in R1<br>$t_2$ in R0 |
| $d := t_3 + t_2$   | ADD R1,R1,R0                        | R1含d<br>R0含 $t_2$   | d in R1<br>$t_2$ in R0     |
|                    | ST d,R1                             |                     | d在R1和内存中                   |



(1)  $t = a * b$  (指令1分，描述符1分)

LD R1,a

LD R2,b

MUL R1,R1,R2

|    |    |   |   |      |    |   |
|----|----|---|---|------|----|---|
| R1 | R2 | x | a | b    | t  | u |
| T  | b  | x | a | b,R2 | R1 | u |

(2)  $u = x + t$  (指令2分)

LD R2,x

ADD R2 R1 R2

|    |    |   |   |   |    |    |
|----|----|---|---|---|----|----|
| R1 | R2 | x | a | b | t  | u  |
| T  | u  | x | a | b | R1 | R2 |

(3)  $x = u$  (描述符3分)

|    |     |    |   |   |    |    |
|----|-----|----|---|---|----|----|
| R1 | R2  | x  | a | b | t  | u  |
| T  | u,x | R2 | a | b | R1 | R2 |

(4) (指令3分)

ST t,R1

ST u,R2

ST x,R2

|    |     |      |   |   |      |      |
|----|-----|------|---|---|------|------|
| R1 | R2  | x    | a | b | t    | u    |
| t  | u,x | x,R2 | a | b | t,R1 | u,R2 |



八、简答题（共1题，共10分）

假设有两个可用的寄存器 R1 和 R2，使用简单代码生成算法，

为如下三地址代码生成机器代码。

$t = a * b$

$u = x + t$

$x = u$

- 给出每条三地址代码生成机器码后的寄存器描述符和内存描述符（请在表格左边空白部分写出三地址代码，并在表格中补充写入对应的描述符内容）。

- 结束时将寄存器中的变量写回内存中。

附机器代码格式：

取数指令：LD R, var // 将内存空间中变量 var 的值存入寄存器 R 中。

存数指令：ST var, R // 将寄存器 R 中的值存入变量 var 的内存空间中。

双目运算指令：OP dest, src1, src2 // 使用 OP 对寄存器 src1 和 src2 中的值进行计算，结果存入寄存器 dest 中（OP 包括 ADD、SUB、MUL、DIV）。

初始状态：

|    |    |   |   |   |   |   |
|----|----|---|---|---|---|---|
| R1 | R2 | x | a | b | t | u |
|    |    | x | a | b | t | u |

(1)  $t = a * b$

|    |    |   |   |   |    |   |
|----|----|---|---|---|----|---|
| R1 | R2 | x | a | b | t  | u |
|    |    | x | a | b | R1 | u |

(2)  $u = x + t$

|    |    |   |   |   |    |    |
|----|----|---|---|---|----|----|
| R1 | R2 | x | a | b | t  | u  |
| t  | u  | x | a | b | R1 | R2 |

(3)  $x = u$

|    |    |   |   |   |    |   |
|----|----|---|---|---|----|---|
| R1 | R2 | x | a | b | t  | u |
| t  |    | x | a | b | R1 |   |

(4)

|    |     |      |   |   |      |      |
|----|-----|------|---|---|------|------|
| R1 | R2  | x    | a | b | t    | u    |
| t  | u,x | x,R2 | a | b | t,R1 | u,R2 |



# 复习课（七） （机器无关优化）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn



## 优化的来源



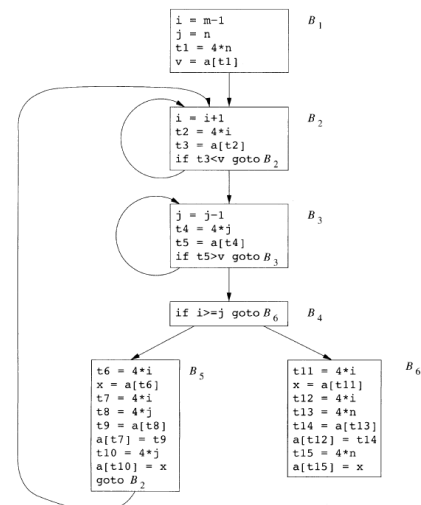
- （一）优化主要来源
- 编译器只能通过一些相对低层的语义等价转换来优化代码
- 冗余运算的原因
  - 源程序中的冗余
  - 高级程序设计语言编程的副产品
    - 比如  $A[i][j].f = 0; A[i][j].k = 1;$  中的冗余运算
- 语义不变的优化
  - 公共子表达式消除
  - 复制传播
  - 死代码消除
  - 常量折叠



## 优化的来源



- （一）优化主要来源
- Quicksort 的流程图
  - $B_2$
  - $B_3$
  - $B_2$ 、 $B_3$ 、 $B_4$ 、 $B_5$



## 引言



- 代码优化
  - 在目标代码中 **消除** 不必要的指令
  - 把一个指令序列 **替换** 为一个完成相同功能的更快的指令序列
- 全局优化
- 基于数据流分析技术
  - 用以收集程序相关信息的算法



## 优化的来源



- （一）优化主要来源
- 三地址代码
 

|      |                       |      |                |
|------|-----------------------|------|----------------|
| (1)  | $i = m-1$             | (16) | $t7 = 4*i$     |
| (2)  | $j = n$               | (17) | $t8 = 4*j$     |
| (3)  | $t1 = 4*n$            | (18) | $t9 = a[t8]$   |
| (4)  | $v = a[t1]$           | (19) | $a[t7] = t9$   |
| (5)  | $i = i+1$             | (20) | $t10 = 4*j$    |
| (6)  | $t2 = 4*i$            | (21) | $a[t10] = x$   |
| (7)  | $t3 = a[t2]$          | (22) | goto (5)       |
| (8)  | if $t3 < v$ goto (5)  | (23) | $t11 = 4*i$    |
| (9)  | $j = j-1$             | (24) | $x = a[t11]$   |
| (10) | $t4 = 4*j$            | (25) | $t12 = 4*i$    |
| (11) | $t5 = a[t4]$          | (26) | $t13 = 4*n$    |
| (12) | if $t5 > v$ goto (9)  | (27) | $t14 = a[t13]$ |
| (13) | if $i >= j$ goto (23) | (28) | $a[t12] = t14$ |
| (14) | $t6 = 4*i$            | (29) | $t15 = 4*n$    |
| (15) | $x = a[t6]$           | (30) | $a[t15] = x$   |



## 数据流分析



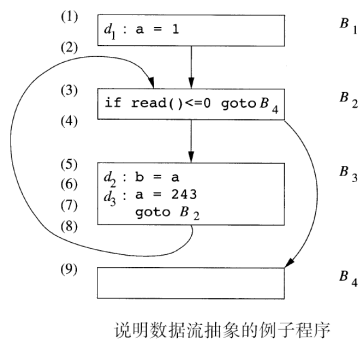
- 数据流分析
  - 用于获取数据沿着程序执行路径流动的信息的相关技术
  - 是优化的基础
- 例如
  - 两个表达式是否一定计算得到相同的值？（公共子表达式）
  - 一个语句的计算结果有没有可能被后续语句使用？（死代码消除）



- 数据流抽象
- 程序点
  - 三地址语句之前或之后的位置
  - 基本块内部：一个语句之后的程序点等于下一个语句之前的程序点
  - 如果流图中有 $B_1$ 到 $B_2$ 的边，那么 $B_2$ 的第一个语句之前的点可能紧跟在 $B_1$ 的最后语句之后的点后面执行
- 从 $p_1$ 到 $p_2$ 的执行路径：  $p_1, p_2, \dots, p_n$ 
  - 要么 $p_i$ 是一个语句之前的点，且 $p_{i+1}$ 是该语句之后的点
  - 要么 $p_i$ 是某个基本块的结尾，且 $p_{i+1}$ 是该基本块的某个后继的开头



- 示例：
- 路径
  - 1, 2, 3, 4, 9
  - 1, 2, 3, 4, 5, 6, 7, 8, 9
  - ...
- 第一次到达(5)， $a=1$ ；第二次到达(5)， $a=243$ ；且之后都是243
- 我们可以说：
  - 点(5)具有特性 $a=1$  or  $a=243$
  - 表示成为 $\langle a, \{1, 243\} \rangle$



- 基本块的控制流非常简单
  - 从头到尾不会中断
  - 没有分支
- 基本块的效果就是各个语句的效果的复合
- 可以预先处理基本块内部的数据流关系，给出基本块对应的传递函数；
 
$$IN[B] = f_B(OUT[B]) \quad \text{或者} \quad OUT[B] = f_B(IN[B])$$
- 设基本块包含语句 $s_1, s_2, \dots, s_n$ 

$$f_B = f_{s_n} \circ \dots \circ f_{s_2} \circ f_{s_1}$$



- 数据流抽象
- 出现在某个程序点的程序状态
  - 在某个运行时刻，当指令指针指向这个程序点时，各个变量和动态内存中存放的值
  - 指令指针可能多次指向同一个程序点
    - 因此一个程序点可能对应多个程序状态
- 数据流分析把可能出现在某个程序点上的程序状态集合总结为一些特性
  - 不管程序怎么运行，当它到达某个程序点时，程序状态总是满足分析得到的特性
  - 不同的分析技术关心不同的信息
- 为了高效、自动地进行数据流分析，通常要求这些特性能够被高效地表示和求解



- 数据流分析模式
- 数据流分析：对一组约束求解
  - $IN[s]$ 和 $OUT[s]$
- 基于语句语义的约束(传递函数)
  - $IN[s] = f_s(OUT[s])$  (逆向)
  - $OUT[s] = f_s(IN[s])$  (正向)
- 基于控制流的约束
  - $IN[s_{i+1}] = OUT[s_i]$



- 到达定值分析
- 活跃变量分析
- 可用表达式分析



- 到达定值
  - 如果存在一条从定值d后面的程序点到达某个点p的路径，且这条路径上d没有被杀死，那么定值d到达p
- 杀死：路径上对x的其他定值杀死了之前对x的定值
- 直观含义
  - 如果d到达p，那么在p点使用的值可能就是由d定值的
- 思考：不确定是否赋值该怎么办？
  - $*p = 3$  （不确定p的指向）
  - 过程参数、数组、指针等等



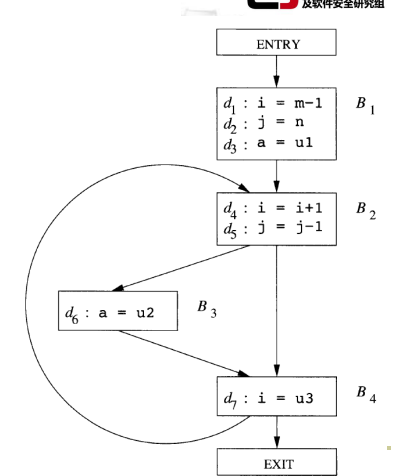
- 语句/基本块的传递方程（1）
- 定值  $d: u=v+w$ 
  - 生成了对变量u的定值d，杀死了其他对u的定值
  - 生成-杀死：
  - $OUT[B] = f_B(IN[B])$
  - $f_d(x) = gen_d \cup (x - kill_d)$ 
    - 把  $x = IN[B]$  带入
    - 特别地，其中  $gen_d = \{d\}$ ,  $kill_d = \{\text{程序中其他对u的定值}\}$



- 语句/基本块的传递方程（2）
- 设B有n个语句，第i个语句的传递函数为  $f_i$
- $f_B(x) = gen_B \cup (x - kill_B)$
- $gen_B = gen_n \cup (gen_{n-1} - kill_n) \cup (gen_{n-2} - kill_{n-1} - kill_n) \cup (gen_1 - kill_2 - kill_3 \dots - kill_n)$
- $kill_B = kill_1 \cup kill_2 \cup \dots \cup kill_n$
- $kill_B$  为被B各个语句杀死的定值的并集
- $gen_B$  是被第i个语句生成，且没有被其后的句子杀死的定值的集合



- 示例：
  - $B_1$  中全部定值到达  $B_2$  的开头
  - $d_5$  到达  $B_2$  的开头（循环）
  - $d_2$  被  $d_5$  杀死，不能到达  $B_3$ 、 $B_4$  的开头
  - $d_4$  不能到达  $B_2$  的开头，因为被  $d_7$  杀死
  - $d_6$  到达  $B_2$  的开头



- 语句/基本块的传递方程（1）
- $OUT[B] = gen_d \cup (IN[B] - kill_d)$
- 生成-杀死形式的函数的并置（复合）
  - $f_2(f_1(x)) = gen_2 \cup (gen_1 \cup (x - kill_1) - kill_2)$   
 $= (gen_2 \cup (gen_1 - kill_2)) \cup (x - kill_1 \cup kill_2)$
  - 生成的定值：由第二部分生成、以及由第一个部分生成且没有被第二部分杀死
  - 杀死的定值：被第一部分杀死的定值、以及被第二部分杀死的定值



- 到达定值的控制流方程：
- 只要一个定值能够沿某条路径到达一个程序点，这个定值就是到达定值
- $IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱基本块}} OUT[P]$ 
  - 如果从基本块P到B有一条控制流边，那么  $OUT[P]$  在  $IN[B]$  中
  - 一个定值必然先在某个前驱的OUT值中，才能出现在B的IN中
- $\cup$  称为到达定值的交汇运算符



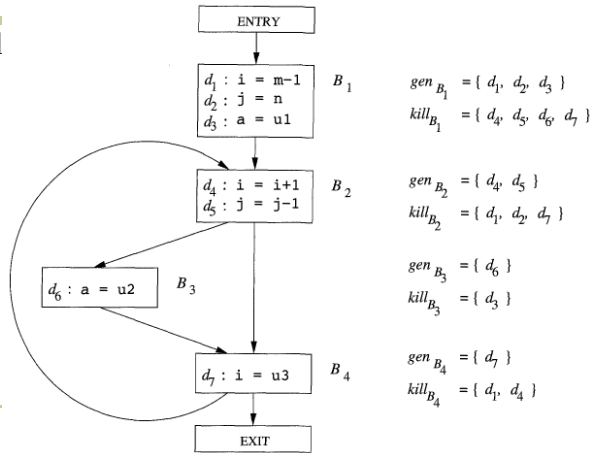
## 数据流分析：到达定值

- 控制流方程的迭代解法（1）：
- ENTRY基本块的传递函数是常函数  
 $OUT[ENTRY] = \text{空集}$
- 其他基本块  
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$   
 $IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱基本块}} OUT[P]$
- 迭代解法
  - 首先求出各基本块的gen和kill
  - 令所有的OUT[B]都是空集，然后不停迭代，得到最小不动点的解

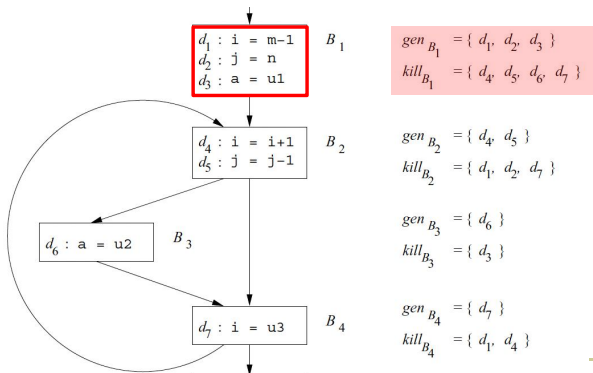


## 数据流分析：到达定值

### ■ gen和kil

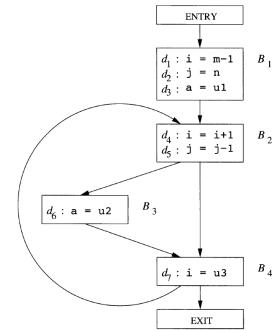


## 数据流分析：到达定值

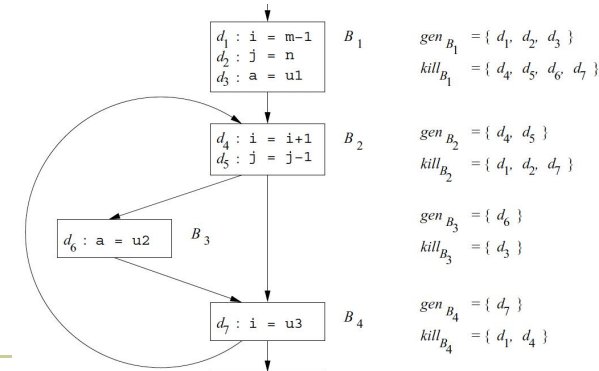


## 数据流分析：到达定值

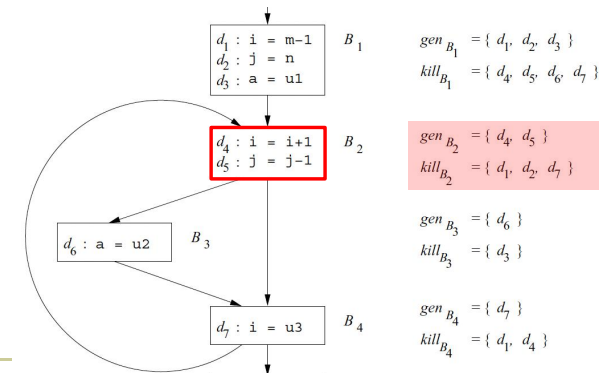
- 控制流方程的迭代解法（2）：
- 输入：流图、各基本块的kill和gen集合
- 输出：IN[B] 和 OUT[B]
- 方法：
  - 1)  $OUT[ENTRY] = \emptyset$ ;
  - 2) **for** (除 ENTRY 之外的每个基本块  $B$ )  $OUT[B] = \emptyset$ ;
  - 3) **while** (某个 OUT 值发生了改变)
  - 4)     **for** (除 ENTRY 之外的每个基本块  $B$ ) {
  - 5)          $IN[B] = \bigcup_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$ ;
  - 6)          $OUT[B] = gen_B \cup (IN[B] - kill_B)$ ;
  - }



## 数据流分析：到达定值

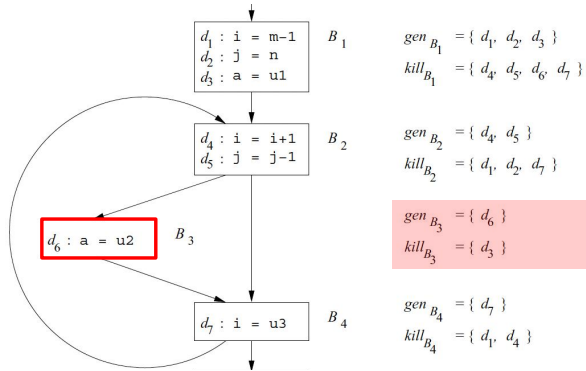


## 数据流分析：到达定值

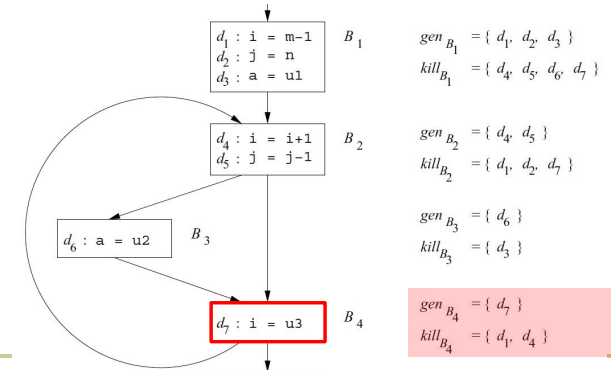




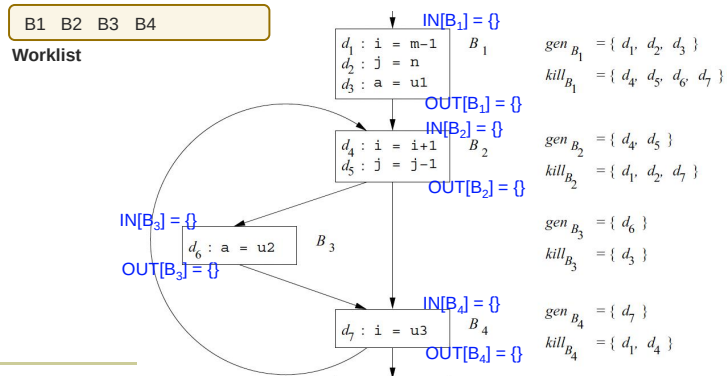
## 数据流分析：到达定值



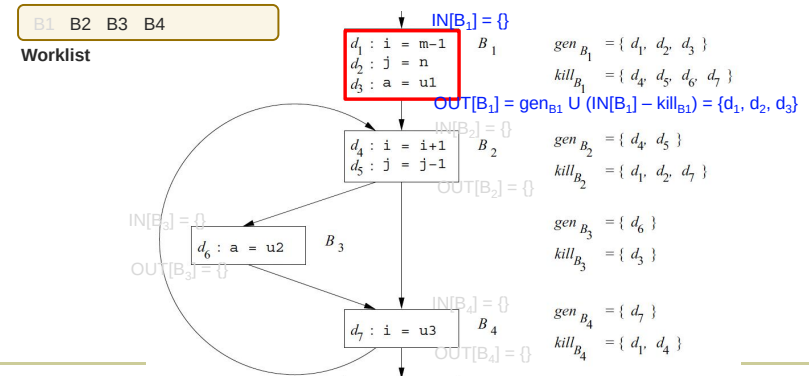
## 数据流分析：到达定值



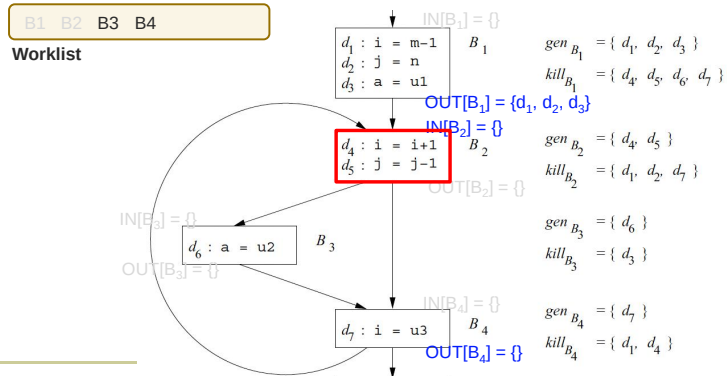
## 数据流分析：到达定值



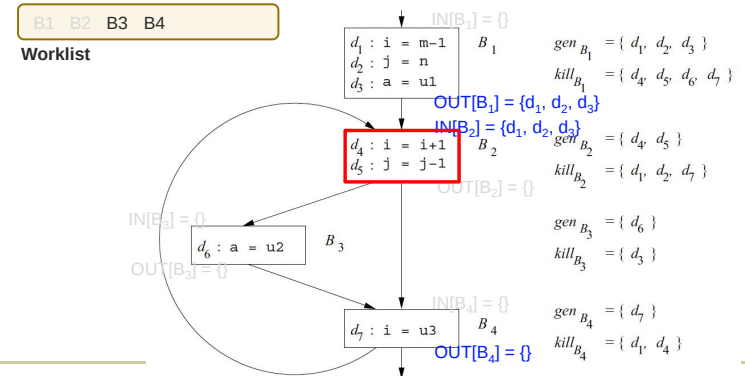
## 数据流分析：到达定值



## 数据流分析：到达定值



## 数据流分析：到达定值



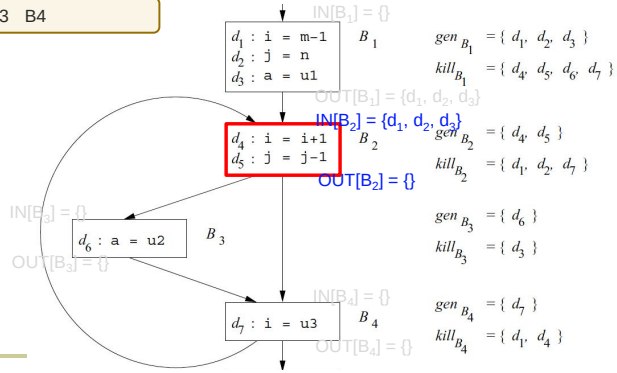




## 数据流分析：到达定值

B1 B2 B3 B4

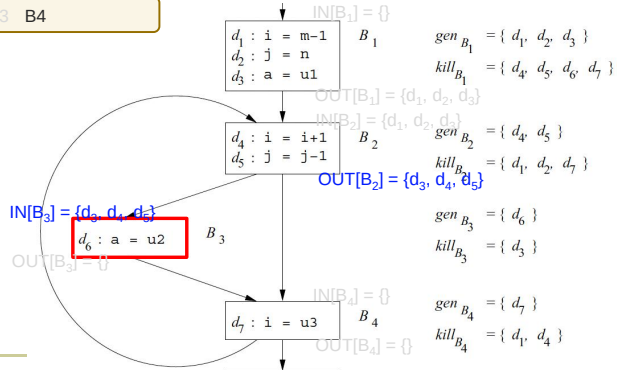
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

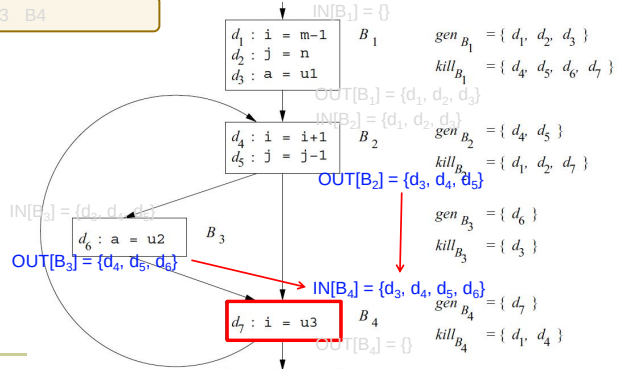
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

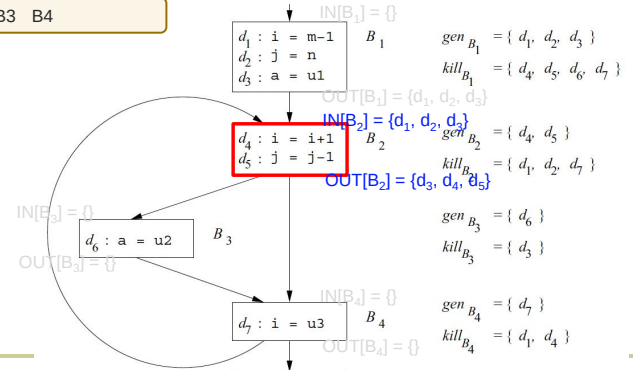
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

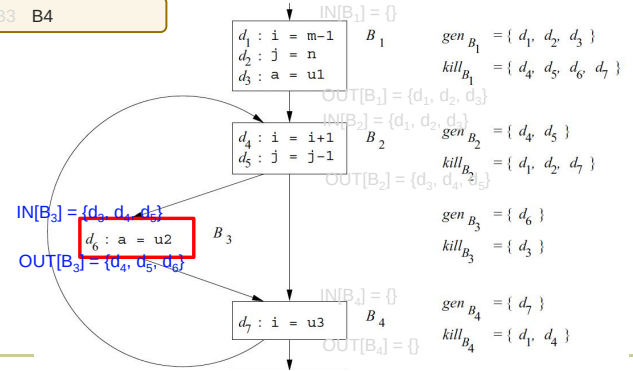
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

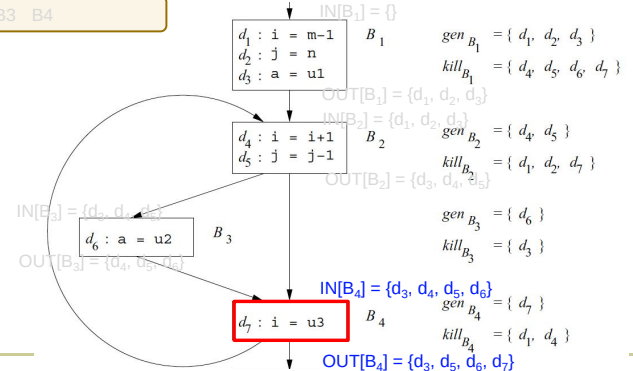
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

Worklist

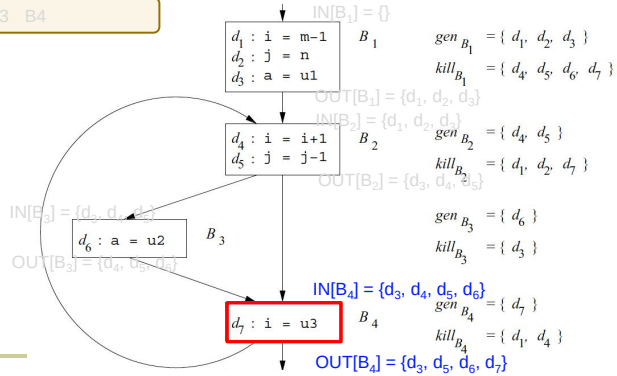




## 数据流分析：到达定值

B1 B2 B3 B4

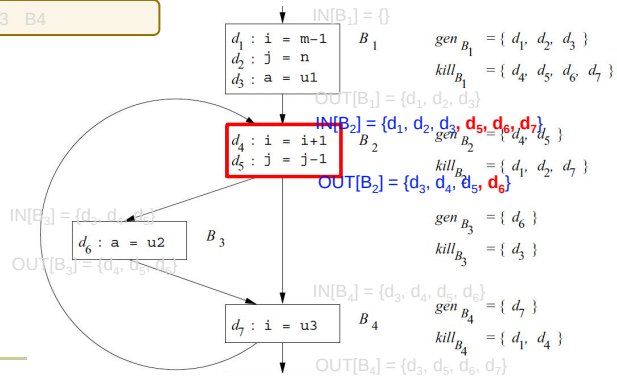
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

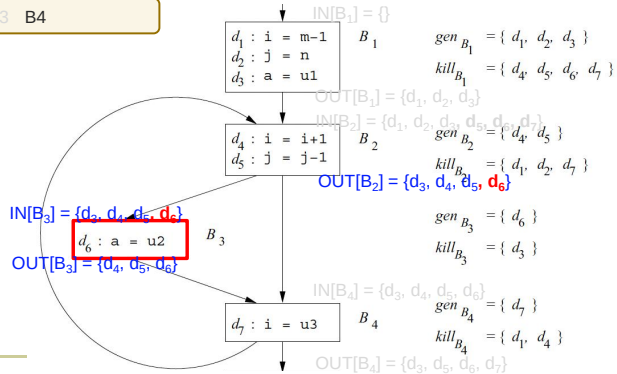
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

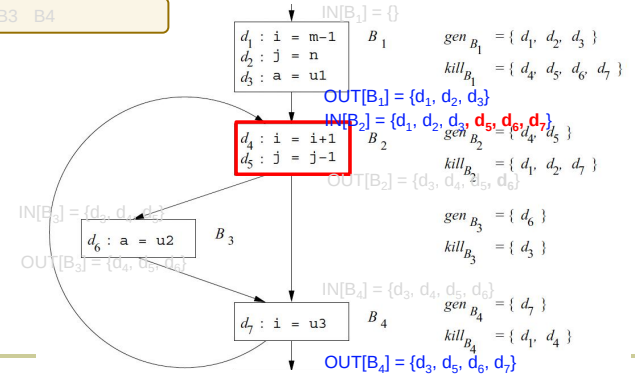
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

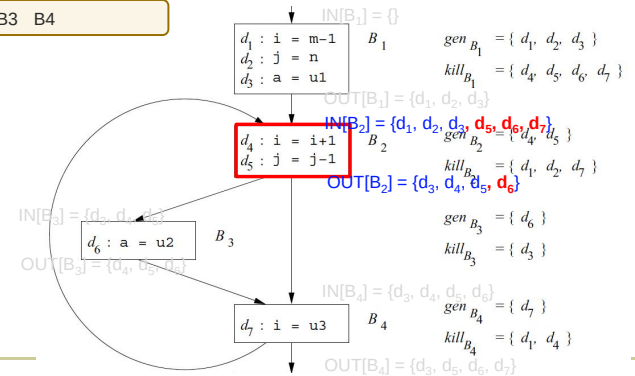
Worklist



## 数据流分析：到达定值

B1 B2 B3 B4

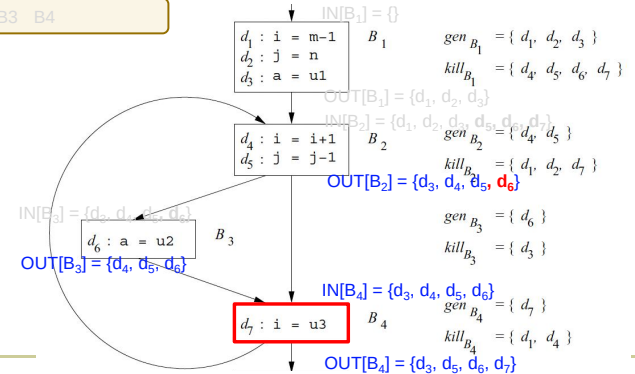
Worklist



## 数据流分析：到达定值

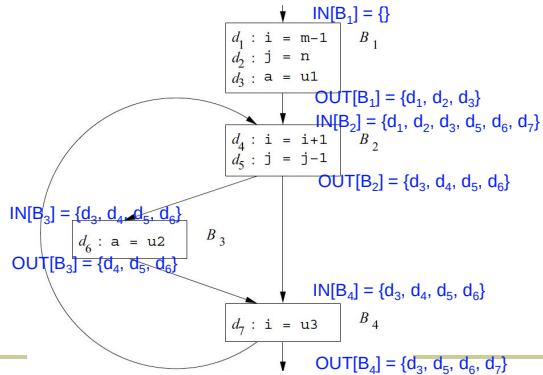
B1 B2 B3 B4

Worklist





## 数据流分析：到达定值



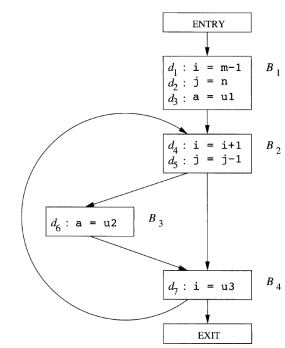
## 数据流分析：到达定值

- 控制流方程的迭代解法（3）
- 解法的正确性
  - 直观解释：不断向前传递各个定值，直到该定值被杀死为止
- 算法为什么能终止？
  - 各个OUT[B]在算法执行过程中不会变小
  - 且OUT[B]显然有有穷的上界
  - 只有一次迭代之后增大了某个OUT[B]的值，算法才会进行下一次迭代
- 最大迭代次数是流图的结点数n
  - 定值经过n步必然已经到达所有可能到达的结点
- 算法结束时，各个OUT/IN值必然满足数据流方程



## 数据流分析：到达定值

- 到达定值求解示例
- 7个bit从左到右表示 $d_1, d_2, \dots, d_n$
- for循环时依次遍历 $B_1, B_2, B_3, B_4$
- 每一列表示一次迭代计算；
- $B_1$ 生成 $d_1, d_2, d_3$ ，杀死 $d_4, d_5, d_6, d_7$
- $B_2$ 生成 $d_4, d_5$ ，杀死 $d_1, d_2, d_7$
- $B_3$ 生成 $d_6$ ，杀死 $d_3$
- $B_4$ 生成 $d_7$ ，杀死 $d_1, d_4$



| Block B        | OUT[B] <sup>0</sup> | IN[B] <sup>1</sup> | OUT[B] <sup>1</sup> | IN[B] <sup>2</sup> | OUT[B] <sup>2</sup> |
|----------------|---------------------|--------------------|---------------------|--------------------|---------------------|
| B <sub>1</sub> | 000 0000            | 000 0000           | 111 0000            | 000 0000           | 111 0000            |
| B <sub>2</sub> | 000 0000            | 111 0000           | 001 1100            | 111 0111           | 001 1110            |
| B <sub>3</sub> | 000 0000            | 001 1100           | 000 1110            | 001 1110           | 000 1110            |
| B <sub>4</sub> | 000 0000            | 001 1110           | 001 0111            | 001 1110           | 001 0111            |
| EXIT           | 000 0000            | 001 0111           | 001 0111            | 001 0111           | 001 0111            |

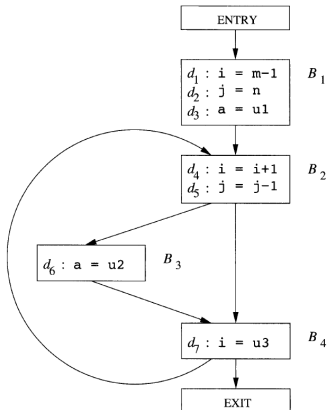


## 数据流分析：活跃变量分析

- 活跃变量分析
  - x在p上的值是否会在某条从p出发的路径中使用
  - 一个变量x在p上活跃，当且仅当存在一条从p点开始的路径，该路径的末端使用了x，且路径上没有对x进行覆盖
- 用途
  - 寄存器分配/死代码删除/无用赋值...
  - （考虑我们前面讲的优化）
- 数据流值
  - （活跃）变量的集合



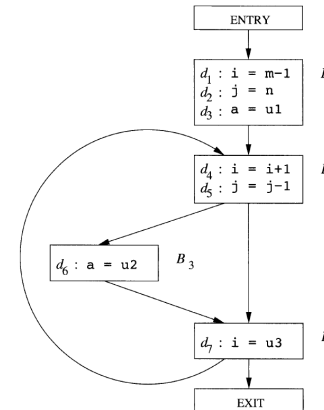
## 数据流分析：活跃变量分析



| OUT[B] | B1 | B2 | B3 | B4 |
|--------|----|----|----|----|
| a      |    |    |    |    |
| i      |    |    |    |    |
| j      |    |    |    |    |
| m      |    |    |    |    |
| n      |    |    |    |    |
| u1     |    |    |    |    |
| u2     |    |    |    |    |
| u3     |    |    |    |    |



## 数据流分析：活跃变量分析



| OUT[B] | B1 | B2 | B3 | B4 |
|--------|----|----|----|----|
| a      | x  | x  | x  | x  |
| i      | √  | x  | x  | √  |
| j      | √  | √  | √  | √  |
| m      | x  | x  | x  | x  |
| n      | x  | x  | x  | x  |
| u1     | x  | x  | x  | x  |
| u2     | √  | √  | √  | √  |
| u3     | √  | √  | √  | √  |



- 基本块内的数据流方程:
- 基本块的传递函数仍然是生成-杀死形式, 但是从OUT值计算出IN值 (逆向)
- $IN[B] = f_B(OUT[B])$
- $f_B(x) = use_B \cup (x - def_B)$ 
  - $use_B$ , 在基本块B中引用, 但是引用前在B中没有被定值的集合; 可能在B中先于定值被使用 (GEN)
  - $def_B$ , 在基本块中定值, 但是定值前在B中没有被引用的变量的集合; 在B中一定先于使用被定值 (KILL)



- USEB和DEFB的用法:
- 语句的传递函数
  - $s: x = y + z$
  - $use_s = \{y, z\}$
  - $def_s = \{x\} - \{y, z\}$  //  $x = y + z$  是模板,  $y, z$  可能和  $x$  相同
- 假设基本块中包含语句  $s_1, s_2, \dots, s_n$ , 那么
- $use_B = use_1 \cup (use_2 - def_1) \cup (use_3 - def_1 - def_2) \cup (use_n - def_1 - def_2 \dots - def_{n-1})$
- $def_B = def_1 \cup def_2 \cup \dots \cup def_n$



输入: 流图G, 各个基本块B的use和def  
输出:  $IN[B]$  和  $OUT[B]$

```

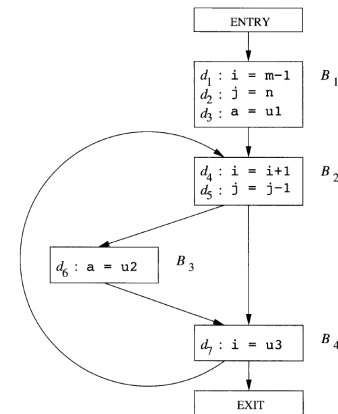
IN[EXIT] = ∅;
for (除 EXIT 之外的每个基本块 B) IN[B] = ∅;
while (某个 IN 值发生了改变)
 for (除 EXIT 之外的每个基本块 B) {
 OUT[B] = ∪S 是 B 的一个后继 IN[S];
 IN[B] = use_B ∪ (OUT[B] - def_B);
 }

```



- 活跃变量分析的迭代方法

$\triangleright use_{B_1} = \{m, n, u1\}$   
 $\triangleright def_{B_1} = \{i, j, a\}$   
 $\triangleright use_{B_2} = \{i, j\}$   
 $\triangleright def_{B_2} = \Phi$   
 $\triangleright use_{B_3} = \{u2\}$   
 $\triangleright def_{B_3} = \{a\}$   
 $\triangleright use_{B_4} = \{u3\}$   
 $\triangleright def_{B_4} = \{i\}$



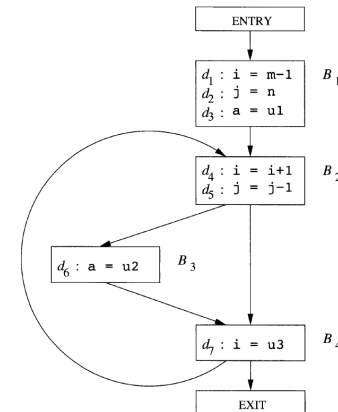
- 活跃变量数据流方程:
- 任何变量在程序出口处不再活跃
  - $IN[EXIT] = \text{空集}$
- 对于所有不等于EXIT的基本块
  - $IN[B] = use_B \cup (OUT[B] - def_B)$
  - $OUT[B] = \cup_{S \text{ 是 } B \text{ 的后继基本块}} IN[S]$
- 和到达定值相比较
  - 都使用并集运算  $\cup$  作为交汇运算
  - 数据流值的传递方向相反: 因此初始化的值不一样



- 活跃变量数据流方程求解例子

|       | $OUT[B]^1$   | $IN[B]^1$          | $OUT[B]^2$   | $IN[B]^2$          | $OUT[B]^3$   | $IN[B]^3$          |
|-------|--------------|--------------------|--------------|--------------------|--------------|--------------------|
| $B_4$ |              | $u3$               | $ij, u2, u3$ | $j, u2, u3$        | $ij, u2, u3$ | $j, u2, u3$        |
| $B_3$ | $u3$         | $u2, u3$           | $j, u2, u3$  | $j, u2, u3$        | $j, u2, u3$  | $j, u2, u3$        |
| $B_2$ | $u2, u3$     | $ij, u2, u3$       | $j, u2, u3$  | $ij, u2, u3$       | $j, u2, u3$  | $ij, u2, u3$       |
| $B_1$ | $ij, u2, u3$ | $m, n, u1, u2, u3$ | $ij, u2, u3$ | $m, n, u1, u2, u3$ | $ij, u2, u3$ | $m, n, u1, u2, u3$ |

$\triangleright use_{B_1} = \{m, n, u1\}$   
 $\triangleright def_{B_1} = \{i, j, a\}$   
 $\triangleright use_{B_2} = \{i, j\}$   
 $\triangleright def_{B_2} = \Phi$   
 $\triangleright use_{B_3} = \{u2\}$   
 $\triangleright def_{B_3} = \{a\}$   
 $\triangleright use_{B_4} = \{u3\}$   
 $\triangleright def_{B_4} = \{i\}$



4 \* i is **NOT** available before B<sub>2</sub>



- 可用表达式传递函数
  - 对于可用表达式数据流模式而言，
  - 如果基本块B对x或者y进行了（或可能进行）定值，且以后没有重新计算x+y，则称B杀死表达式x+y。
  - 如果基本块B对x+y进行计算，并且之后没有重新定值x或y，则称B生成表达式x+y
- $f_B(x) = e\_gen_B \cup (x - e\_kill_B)$ 
  - $e\_gen_B$ ：基本块B所生成的可用表达式的集合
  - $e\_kill_B$ ：基本块B所杀死的U中的表达式的集合；U表示所有出现在程序中一个或多个语句的右部的表达式的全集



- 计算基本块生成的表达式：
- 初始化  $S = \{ \}$
- 从头到尾逐个处理基本块中的指令  $x = y + z$ 
  - 把y+z添加到S中；
  - 从S中删除任何涉及变量x的表达式
- 遍历结束时得到基本块生成的表达式集合：
- 杀死的表达式集合
  - 表达式的某个分量在基本块中定值，且没有被再次生成



- 基本块生成/杀死的表达式的例子

| 语 句         | 可用表达式       |
|-------------|-------------|
|             | $\emptyset$ |
| $a = b + c$ | $\{b + c\}$ |
| $b = a - d$ | $\{a - d\}$ |
| $c = b + c$ | $\{a - d\}$ |
| $d = a - d$ | $\emptyset$ |



- 可用表达式的数据流方程

- ENTRY结点的出口处没有可用表

- $OUT[ENTRY] = \{ \}$

- 其他基本块的方程

- $OUT[B] = e\_gen_B \cup (IN[B] - e\_kill_B)$
- $IN[B] = \bigcap_{P \text{ 是 } B \text{ 的前驱基本块}} OUT[P]$

- 和其他方程类比

- 前向传播
- 交汇运算是交集运算

| 语 句         | 可用表达式       |
|-------------|-------------|
|             | $\emptyset$ |
| $a = b + c$ | $\{b + c\}$ |
| $b = a - d$ | $\{a - d\}$ |
| $c = b + c$ | $\{a - d\}$ |
| $d = a - d$ | $\emptyset$ |



- 可用表达式分析的迭代方法
- 注意：OUT值的初始化值是全集
  - 这样的初始集合可以求得更有用的解

```

OUT[ENTRY] = U;
for (除 ENTRY 之外的每个基本块 B) OUT[B] = U;
while (某个 OUT 值发生了改变)
 for (除 ENTRY 之外的每个基本块 B) {
 IN[B] = $\bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$;
 OUT[B] = $e_gen_B \cup (IN[B] - e_kill_B)$;
 }

```



|                  | 到达定值                                                             | 活跃变量                                                             | 可用表达式                                                            |
|------------------|------------------------------------------------------------------|------------------------------------------------------------------|------------------------------------------------------------------|
| 域                | Sets of definitions                                              | Sets of variables                                                | Sets of expressions                                              |
| 方向               | Forwards                                                         | Backwards                                                        | Forwards                                                         |
| 传递函数             | $gen_B \cup (x - kill_B)$                                        | $use_B \cup (x - def_B)$                                         | $e\_gen_B \cup (x - e\_kill_B)$                                  |
| 边界条件             | $OUT[ENTRY] = \emptyset$                                         | $IN[EXIT] = \emptyset$                                           | $OUT[ENTRY] = \emptyset$                                         |
| 交汇运算( $\wedge$ ) | $\cup$                                                           | $\cup$                                                           | $\cap$                                                           |
| 方程组              | $OUT[B] = f_B(IN[B])$<br>$IN[B] = \bigwedge_{P, pred(B)} OUT[P]$ | $IN[B] = f_B(OUT[B])$<br>$OUT[B] = \bigwedge_{S, succ(B)} IN[S]$ | $OUT[B] = f_B(IN[B])$<br>$IN[B] = \bigwedge_{P, pred(B)} OUT[P]$ |
| 初始值              | $OUT[B] = \emptyset$                                             | $IN[B] = \emptyset$                                              | $OUT[B] = U$                                                     |





- 循环的重要性
  - 程序的大部分执行时间都花在循环上
- 相关概念
  - 支配结点
  - 深度优先排序
  - 回边
  - 图的深度
  - 可归约性
- 两个重要问题
  - 如何寻找循环
  - 数据流分析的收敛速度



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

69



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

70



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

71



```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

72



## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

73



## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

75



## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

77



## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

74



## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

76



## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```

78

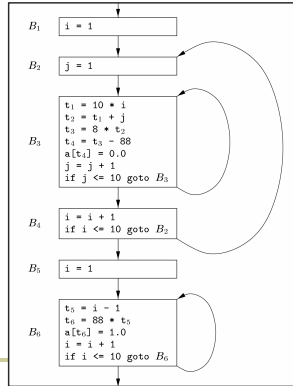


## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```



79

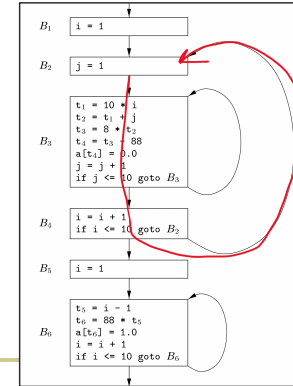


## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```



80

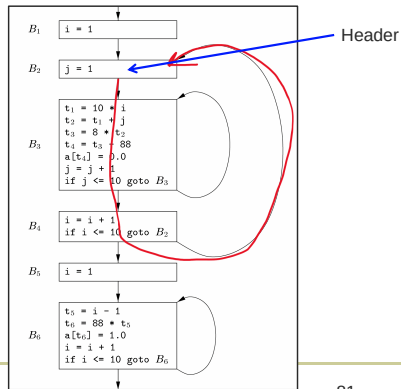


## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```



81

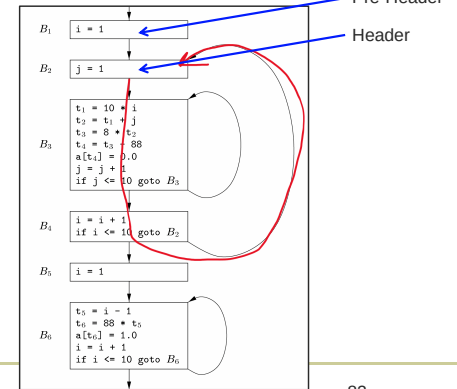


## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```



82

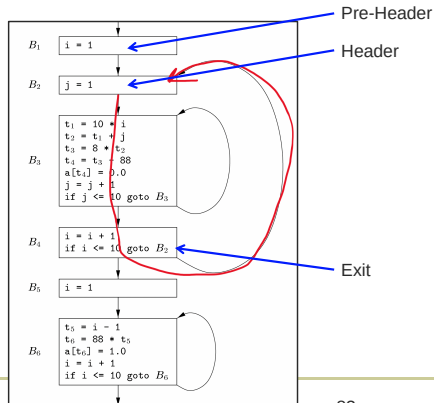


## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

```



83

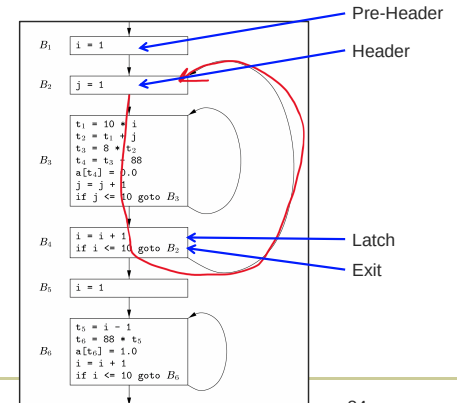


## 流程图中的循环

```

1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)

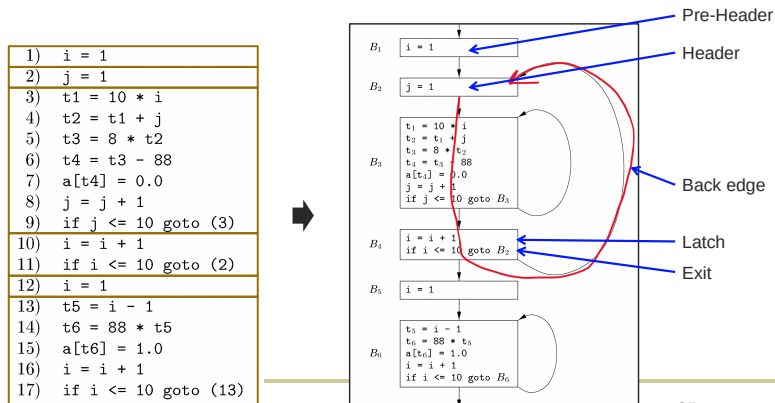
```



84



## 流程图中的循环



85



## 流程图中的循环

### ■ 自然循环:

- 1. 单个循环头 (header)
- 2. 最大强连通分量
  - 强连通分量: 一个有向图的强连通分量是一个最大的子图, 在这个子图中, 任意两个节点都是互相可达的。也就是说, 从节点A可以走到节点B, 从节点B也可以走回节点A
  - 在控制流图的上下文中: 一个循环正是一个强连通分量。因为你在循环里转圈, 可以从任何一点走到任何其他一点 (通过继续执行循环)
  - 最大的: 这个前缀意味着我们取的是包含该循环头在内的、最大的那个强连通分量。这确保了我们将整个循环体都包含进来, 而不仅仅是其中的一部分

87



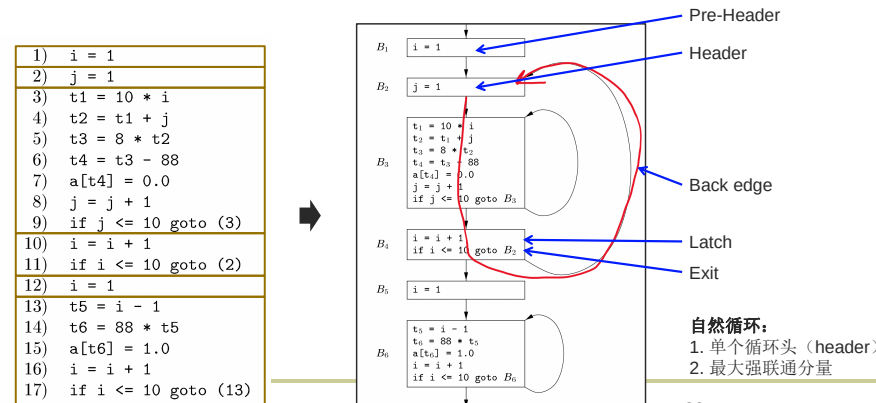
## 流程图中的循环

### ■ 支配的定义:

- A **dom** B
  - if all paths from Entry to B goes through A
- A **post-dom** B
  - if all paths from B to Exit goes through A
- **Strict** (post-)dominance – A (post-)dom B but A ≠ B
- **Immediate** dominance – A strict-dom B, but there's no C, such that A strict-dom C, C strict-dom B



## 流程图中的循环



86



## 流程图中的循环

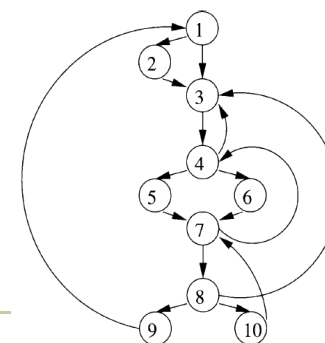
### ■ 自然循环:

- 自然循环的性质
  - 有一个唯一的入口结点 (循环头 header)。这个结点支配循环中的所有结点
  - 必然存在进入循环头的回边 (否则怎么构成循环?)
- 假设流图中存在两个结点d和n, 满足d dom n:
  - 自然循环的定义
    - 给定回边n→d的自然循环是d加上不经过d就能够到达n的结点的集合
    - d是这个循环的头



## 流程图中的循环

- 支配结点 (必经节点):
- 如果每一条从入口结点到达n的路径都经过d, 我们就说d支配 (dominate) n, 记为d dom n
- 每个节点都支配自己
- 右图:
  - 2只支配自己
  - 3支配除了1, 2之外的其它所有结点
  - 4支配1、2、3之外的其它结点
  - 5、6只支配自身
  - 7支配7, 8, 9, 10
  - 8支配8, 9, 10
  - 9, 10只支配自身

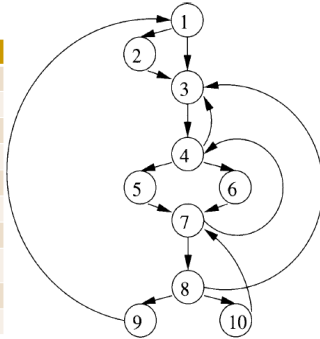




## 流图中的循环

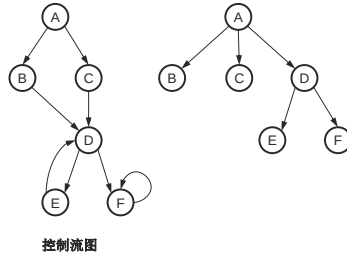
- 支配结点(必经节点):
- 如果每一条从入口结点到达n的路径都经过d, 我们就说d支配(dominate)n, 记为 $d \text{ dom } n$
- 每个节点都支配自己

| 支配节点 | 支配对象 |
|------|------|
| 1    | 1~10 |
| 2    | 2    |
| 3    | 3~10 |
| 4    | 4~10 |
| 5    | 5    |
| 6    | 6    |
| 7    | 7~10 |
| 8    | 8~10 |
| 9    | 9    |
| 10   | 10   |



## 流图中的循环

- 支配结点树:
  - 节点: 基本块
  - 边: 直接支配关系

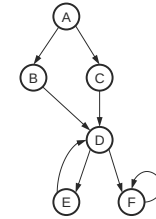


控制流图



## 流图中的循环

- 支配结点树:
  - 节点: 基本块
  - 边: 直接支配关系



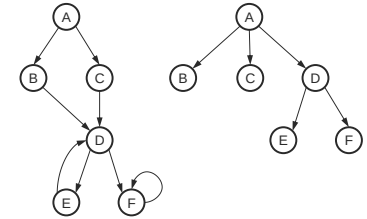
控制流图

Try!



## 流图中的循环

- 支配结点树:
  - 节点: 基本块
  - 边: 直接支配关系
- (最近) 公共支配者
  - 最低公共祖先
    - 程序的支配者树 就是一棵树, 在这棵树上, 两个节点的 LCA 正好就是它们在控制流图中的最近公共支配者
  - 近似线性的预处理时间
- 应用: 确定变量定义的位置



控制流图



## 流图中的循环

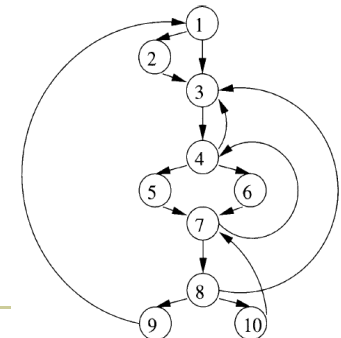
- (最近) 公共支配者
  - 最低公共祖先
    - 程序的支配者树 就是一棵树, 在这棵树上, 两个节点的 LCA 正好就是它们在控制流图中的最近公共支配者
  - 近似线性的预处理时间
- 应用: 确定变量定义的位置
  - 部分冗余消除, 目标: 消除在部分路径上重复计算的表达式
  - 静态单赋值形式的  $\Phi$  函数插入, 目标: 在构建 SSA 形式时, 正确插入  $\Phi$  函数

利用控制流图、支配者树和图论算法(如 LCA 查询), 智能地、自动化地为变量或表达式的计算结果选择一个最优的定义位置, 以达到消除冗余、优化代码的目的。



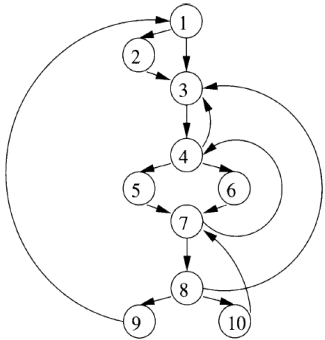
## 流图中的循环

- 支配结点树:
- 支配结点树可以表示支配关系
  - 根结点: 入口结点
  - 每个结点d支配且只支配树中的后代结点
- 直接支配结点
  - 从入口结点到达n的任何路径(不含n)中, 它是路径中最后一个支配n的结点
  - 前面的例子: 1直接支配3, 3直接支配4
  - n的直接支配结点m具有如下性质: 如果 $d \text{ dom } n$ , 那么 $d \text{ dom } m$

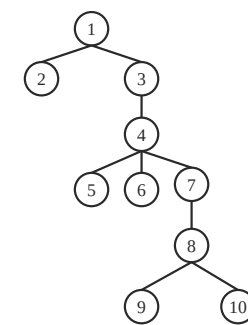
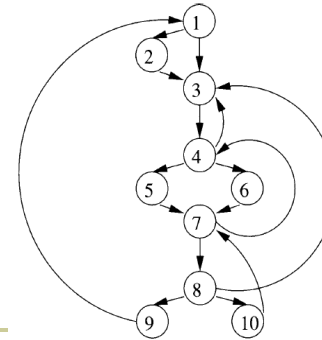




■ 支配结点树:



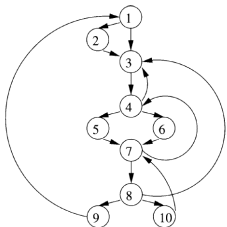
■ 支配结点树:



■ 寻找支配结点:

■ 输入: 流图G

■ 输出: 对于N中的各个节点n, 给出D(n), 即支配n的所有结点的集合.

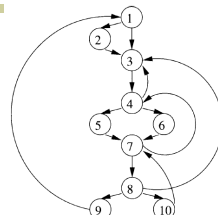


|                  | 支配结点                                                                |
|------------------|---------------------------------------------------------------------|
| 域                | The power set of $N$                                                |
| 方向               | Forwards                                                            |
| 传递函数             | $f_B(x) = x \cup \{B\}$                                             |
| 边界条件             | $OUT[ENTRY] = \{ENTRY\}$                                            |
| 交汇运算( $\wedge$ ) | $\cap$                                                              |
| 方程式              | $OUT[B] = f_B(IN[B])$<br>$IN[B] = \bigwedge_{P \in pred(B)} OUT[P]$ |
| 初始化设置            | $OUT[B] = N$                                                        |

一个计算支配结点的数据流算法



|    | OUT0 | IN1           | OUT1             |
|----|------|---------------|------------------|
| E  | E    |               |                  |
| 1  | N    | {E}           | {E,1}            |
| 2  | N    | {E,1}         | {E,1,2}          |
| 3  | N    | {E,1}         | {E,1,3}          |
| 4  | N    | {E,1,3}       | {E,1,3,4}        |
| 5  | N    | {E,1,3,4}     | {E,1,3,4,5}      |
| 6  | N    | {E,1,3,4}     | {E,1,3,4,6}      |
| 7  | N    | {E,1,3,4}     | {E,1,3,4,7}      |
| 8  | N    | {E,1,3,4,7}   | {E,1,3,4,7,8}    |
| 9  | N    | {E,1,3,4,7,8} | {E,1,3,4,7,8,9}  |
| 10 | N    | {E,1,3,4,7,8} | {E,1,3,4,7,8,10} |



```

1) OUT[ENTRY] = v_ENTRY;
2) for (除 ENTRY 之外的每个基本块 B) OUT[B] = T;
3) while (某个 OUT 值发生了改变)
4) for (除 ENTRY 之外的每个基本块 B) {
5) IN[B] = \bigwedge_{P \in pred(B)} OUT[P];
6) OUT[B] = f_B(IN[B]);
 }

```



■ 流图的边的分类

■ 前进边

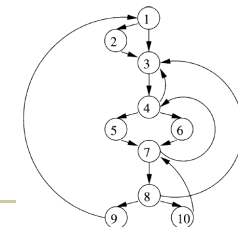
- 从结点m到达m在DFST树中的一个真后代的边
- DFST中的所有边都是前进边

■ 后退边

- 从m到达m在DFST树中的某个祖先的边

■ 交叉边

- 边的src和dest都不是对方的祖先
- 一个结点的子结点按照它们被加入到树中的顺序从左到右排列, 那么所有的交叉边都是从右到左的







## 流图中的循环

- 回边和可归约性:
- 回边
  - 边 $a \rightarrow b$ , 头 $b$ 支配了尾 $a$
  - 每条回边都是后退边, 但不是所有后退边都是回边
- 如果一个流图的任何优先生成树中的所有后退边都是回边, 那么该流图就是可归约的
  - 可归约流图的DFST的后退边集合就是回边集合
  - 不可归约流图的DFST中可能有一些后退边不是回边, 但是所有的回边仍然都是后退边
- 实践中出现的流图基本都是可归约的

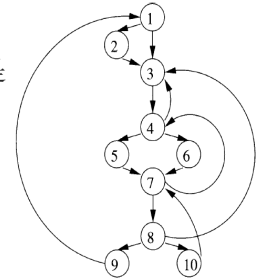


## 流图中的循环

- 回边和可归约性:
  - 回边
    - 边 $a \rightarrow b$ , 头 $b$ 支配了尾 $a$
- 但并不是所有后退边都是回边

每个回边都是后退边

|    | OUT0 | IN1           | OUT1             |
|----|------|---------------|------------------|
| E  | E    |               |                  |
| 1  | N    | {E}           | {E,1}            |
| 2  | N    | {E,1}         | {E,1,2}          |
| 3  | N    | {E,1}         | {E,1,3}          |
| 4  | N    | {E,1,3}       | {E,1,3,4}        |
| 5  | N    | {E,1,3,4}     | {E,1,3,4,5}      |
| 6  | N    | {E,1,3,4}     | {E,1,3,4,6}      |
| 7  | N    | {E,1,3,4}     | {E,1,3,4,7}      |
| 8  | N    | {E,1,3,4,7}   | {E,1,3,4,7,8}    |
| 9  | N    | {E,1,3,4,7,8} | {E,1,3,4,7,8,9}  |
| 10 | N    | {E,1,3,4,7,8} | {E,1,3,4,7,8,10} |



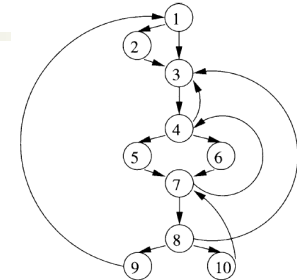
## 流图中的循环

- 自然循环构造算法:
- 输入: 流图 $G$ 和回边 $n \rightarrow d$ ;
- 输出: 回边 $n \rightarrow d$ 的自然循环中的所有结点的集合 $loop$ ;
- 方法
  - $loop = \{n, d\}$ ,  $d$ 标记为visited
  - 从 $n$ 开始, 逆向地对数据流图进行深度优先搜索, 把所有访问到的结点都加入 $loop$ , 加入 $loop$ 的结点都标记为visited。搜索过程中, 不越过标记为visited的结点



## 流图中的循环

- 自然循环示例:
- 回边:  $10 \rightarrow 7$ 
  - $\{7, 8, 10\}$
- 回边:  $7 \rightarrow 4$ 
  - $\{4, 5, 6, 7, 8, 10\}$
  - 包含了前面的循环
- 回边 $4 \rightarrow 3, 8 \rightarrow 3$ 
  - 同样的头
  - 同样的结点集合  $\{3, 4, 5, 6, 7, 8, 10\}$
- 回边 $9 \rightarrow 1$ 
  - 整个流图



| 回边      | 自然循环           |
|---------|----------------|
| 4 -> 3  | 3,4,5,6,7,8,10 |
| 7 -> 4  | 4,5,6,7,8,10   |
| 8 -> 3  | 3,4,5,6,7,8,10 |
| 9 -> 1  | 1-10           |
| 10 -> 7 | 7,8,10         |



## 流图中的循环

- 自然循环的性质:
  - 除非两个循环具有同样的循环头, 他们
    - 要么互不相交 (分离的)
    - 要么一个嵌套于另一个中
  - 最内层循环
    - 不包含其它循环的循环
    - 通常是最需要进行优化的地方



1. (7 分) 请将图中给定的三地址码格式的程序划分为程序基本块。建议采用课程中使用的方框中填写代码的形式划分这个流程图, 当然, 也可以采用基本块的格式为 $B_n = \{(l_1), \dots\}$ 。例如, 如果第 4 行和第 10 行三地址码在第 1 个基本块中, 那么该基本块可以表示成 $B_1 = \{(4), (10)\}$ 。然后, 以你给出的基本块为结点, 画出程序控制流程图。↵

注: 控制流图中还需要补充 Entry 和 Exit 结点, 在本题中这两个结点都是唯一的。↵

```

(1) x = input()
(2) y = x + 3
(3) z = y - 2
(4) if x < z goto (10)
(5) w = z * 2
(6) a = w + y
(7) if a < 30 goto (15)
(8) b = a - 1
(9) goto (18)
(10) c = z + 1
(11) d = c * x
(12) if d > 20 goto (7)
(13) d = d - 1
(14) if d < 5 goto (17)
(15) f = x - y
(16) if f < z * z goto (13)
(17) g = f + 1
(18) h = g * 2

```



2. (8分) 基于你在第1小题中构建的控制流图, 分别尝试使用下面的优化技术对程序进行优化, 描述优化前后的三地址码变动, 可以通过画图表示, 也可以通过上述的

$B_n = \{l_1, \dots\}$ 形式表示。  
a. 公共子表达式删除  
b. 死代码删除  
c. 循环优化

```
(1) x = input()
(2) y = x + 3
(3) z = y - 2
(4) if x < z goto (10)
(5) w = z * 2
(6) a = w + y
(7) if a < 30 goto (15)
(8) b = a - 1
(9) goto (18)
(10) c = z + 1
(11) d = c * x
(12) if d > 20 goto (7)
(13) d = d - 1
(14) if d < 5 goto (17)
(15) f = x - y
(16) if f < z*z goto (13)
(17) g = f + 1
(18) h = g * 2
```



a.公共子表达式删除: 在代码中, 第16行 条件检查中 $z * z$ 被重复计算。我们可以在第3行后添加一条新指令, 计算  $t' = z * z$ , 然后在第16行使用  $t'$  来替代  $z * z$ 。减少每次循环对  $y * y$  的重复计算。  
b.死代码删除: 第8行 (说第18行、第17行也对)  
c.循环优化: 第15行的 $f=x-y$ 在每次进入循环时都会重新计算, 如果  $x$  和  $y$  在循环中不变, 那么这个计算可以移到循环外进行。这样, 可以减少每次循环中的计算量 (说第16行的 $f=-3<z*z$ 永假也对。)

```
(1) x = input()
(2) y = x + 3
(3) z = y - 2
(4) if x < z goto (10)
(5) w = z * 2
(6) a = w + y
(7) if a < 30 goto (15)
(8) b = a - 1
(9) goto (18)
(10) c = z + 1
(11) d = c * x
(12) if d > 20 goto (7)
(13) d = d - 1
(14) if d < 5 goto (17)
(15) f = x - y
(16) if f < z*z goto (13)
(17) g = f + 1
(18) h = g * 2
```

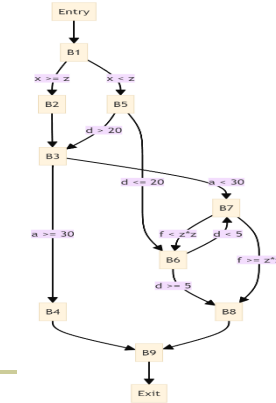


表格如下:

| 指令              | R1            | R2 | t1 | t2 | t3 | i     | x     | a[i+1]     |
|-----------------|---------------|----|----|----|----|-------|-------|------------|
| LOAD R1, i      | i             |    |    |    |    | i, R1 | x     | a[i+1]     |
| ADD R1, R1, #1  | t1            |    | R1 |    |    | i     | x     | a[i+1]     |
| MULT R2, R1, #4 | t1            | t2 | R1 | R2 |    | i     | x     | a[i+1]     |
| LOAD R1, a(R2)  | a[i+1], t3, x | t2 |    | R2 | R1 | i     | R1    | a[i+1], R1 |
| STORE x, R1     | a[i+1], t3, x | t2 |    | R2 | R1 | i     | x, R1 | a[i+1], R1 |



B1: { (1), (2), (3), (4) }  
B2: { (5), (6) }  
B3: { (7) }  
B4: { (8), (9) }  
B5: { (10), (11), (12) }  
B6: { (13), (14) }  
B7: { (15), (16) }  
B8: { (17) }  
B9: { (18) }



```
(1) x = input()
(2) y = x + 3
(3) z = y - 2
(4) if x < z goto (10)
(5) w = z * 2
(6) a = w + y
(7) if a < 30 goto (15)
(8) b = a - 1
(9) goto (18)
(10) c = z + 1
(11) d = c * x
(12) if d > 20 goto (7)
(13) d = d - 1
(14) if d < 5 goto (17)
(15) f = x - y
(16) if f < z*z goto (13)
(17) g = f + 1
(18) h = g * 2
```



假设有两个可用的寄存器 R1 和 R2, 使用简单代码生成算法,

|     |  |
|-----|--|
| 得分  |  |
| 评分人 |  |

将下面的三地址码转换为机器代码。

```
t1 = i + 1
t2 = t1 * 4
t3 = a[t2]
x = t3
```

给出每条三地址码生成机器代码后的寄存器描述符和地址描述符 (请在表格左边空白

部分写出生成的机器代码, 并在表格中补充写入对应的描述符内容。设数组的每个元素占 4 个字节, 表格长度可自行添加)。

附机器代码格式:

取数指令: LOAD R, var // 将内存空间中变量 var 的值存入寄存器 R 中。

存数指令: STORE var, R // 将寄存器 R 中的值存入变量 var 的内存空间中。

双目运算指令: OP dst, src1, src2 // 使用 OP 对寄存器 src1 和 src2 中的值进行计算, 结果存入寄存器 dst 中 (OP 包括 ADD、SUB、MULT、DIV)。

表格如下:

| 指令 | R1 | R2 | t1 | t2 | t3 | i | x | a[i+1] |
|----|----|----|----|----|----|---|---|--------|
|----|----|----|----|----|----|---|---|--------|

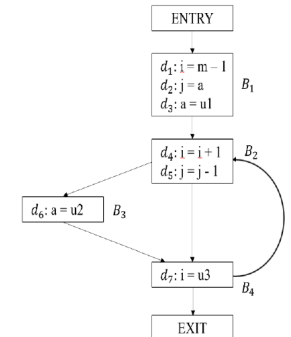
考虑右图所示流图, 完成以下题目。

1. 如果要对其进行到达值分析, 请写出各个基本块的 def 和 use 集合;

2. 请对流图进行活跃变量分析, 写出分析算法每次迭代的结果。

(提示: 注意活跃变量分析的方向)

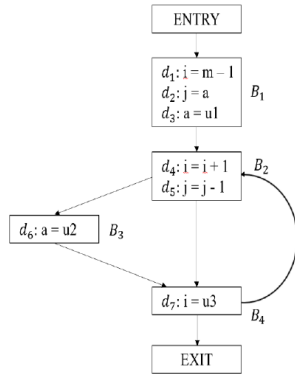
1.  $USE_{B1} = \{ \}$   
 $DEF_{B1} = \{ \}$   
 $USE_{B2} = \{ \}$   
 $DEF_{B2} = \{ \}$   
 $USE_{B3} = \{ \}$   
 $DEF_{B3} = \{ \}$   
 $USE_{B4} = \{ \}$   
 $DEF_{B4} = \{ \}$





2.

|       | $OUT[B]^i$ | $IN[B]^i$ | $OUT[B]^j$ | $IN[B]^j$ | $OUT[B]^i$ | $IN[B]^i$ |
|-------|------------|-----------|------------|-----------|------------|-----------|
| $B_4$ |            |           |            |           |            |           |
| $B_1$ |            |           |            |           |            |           |
| $B_2$ |            |           |            |           |            |           |
| $B_3$ |            |           |            |           |            |           |



## 九、简答题（共 1 题，共 10 分）

答案：

1. (5 分)

$USE_{B1} = \{m, a, u1\}$

$DEF_{B1} = \{i, j\}$

$USE_{B2} = \{i, j\}$

$DEF_{B2} = \{j\}$

$USE_{B3} = \{u2\}$

$DEF_{B3} = \{a\}$

$USE_{B4} = \{u3\}$

$DEF_{B4} = \{i\}$

2. (5 分)

|       | $OUT[B]^i$ | $IN[B]^i$    | $OUT[B]^j$ | $IN[B]^j$    | $OUT[B]^i$ | $IN[B]^i$    |
|-------|------------|--------------|------------|--------------|------------|--------------|
| $B_4$ | -          | u3           | i,j,u2,u3  | j,u2,u3      | i,j,u2,u3  | j,u2,u3      |
| $B_3$ | u3         | u2,u3        | j,u2,u3    | j,u2,u3      | j,u2,u3    | j,u2,u3      |
| $B_2$ | u2,u3      | i,j,u2,u3    | j,u2,u3    | i,j,u2,u3    | j,u2,u3    | i,j,u2,u3    |
| $B_1$ | i,j,u2,u3  | m,a,u1,u2,u3 | i,j,u2,u3  | m,a,u1,u2,u3 | i,j,u2,u3  | m,a,u1,u2,u3 |



## 十、简答题（共 1 题，共 10 分）

考虑下图所示代码片段，完成下列题目：

- 请画出代码片段中的各个基本块及其流图，基本块用矩形表示，矩形中需要包括对应的代码片段，基本块用  $B_i$  命名，即  $B_1$  表示第一个基本块；
- 求出流图中自然循环及其结点集合；
- 请对流图给出至少三条优化建议（指出哪些语句可以进行哪些优化即可，不需要画出优化后的流图）。

| 得分  |  |
|-----|--|
| 评分人 |  |

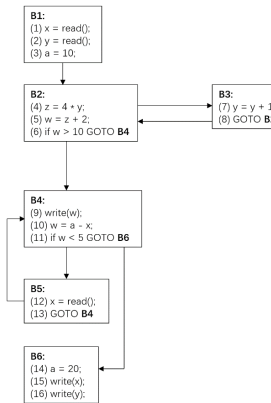
```
(1) x = read();
(2) y = 4;
(3) a = 10;
(4) z = x * y;
(5) w = z + 2;
(6) if w > 10 GOTO (9)
(7) y = y + 1;
(8) GOTO (4)
(9) write(w);
(10) w = a - x;
(11) if w < 5 GOTO (14)
(12) x = read();
(13) GOTO (9)
(14) a = 20;
(15) write(x);
(16) write(y);
```



## 十、简答题（共 1 题，共 10 分）

答案：

1. (3 分)



2. (4 分)

回边  $B3-B2$ :  $\{B2, B3\}$

回边  $B5-B4$ :  $\{B4, B5, B6\}$

3. (3 分)

常量折叠：语句 10 中 a 用 10 代替

死代码消除：语句 14

强度消减：语句 4

```
(1) x = read();
(2) y = 4;
(3) a = 10;
(4) z = x * y;
(5) w = z + 2;
(6) if w > 10 GOTO (9)
(7) y = y + 1;
(8) GOTO (4)
(9) write(w);
(10) w = a - x;
(11) if w < 5 GOTO (14)
(12) x = read();
(13) GOTO (9)
(14) a = 20;
(15) write(x);
(16) write(y);
```